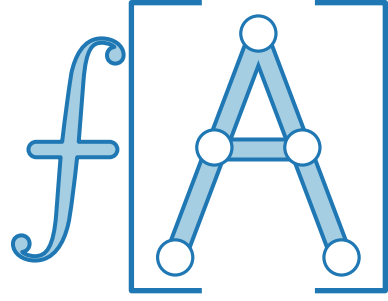
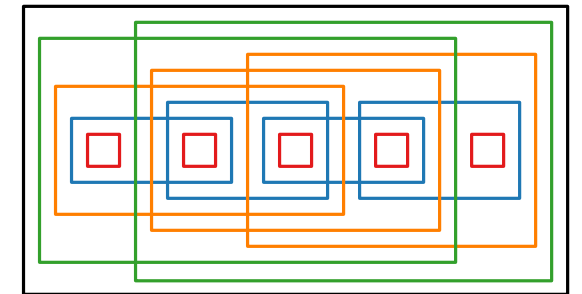
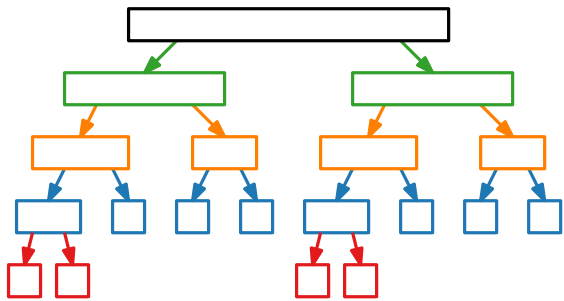




Fortgeschrittene Algorithmen

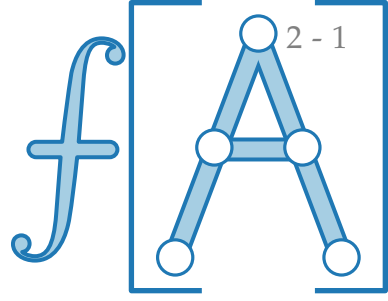
2. Vorlesung Dynamisches Programmieren

Teil I: Fahrplan



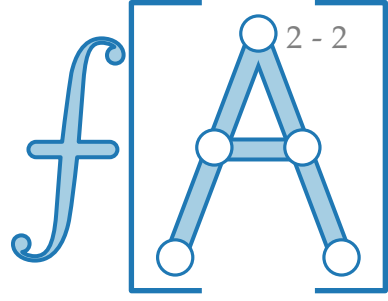
Entwurfstechniken

- Inkrementell
- Rekursiv
- Teile und Herrsche
- Greedy
- Dynamisches Programmieren
- Randomisiert
- Parallel
- [Online]



Entwurfstechniken

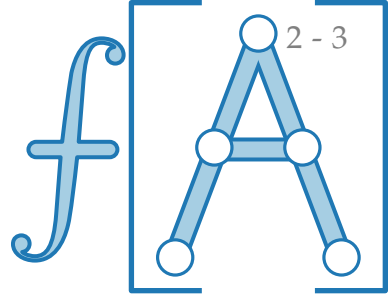
- Inkrementell
- Rekursiv
- Teile und Herrsche
- Greedy
- Dynamisches Programmieren
- Randomisiert
- Parallel
- [Online]



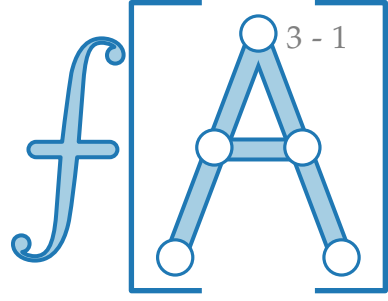
Entwurfstechniken

- Inkrementell
- Rekursiv
- Teile und Herrsche
- Greedy
- Dynamisches Programmieren
- Randomisiert
- Parallel
- [Online]

meint hier das Arbeiten mit einer Tabelle,
nicht das Schreiben eines Computerprogramms.



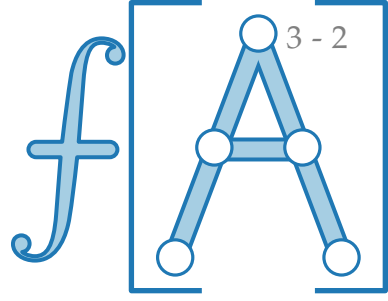
Vergleich



Teile und Herrsche

Dynamisches Programmieren

Vergleich

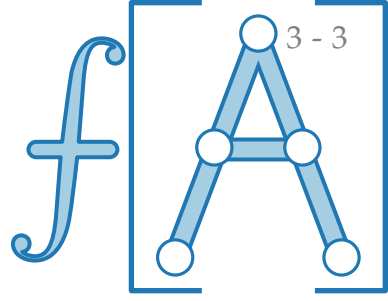


Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

Dynamisches Programmieren

Vergleich

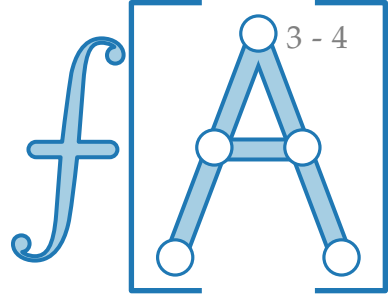
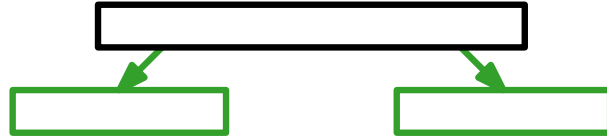


Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

Dynamisches Programmieren

Vergleich

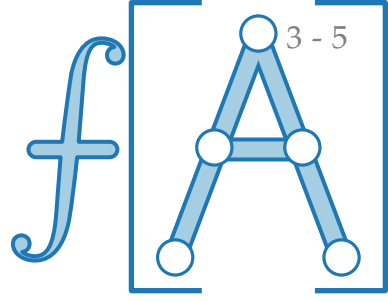
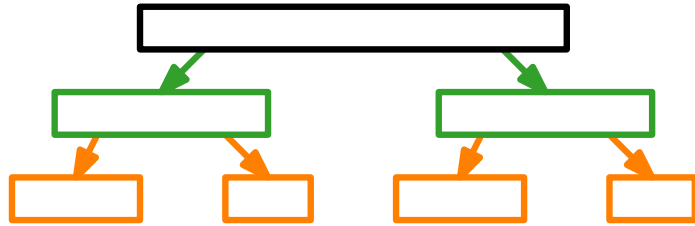


Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

Dynamisches Programmieren

Vergleich

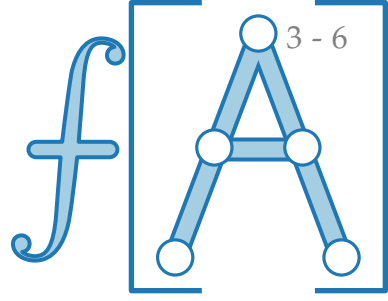
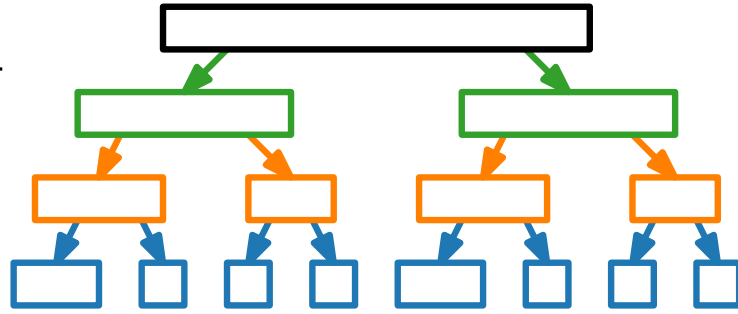


Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

Dynamisches Programmieren

Vergleich

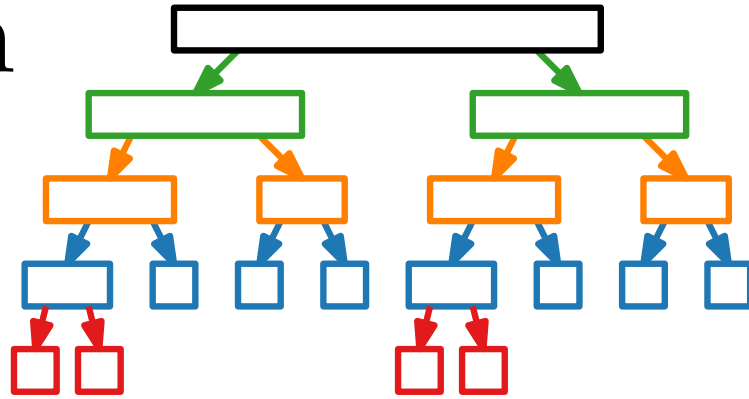


Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

Dynamisches Programmieren

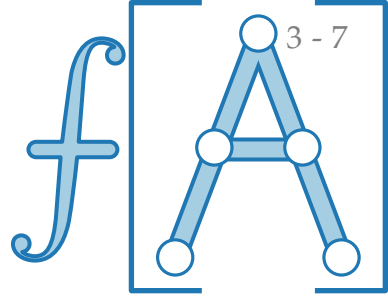
Vergleich



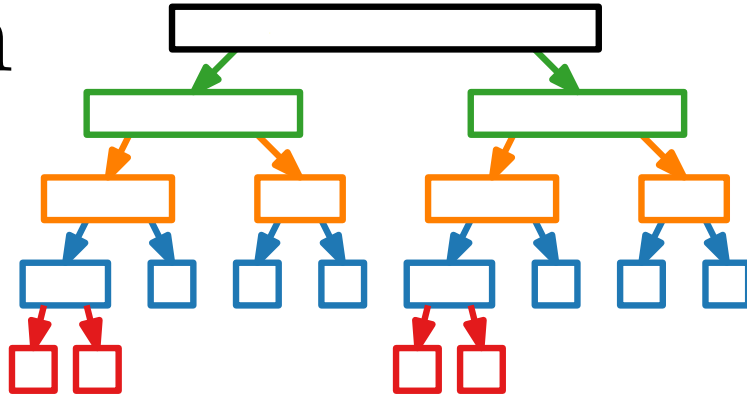
Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

Dynamisches Programmieren



Vergleich

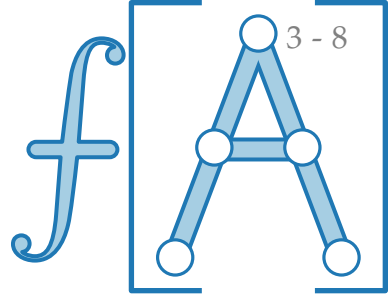


Teile und Herrsche

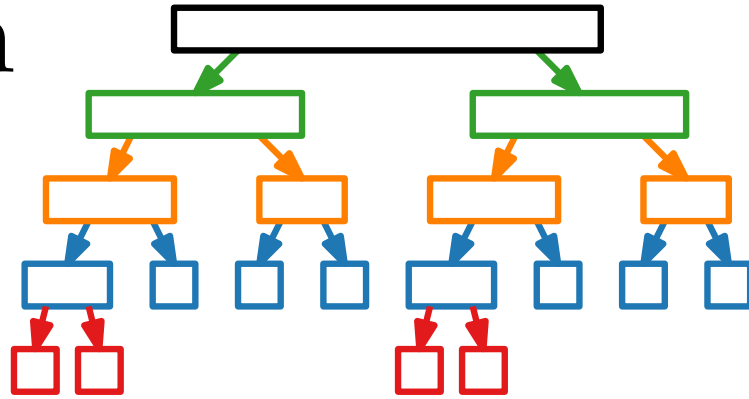
- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen



Vergleich



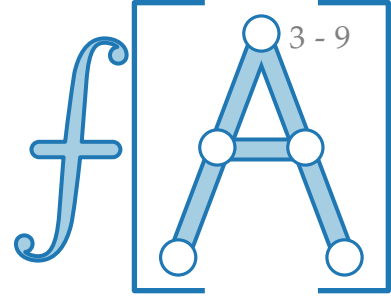
Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

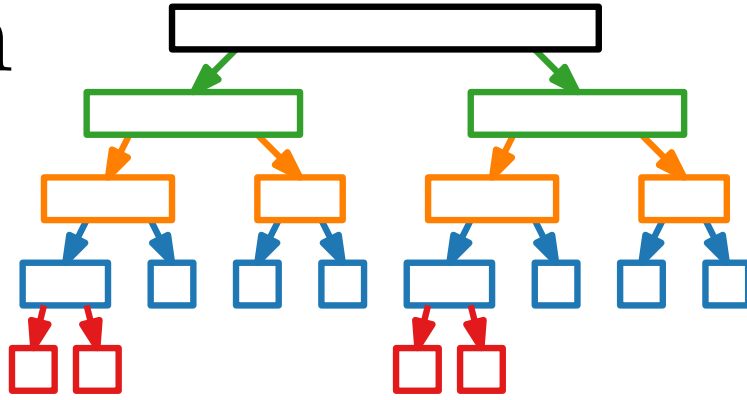


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen

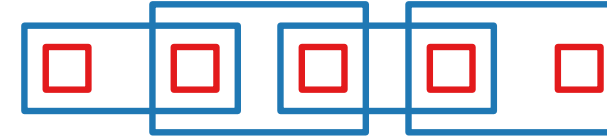


Vergleich



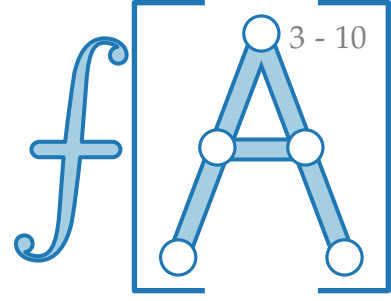
Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

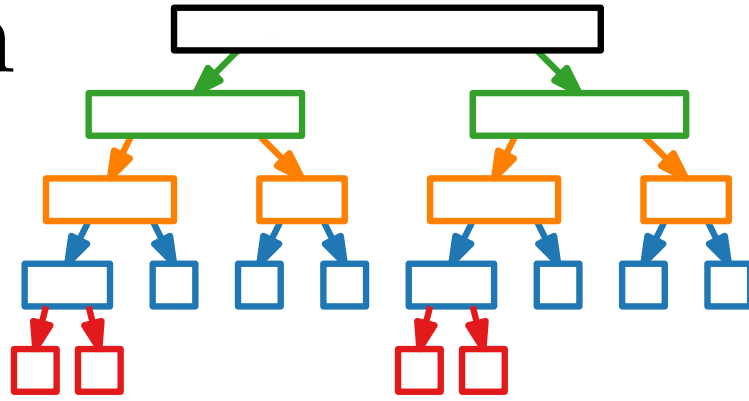


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen

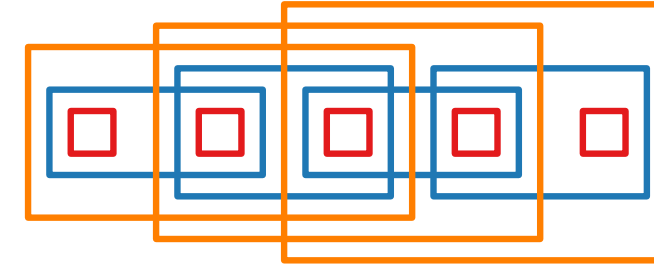


Vergleich



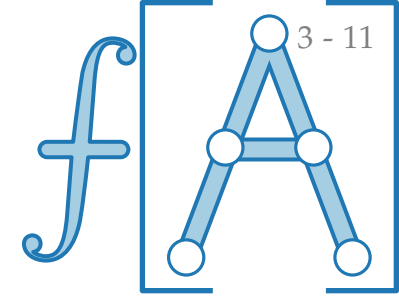
Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

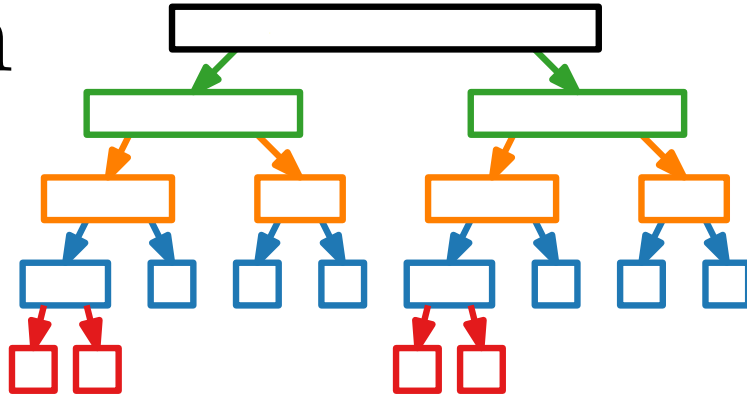


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen

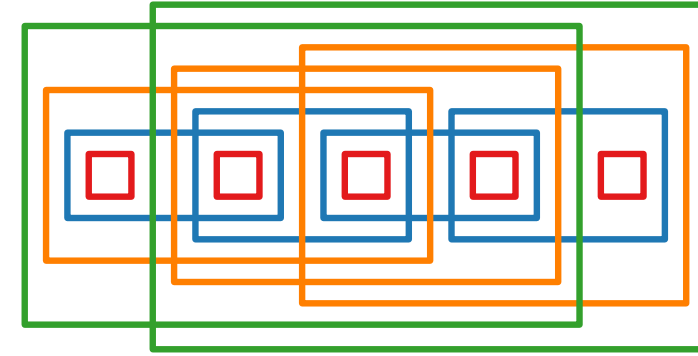


Vergleich



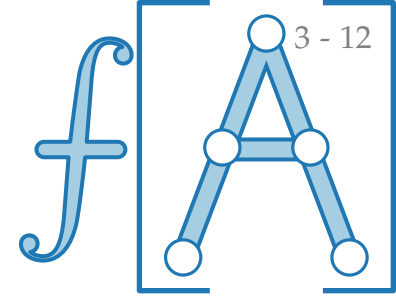
Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

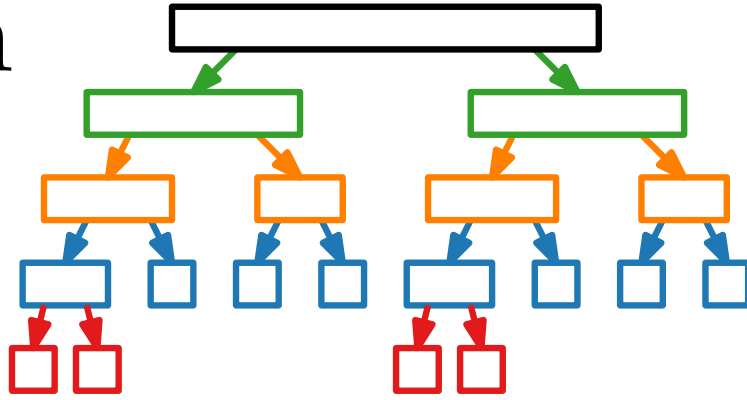


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen

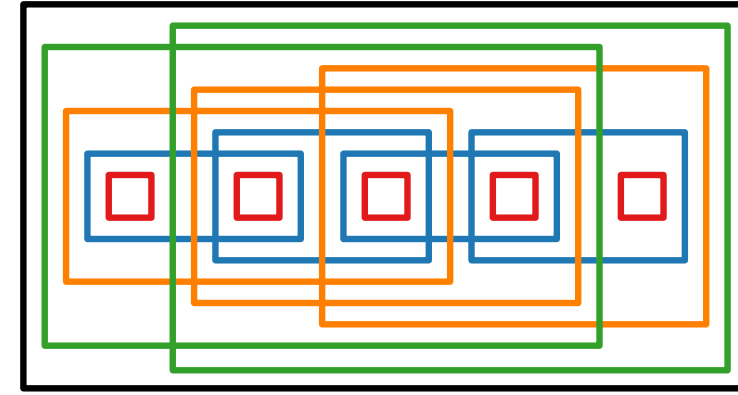


Vergleich



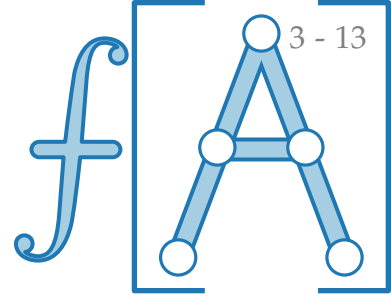
Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

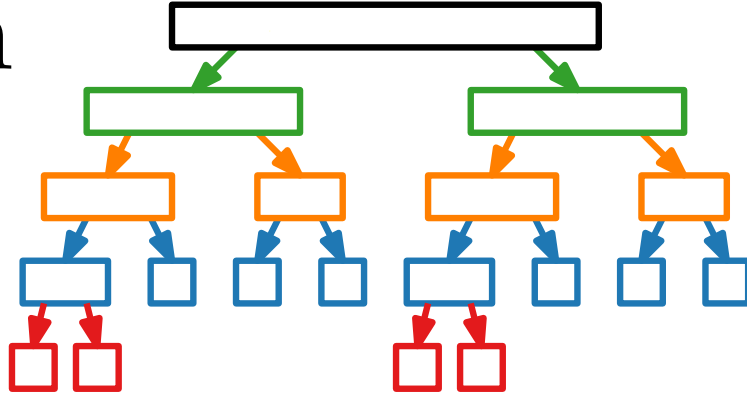


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen

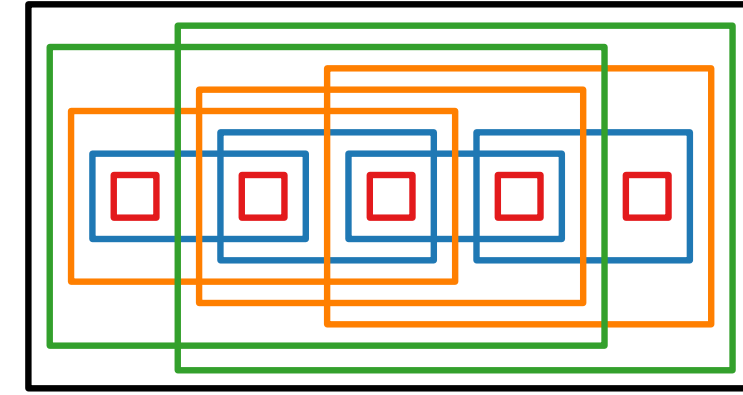


Vergleich



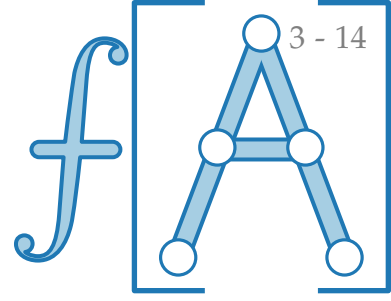
Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

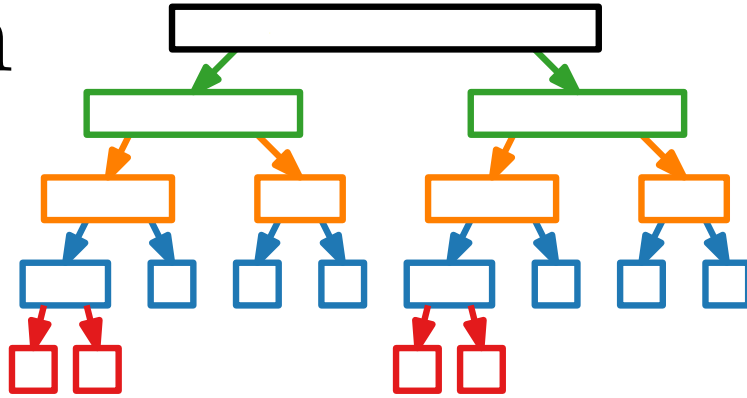


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen, d.h. Teilinstanzen haben z.T. dieselben Teildeinstanzen.

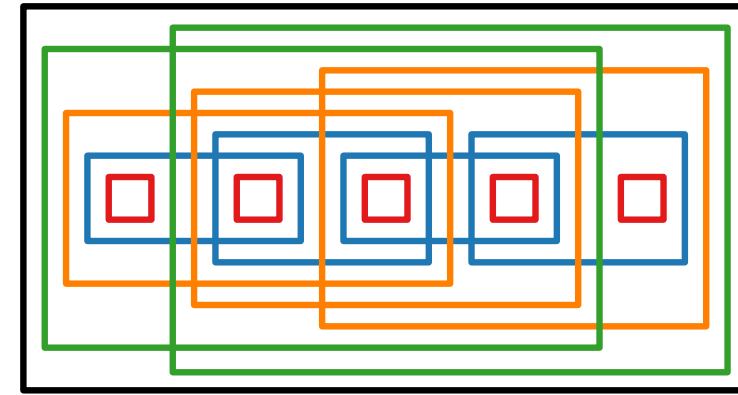


Vergleich



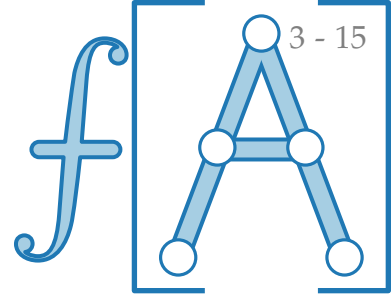
Teile und Herrsche

- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

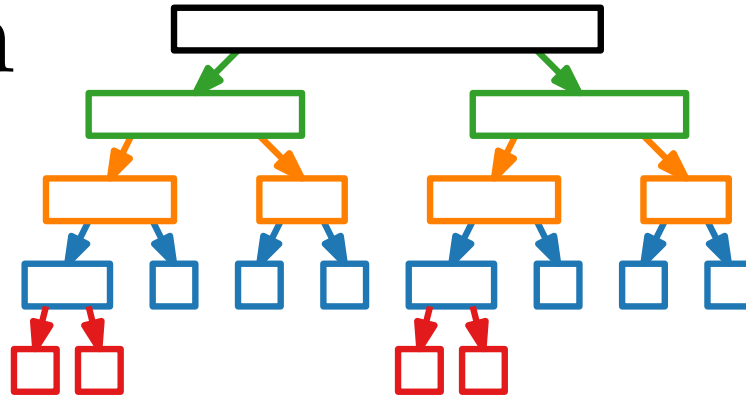


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen, d.h. Teilinstanzen haben z.T. dieselben Teilteilinstanzen. Lösungen von Teilinstanzen werden zwischengespeichert, nicht neu berechnet.



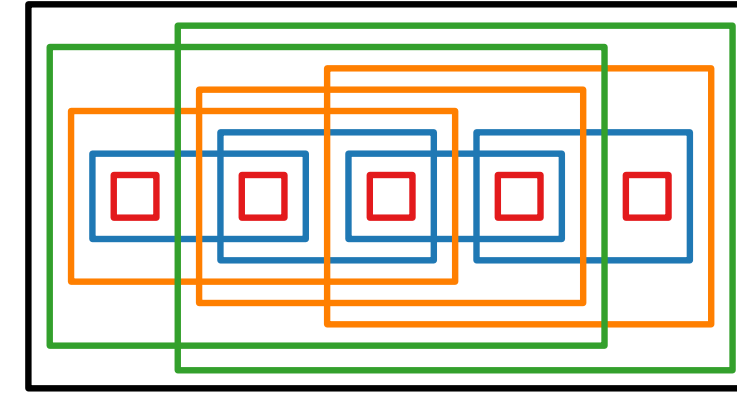
Vergleich



Teile und Herrsche

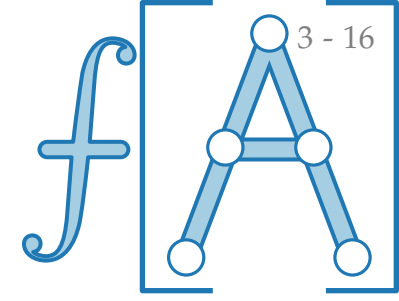
- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

- top-down

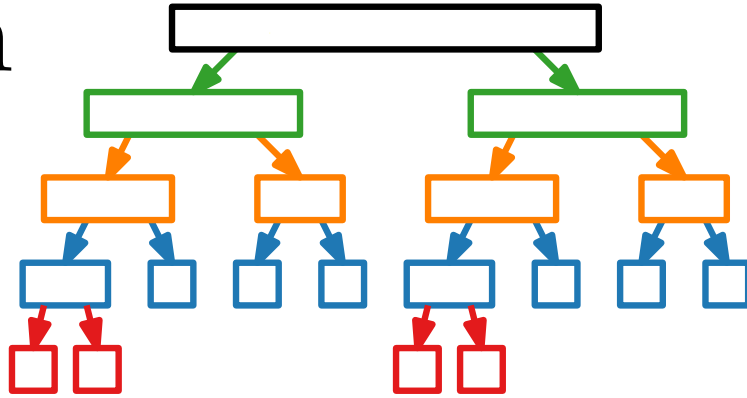


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen, d.h. Teilinstanzen haben z.T. dieselben Teildeinstanzen. Lösungen von Teilinstanzen werden zwischengespeichert, nicht neu berechnet.



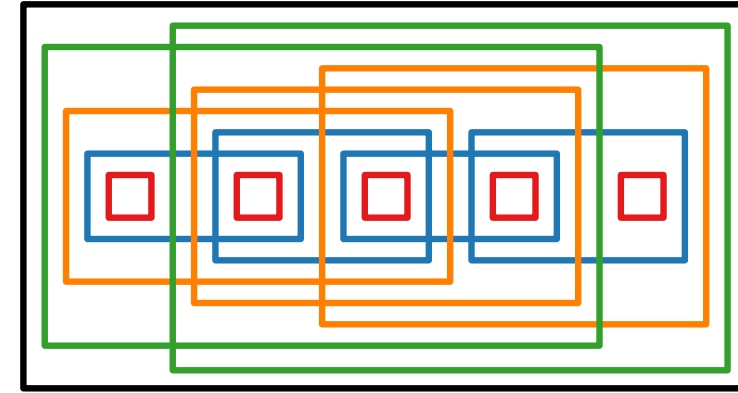
Vergleich



Teile und Herrsche

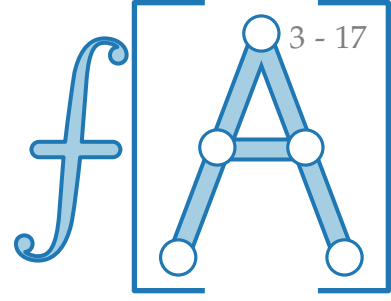
- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

- top-down

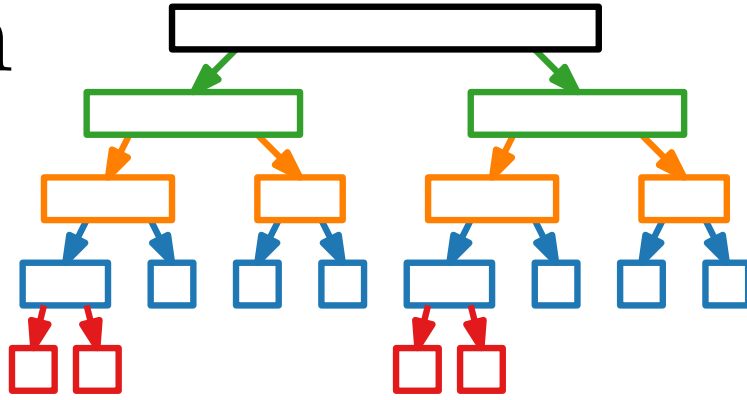


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen, d.h. Teilinstanzen haben z.T. dieselben Teilmusterteilinstanzen. Lösungen von Teilinstanzen werden zwischengespeichert, nicht neu berechnet.
- meist bottom-up



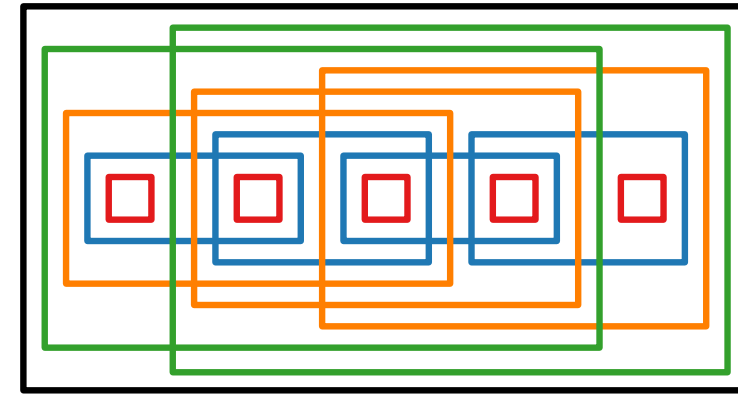
Vergleich



Teile und Herrsche

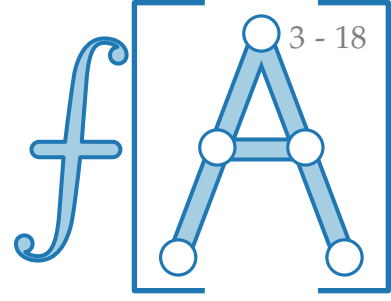
- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

- top-down
- eher für Entscheidungs- oder Berechnungsprobleme

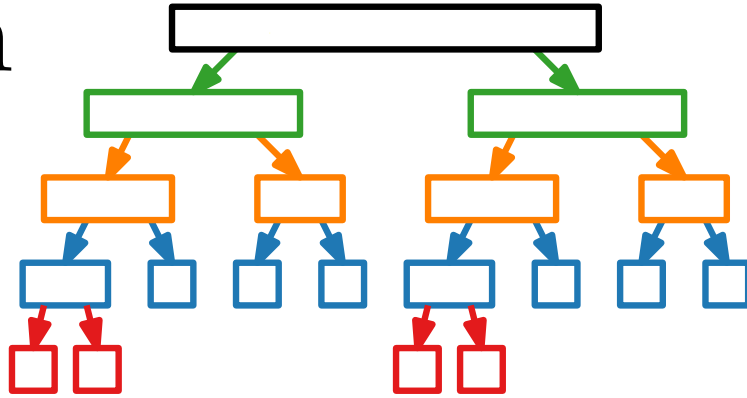


Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen, d.h. Teilinstanzen haben z.T. dieselben Teildeinstanzen. Lösungen von Teilinstanzen werden zwischengespeichert, nicht neu berechnet.
- meist bottom-up



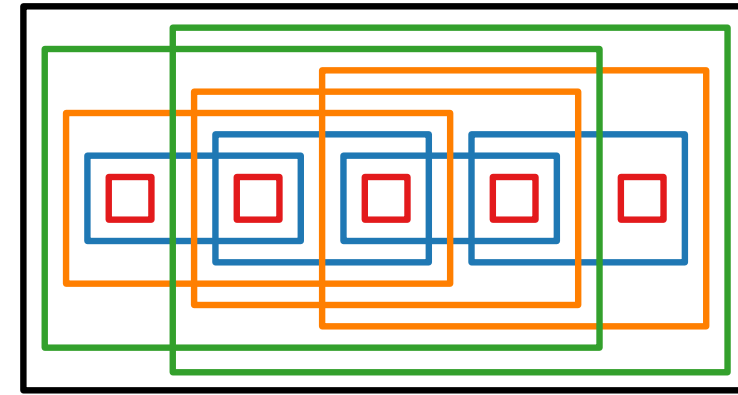
Vergleich



Teile und Herrsche

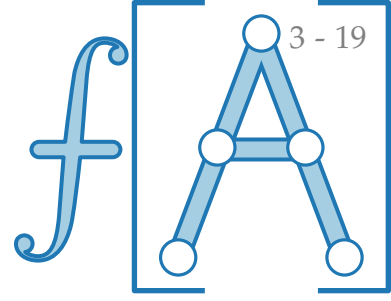
- zerlegt Instanz rekursiv in **disjunkte** Teilinstanzen

- top-down
- eher für Entscheidungs- oder Berechnungsprobleme



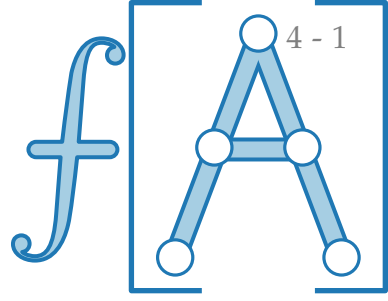
Dynamisches Programmieren

- zerlegt Instanz in **überlappende** Teilinstanzen, d.h. Teilinstanzen haben z.T. dieselben Teilteilinstanzen. Lösungen von Teilinstanzen werden zwischengespeichert, nicht neu berechnet.
- meist bottom-up
- meist für Optimierungsprobleme



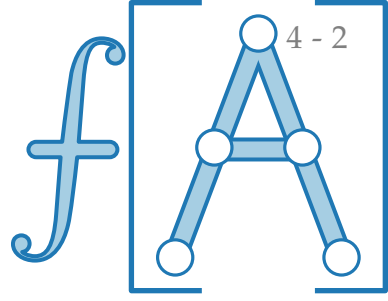
Fahrplan

1. Struktur einer optimalen Lösung charakterisieren



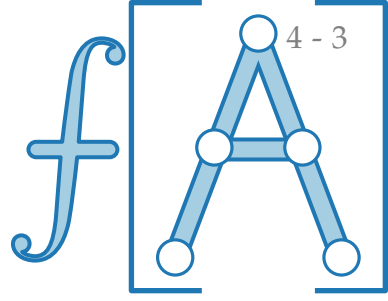
Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren



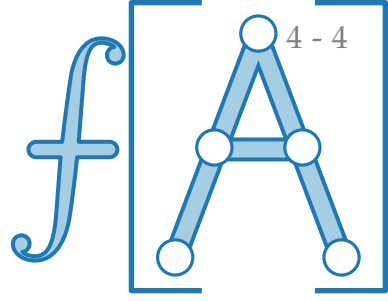
Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)



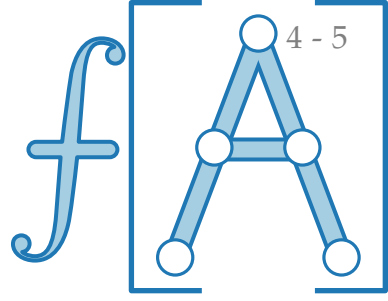
Fahrplan

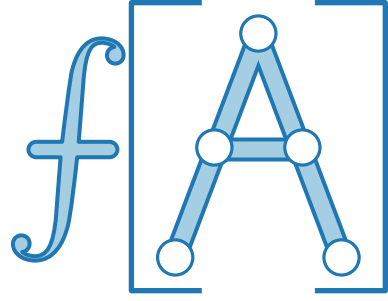
1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)
4. Optimale Lösung aus berechneter Information konstruieren



Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)
4. Optimale Lösung aus berechneter Information konstruieren

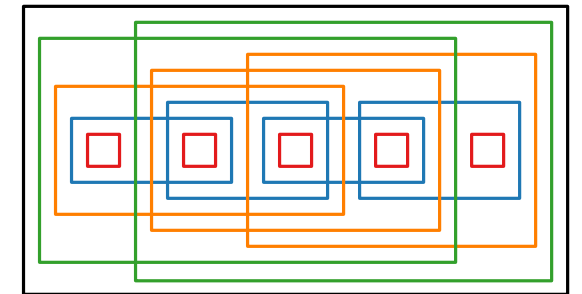
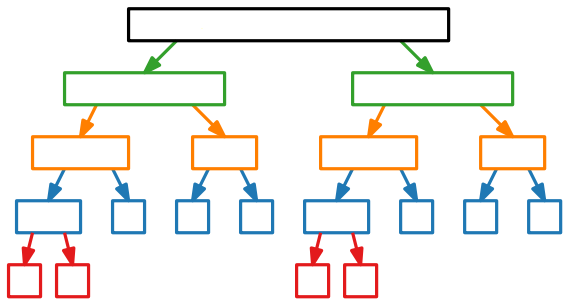




Fortgeschrittene Algorithmen

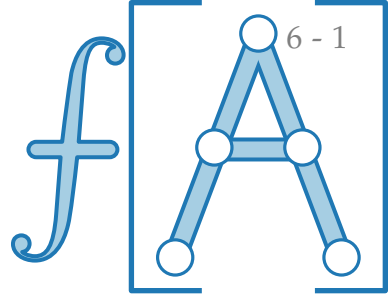
2. Vorlesung Dynamisches Programmieren

Teil II: Zerlegungsproblem



Das Zerlegungsproblem

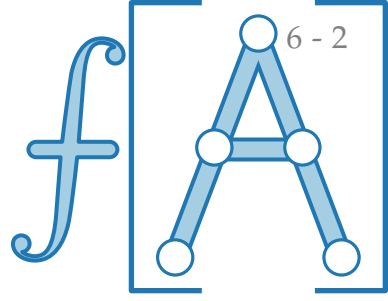
Problem. ZERLEGUNGSPROBLEM



Das Zerlegungsproblem

Problem. ZERLEGUNGSPROBLEM

Geg. Ein Stab der Länge n und die Preise
 $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

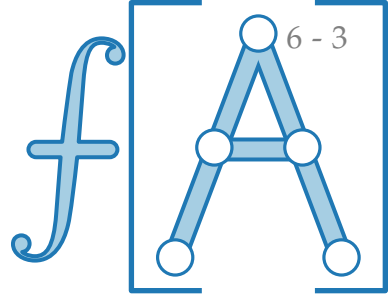


Das Zerlegungsproblem

Problem. ZERLEGUNGSPROBLEM

Geg. Ein Stab der Länge n und die Preise
 $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Ges. Zerlegung des Stabs, die den Ertrag maximiert.

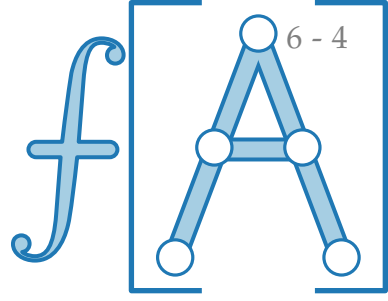


Das Zerlegungsproblem

Problem. ZERLEGUNGSPROBLEM

Geg. Ein Stab der Länge n und die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Ges. Zerlegung des Stabs, die den Ertrag maximiert.

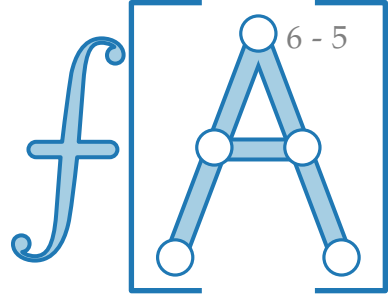


Das Zerlegungsproblem

Problem. ZERLEGUNGSPROBLEM

Geg. Ein Stab der Länge n und die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Ges. Zerlegung des Stabs, die den Ertrag maximiert.

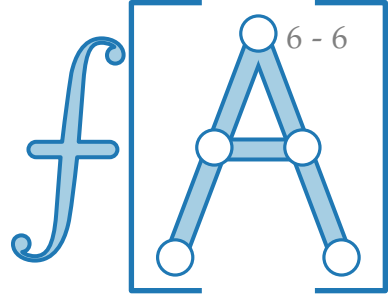


Das Zerlegungsproblem

Problem. ZERLEGUNGSPROBLEM

Geg. Ein Stab der Länge n und die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Ges. Zerlegung des Stabs, die den Ertrag maximiert.

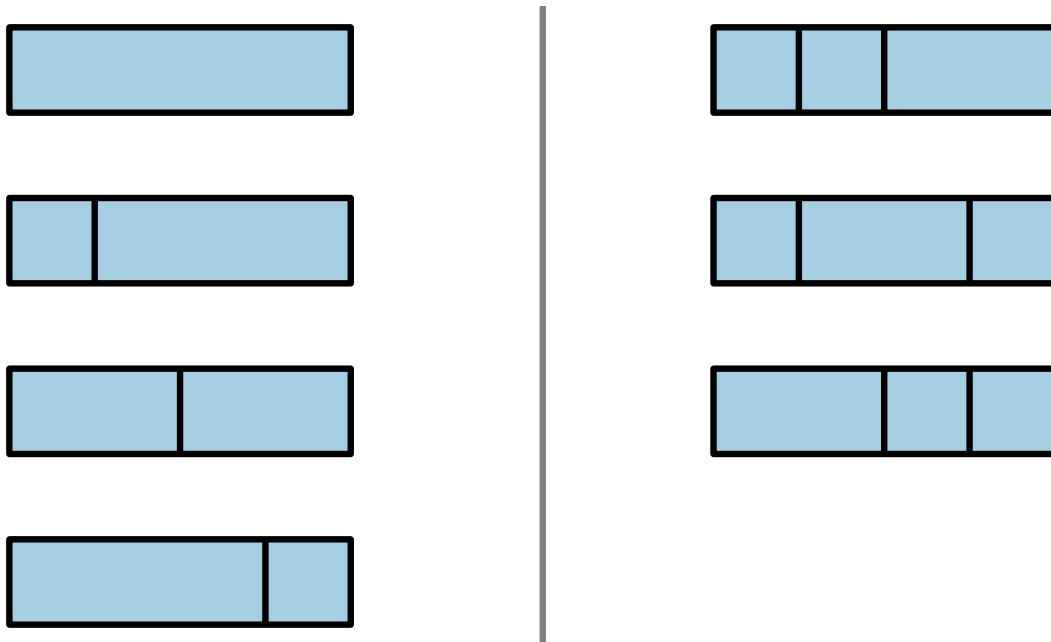
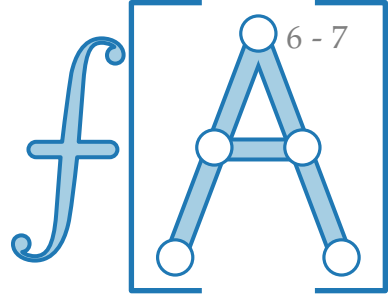


Das Zerlegungsproblem

Problem. ZERLEGUNGSPROBLEM

Geg. Ein Stab der Länge n und die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Ges. Zerlegung des Stabs, die den Ertrag maximiert.

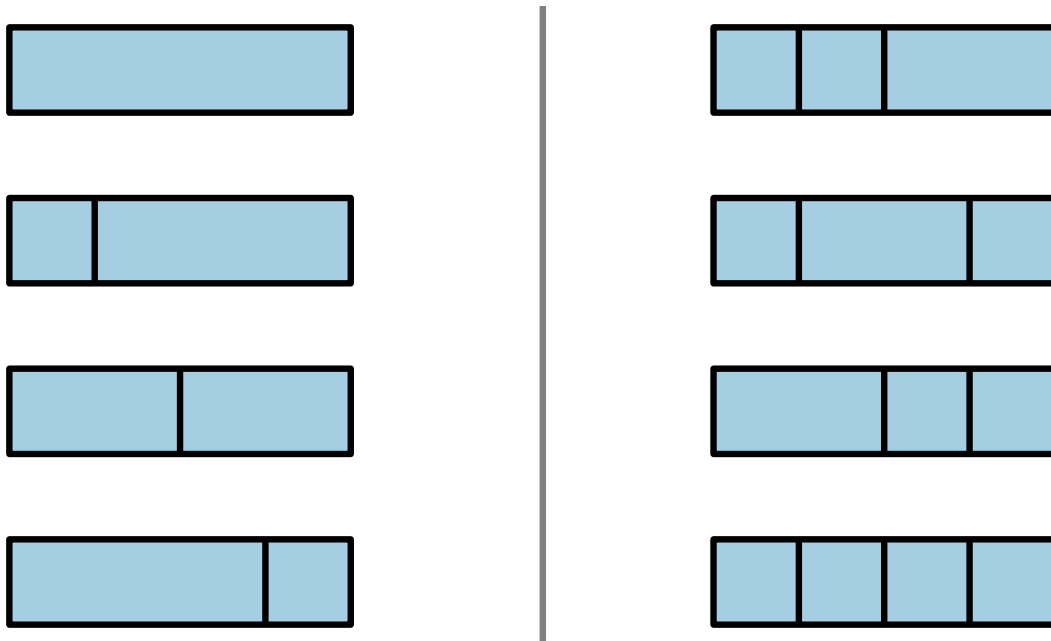
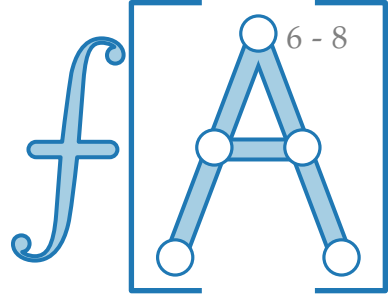


Das Zerlegungsproblem

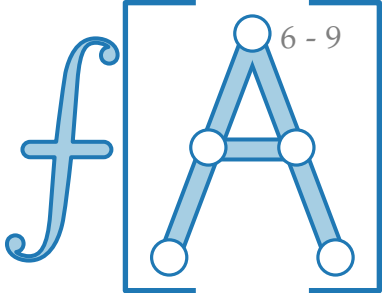
Problem. ZERLEGUNGSPROBLEM

Geg. Ein Stab der Länge n und die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Ges. Zerlegung des Stabs, die den Ertrag maximiert.



Das Zerlegungsproblem



Problem. ZERLEGUNGSPROBLEM

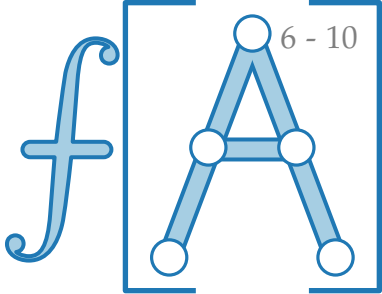
Geg. Ein Stab der Länge n und die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Ges. Zerlegung des Stabs, die den Ertrag maximiert.



Länge i	1	2	3	4
Preis p_i [in €]	1	5	8	9

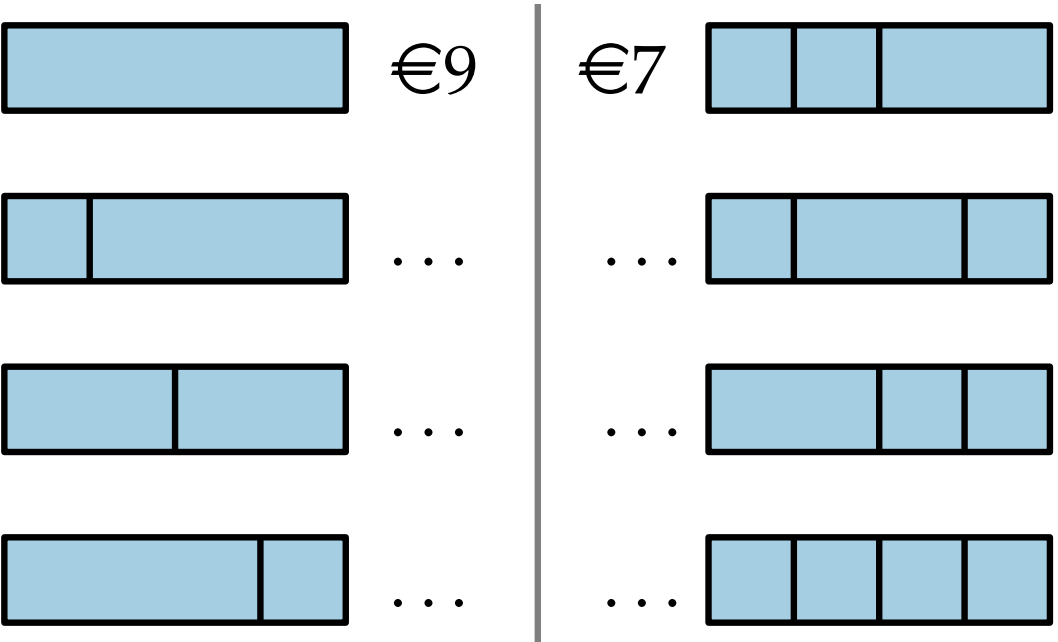
Das Zerlegungsproblem



Problem. ZERLEGUNGSPROBLEM

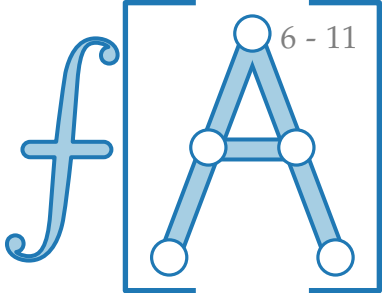
Geg. Ein Stab der Länge n und die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Ges. Zerlegung des Stabs, die den Ertrag maximiert.



Länge i	1	2	3	4
Preis p_i [in €]	1	5	8	9

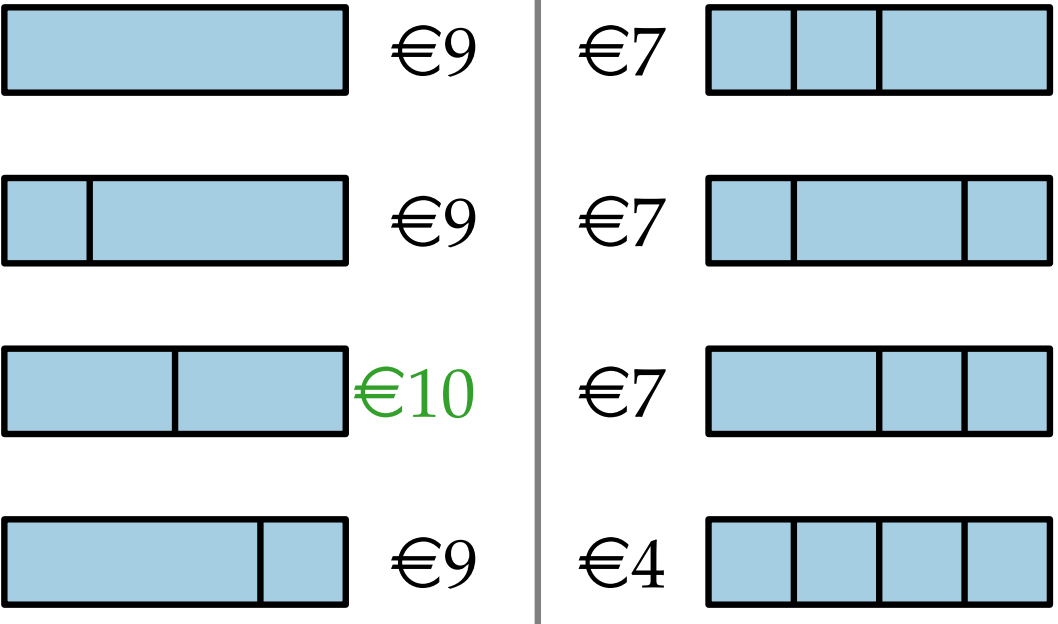
Das Zerlegungsproblem



Problem. ZERLEGUNGSPROBLEM

Geg. Ein Stab der Länge n und die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

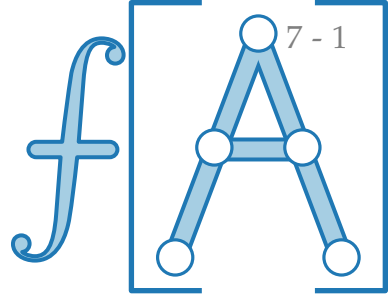
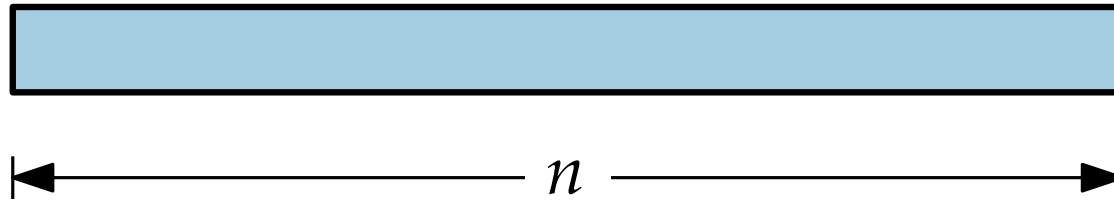
Ges. Zerlegung des Stabs, die den Ertrag maximiert.



Länge i	1	2	3	4
Preis p_i [in €]	1	5	8	9

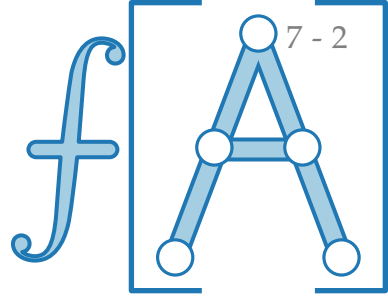
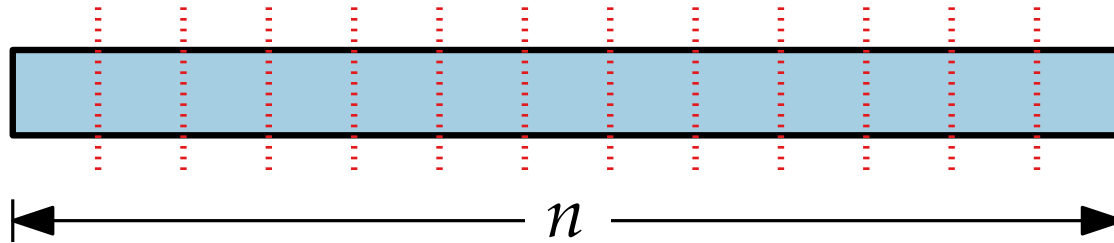
Rohe Gewalt

Frage. Wie viele Möglichkeiten gibt es, einen Stab der Länge n zu zerlegen?

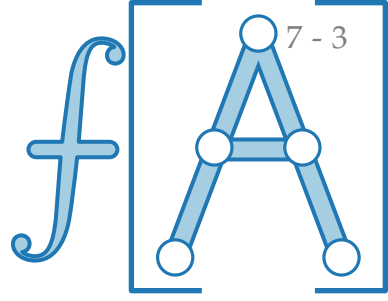


Rohe Gewalt

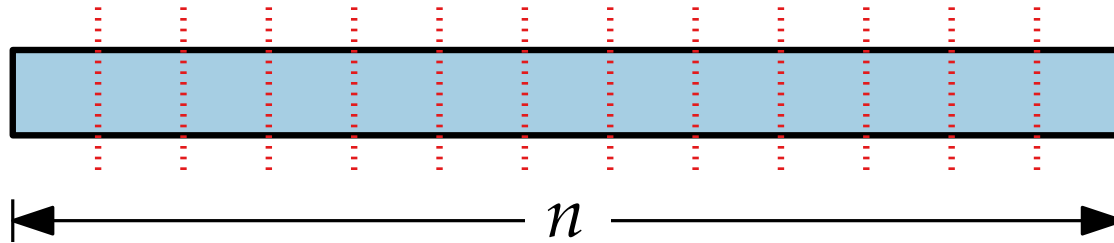
Frage. Wie viele Möglichkeiten gibt es, einen Stab der Länge n zu zerlegen?



Rohe Gewalt

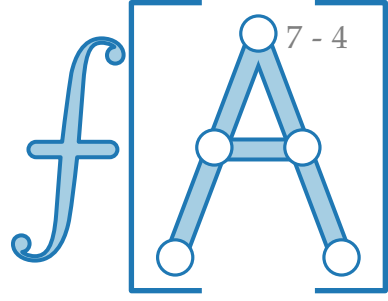


Frage. Wie viele Möglichkeiten gibt es, einen Stab der Länge n zu zerlegen?

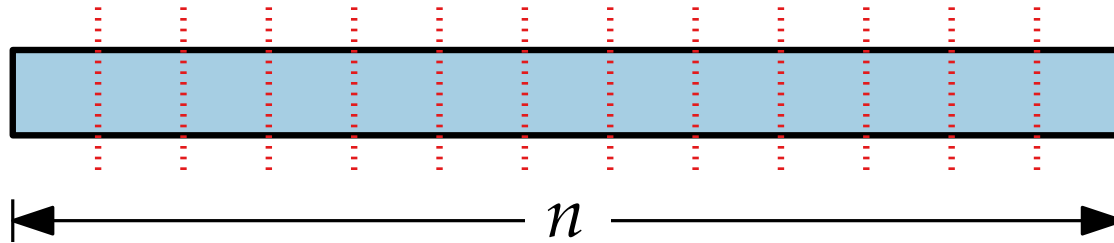


Antwort. Können $n - 1$ mal entscheiden: *schneiden* oder *nicht*.

Rohe Gewalt

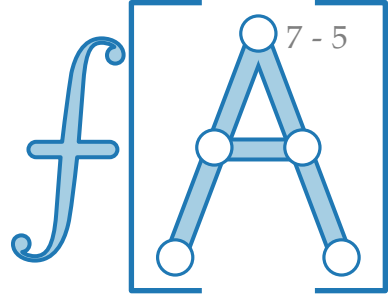


Frage. Wie viele Möglichkeiten gibt es, einen Stab der Länge n zu zerlegen?

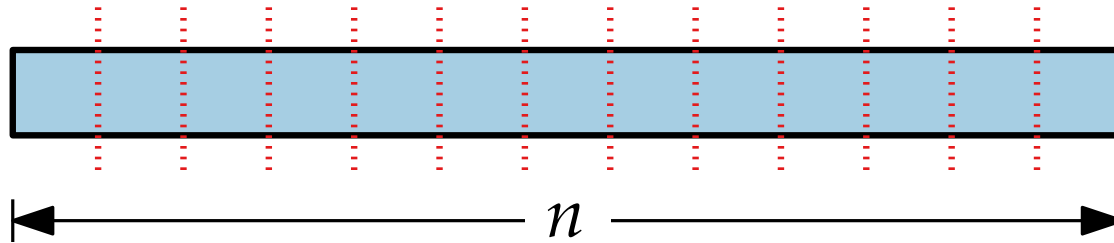


Antwort. Können $n - 1$ mal entscheiden: *schneiden* oder *nicht*.
 $\Rightarrow 2^{n-1}$ verschiedene Zerlegungen

Rohe Gewalt

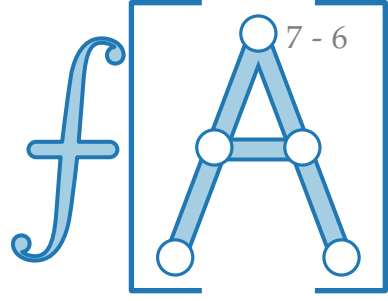


Frage. Wie viele Möglichkeiten gibt es, einen Stab der Länge n zu zerlegen?

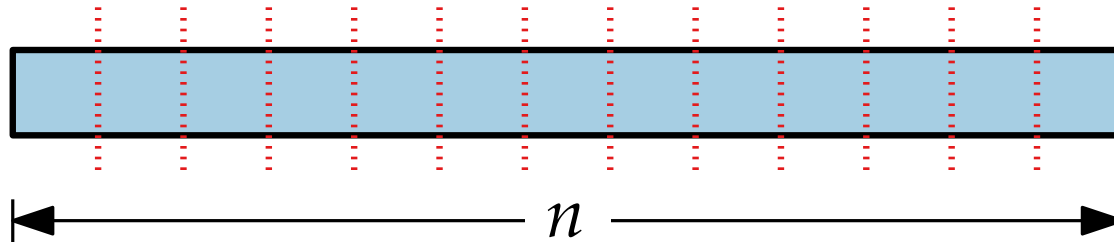


Antwort. Können $n - 1$ mal entscheiden: *schneiden* oder *nicht*.
 $\Rightarrow 2^{n-1}$ verschiedene Zerlegungen

Rohe Gewalt

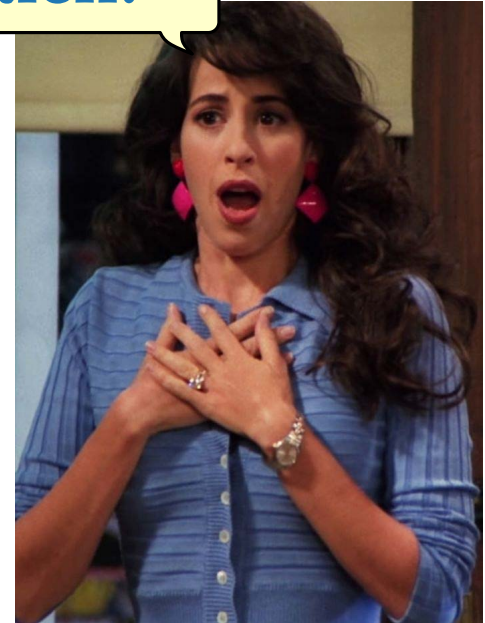


Frage. Wie viele Möglichkeiten gibt es, einen Stab der Länge n zu zerlegen?

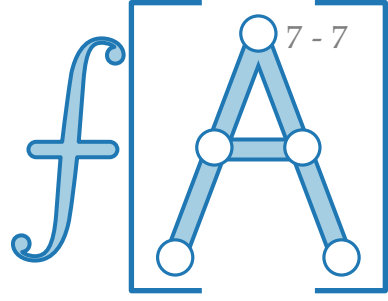


Antwort. Können $n - 1$ mal entscheiden: *schneiden* oder *nicht*.
 $\Rightarrow 2^{n-1}$ verschiedene Zerlegungen

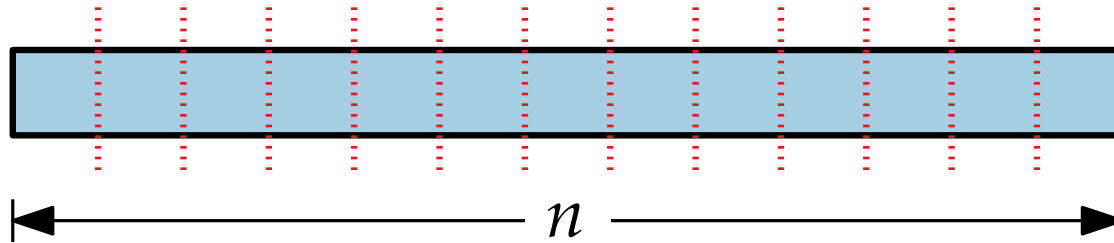
Oh, mein Gott!
Das ist ja **exponentiell**!



Rohe Gewalt



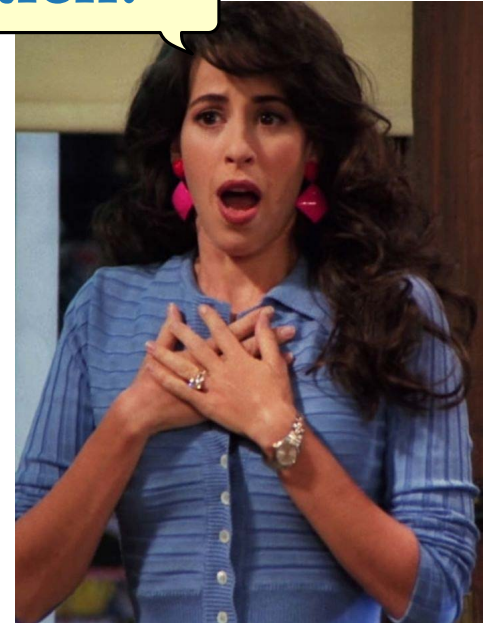
Frage. Wie viele Möglichkeiten gibt es, einen Stab der Länge n zu zerlegen?



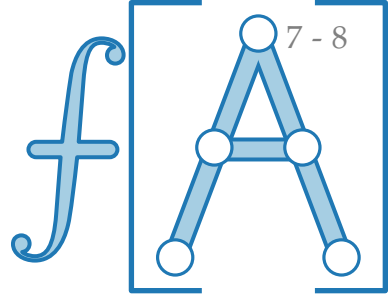
Oh, mein Gott!
Das ist ja **exponentiell!**

Antwort. Können $n - 1$ mal entscheiden: *schneiden* oder *nicht*.
 $\Rightarrow 2^{n-1}$ verschiedene Zerlegungen

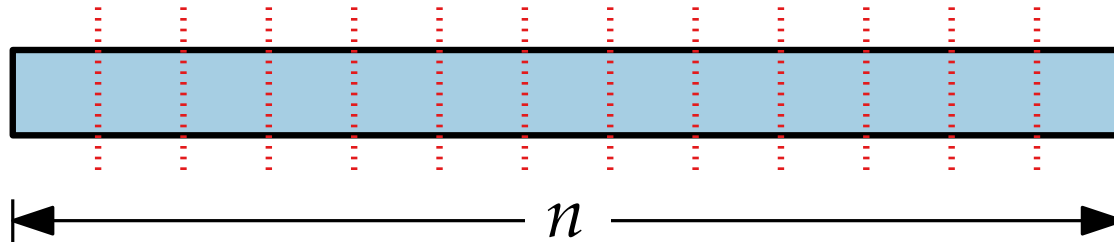
Also können wir es uns nicht leisten alle Zerlegungen durchzugehen und für jede ihren Ertrag zu berechnen.



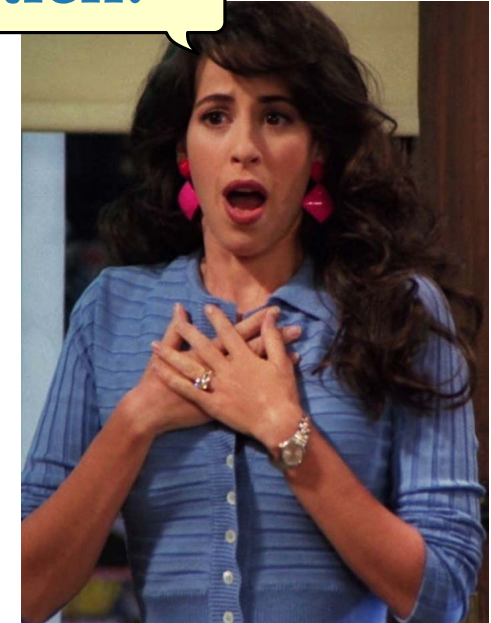
Rohe Gewalt



Frage. Wie viele Möglichkeiten gibt es, einen Stab der Länge n zu zerlegen?



Oh, mein Gott!
Das ist ja **exponentiell!**

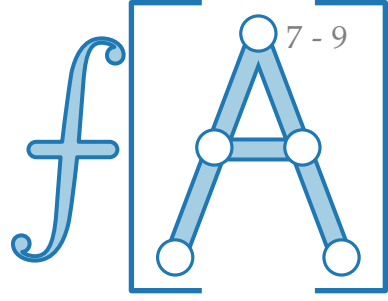


Antwort. Können $n - 1$ mal entscheiden: *schneiden* oder *nicht*.
 $\Rightarrow 2^{n-1}$ verschiedene^{*} Zerlegungen

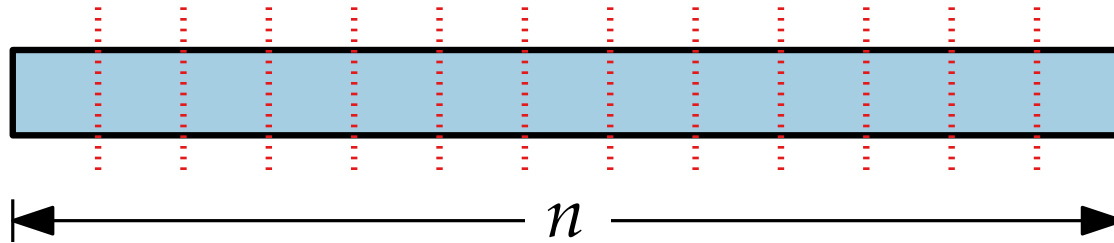
Also können wir es uns nicht leisten alle Zerlegungen durchzugehen und für jede ihren Ertrag zu berechnen.

^{*}) Genauer: die gesuchte Anzahl ist die Anzahl $p(n)$ der *Partitionen* der Zahl n , die angibt, auf wie viele Arten man n als Summe von natürlichen Zahlen schreiben kann. Es gilt $p(n) \approx e^{\pi\sqrt{2n/3}} / (4n\sqrt{3})$

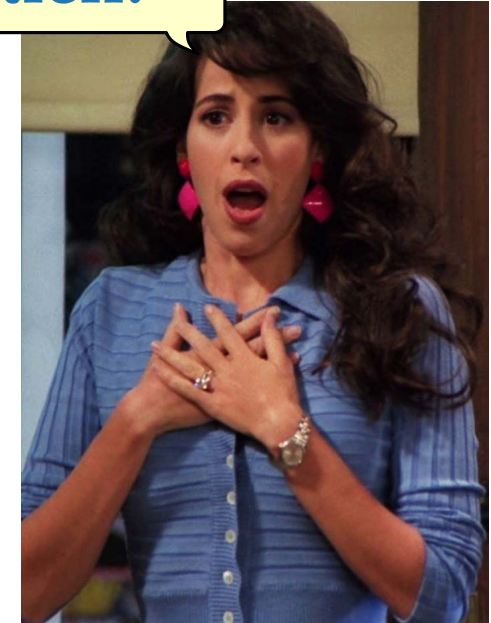
Rohe Gewalt



Frage. Wie viele Möglichkeiten gibt es, einen Stab der Länge n zu zerlegen?



Oh, mein Gott!
Das ist ja **exponentiell!**



Antwort. Können $n - 1$ mal entscheiden: *schneiden* oder *nicht*.
 $\Rightarrow 2^{n-1}$ verschiedene^{*} Zerlegungen

Also können wir es uns nicht leisten alle Zerlegungen durchzugehen und für jede ihren Ertrag zu berechnen.

^{*}) Genauer: die gesuchte Anzahl ist die Anzahl $p(n)$ der *Partitionen* der Zahl n , die angibt, auf wie viele Arten man n als Summe von natürlichen Zahlen schreiben kann. Es gilt $p(n) \approx e^{\pi\sqrt{2n/3}} / (4n\sqrt{3}) \in \Theta^*((13,00195...) \sqrt{n})$.

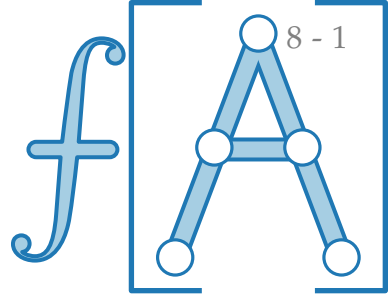
Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Welche Stabzerlegung maximiert den Ertrag?

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9



Ein erster Versuch

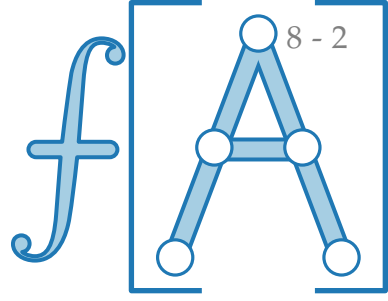
Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Welche Stabzerlegung maximiert den Ertrag?

Greedy:

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9



Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

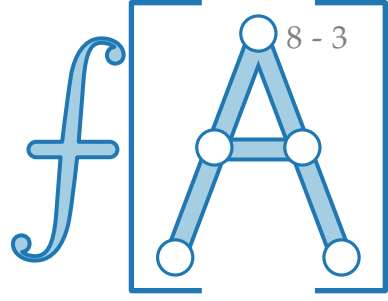
Welche Stabzerlegung maximiert den Ertrag?

Greedy:

- Berechne für $i = 1, \dots, n$ den Preis pro Meter $q_i = p_i / i$.

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9



Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

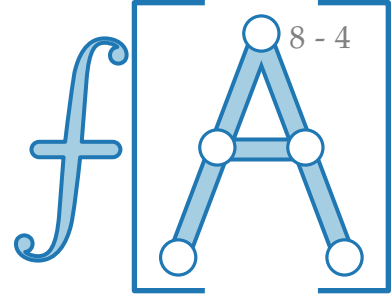
Welche Stabzerlegung maximiert den Ertrag?

Greedy:

- Berechne für $i = 1, \dots, n$ den Preis pro Meter $q_i = p_i / i$.

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9
Quotient q_i [€/m]	1	$2\frac{1}{2}$	$2\frac{2}{3}$	$2\frac{1}{4}$



Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

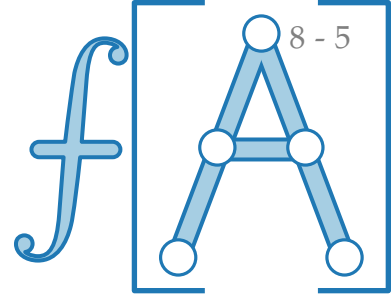
Welche Stabzerlegung maximiert den Ertrag?

Greedy:

- Berechne für $i = 1, \dots, n$ den Preis pro Meter $q_i = p_i / i$.
- Zerlege Stab in möglichst viele Stücke der Länge i mit q_i max.

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9
Quotient q_i [€/m]	1	$2\frac{1}{2}$	$2\frac{2}{3}$	$2\frac{1}{4}$



Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

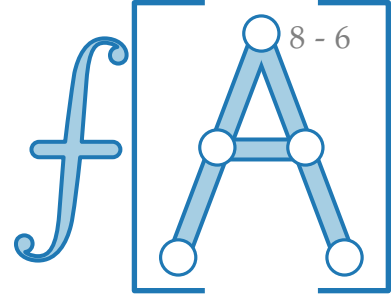
Welche Stabzerlegung maximiert den Ertrag?

Greedy:

- Berechne für $i = 1, \dots, n$ den Preis pro Meter $q_i = p_i / i$.
- Zerlege Stab in möglichst viele Stücke der Länge i mit q_i max.
- Streiche alle Stablängen $\geq i$ aus der Tabelle und wiederhole den Prozess mit dem Stabrest (falls > 0).

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9
Quotient q_i [€/m]	1	$2\frac{1}{2}$	$2\frac{2}{3}$	$2\frac{1}{4}$



Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Welche Stabzerlegung maximiert den Ertrag?

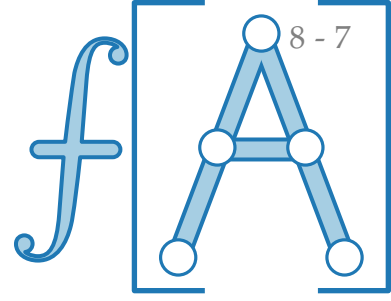
Greedy:

- Berechne für $i = 1, \dots, n$ den Preis pro Meter $q_i = p_i / i$.
- Zerlege Stab in möglichst viele Stücke der Länge i mit q_i max.
- Streiche alle Stablängen $\geq i$ aus der Tabelle und wiederhole den Prozess mit dem Stabrest (falls > 0).

Liefert dieser Greedy-Algorithmus immer das Optimum?

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9
Quotient q_i [€/m]	1	$2\frac{1}{2}$	$2\frac{2}{3}$	$2\frac{1}{4}$



Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Welche Stabzerlegung maximiert den Ertrag?

Greedy:

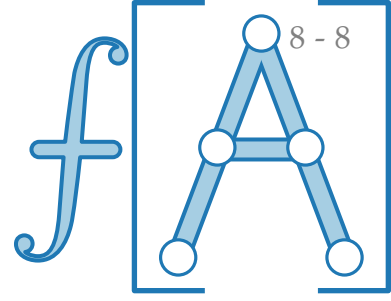
- Berechne für $i = 1, \dots, n$ den Preis pro Meter $q_i = p_i / i$.
- Zerlege Stab in möglichst viele Stücke der Länge i mit q_i max.
- Streiche alle Stablängen $\geq i$ aus der Tabelle und wiederhole den Prozess mit dem Stabrest (falls > 0).

Liefert dieser Greedy-Algorithmus immer das Optimum?

Ja? Beweisen!

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9
Quotient q_i [€/m]	1	$2\frac{1}{2}$	$2\frac{2}{3}$	$2\frac{1}{4}$



Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Welche Stabzerlegung maximiert den Ertrag?

Greedy:

- Berechne für $i = 1, \dots, n$ den Preis pro Meter $q_i = p_i / i$.
- Zerlege Stab in möglichst viele Stücke der Länge i mit q_i max.
- Streiche alle Stablängen $\geq i$ aus der Tabelle und wiederhole den Prozess mit dem Stabrest (falls > 0).

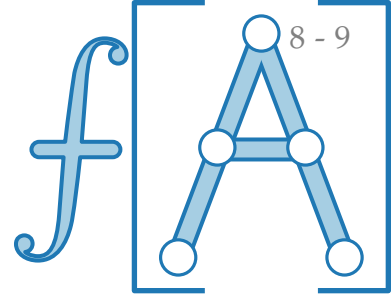
Liefert dieser Greedy-Algorithmus immer das Optimum?

Ja? Beweisen!

Nein? Gegenbeispiel!

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9
Quotient q_i [€/m]	1	$2\frac{1}{2}$	$2\frac{2}{3}$	$2\frac{1}{4}$



Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Welche Stabzerlegung maximiert den Ertrag?

Greedy:

- Berechne für $i = 1, \dots, n$ den Preis pro Meter $q_i = p_i / i$.
- Zerlege Stab in möglichst viele Stücke der Länge i mit q_i max.
- Streiche alle Stablängen $\geq i$ aus der Tabelle und wiederhole den Prozess mit dem Stabrest (falls > 0).

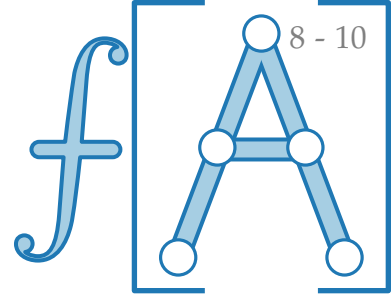
Liefert dieser Greedy-Algorithmus immer das Optimum?

Ja? Beweisen!

Nein? Gegenbeispiel!

Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9
Quotient q_i [€/m]	1	$2\frac{1}{2}$	$2\frac{2}{3}$	$2\frac{1}{4}$



Ein erster Versuch

Wir haben einen Stab der Länge n und kennen die Preise $p_1, p_2, \dots, p_n$ für Stäbe der Längen $1, 2, \dots, n$.

Welche Stabzerlegung maximiert den Ertrag?

Greedy:

- Berechne für $i = 1, \dots, n$ den Preis pro Meter $q_i = p_i / i$.
- Zerlege Stab in möglichst viele Stücke der Länge i mit q_i max.
- Streiche alle Stablängen $\geq i$ aus der Tabelle und wiederhole den Prozess mit dem Stabrest (falls > 0).

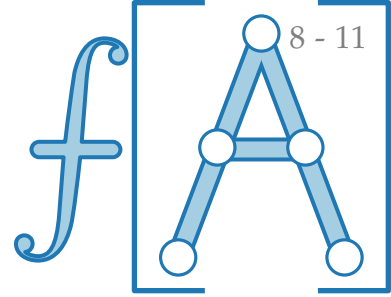
Liefert dieser Greedy-Algorithmus immer das Optimum?

Ja? Beweisen!

Nein? Gegenbeispiel!

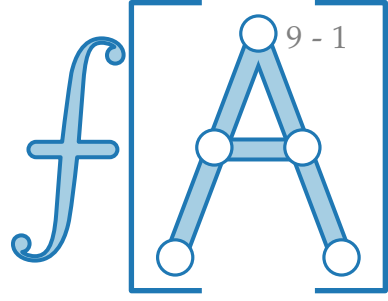
Beispiel: $n = 4$

Länge i [in m]	1	2	3	4
Preis p_i [in €]	1	5	8	9
Quotient q_i [€/m]	1	$2\frac{1}{2}$	$2\frac{2}{3}$	$2\frac{1}{4}$



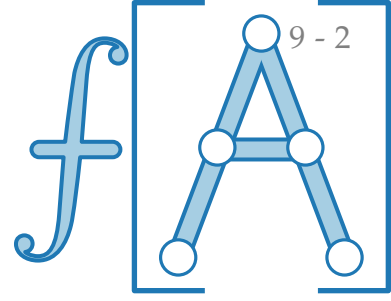
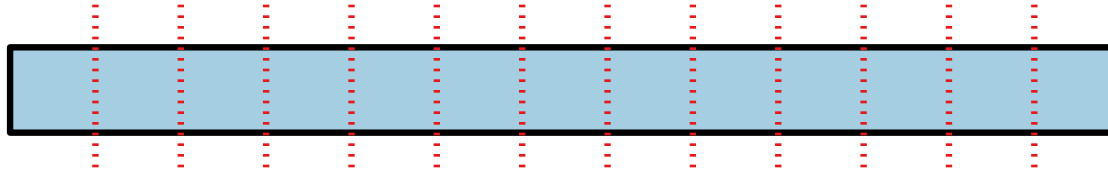
1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



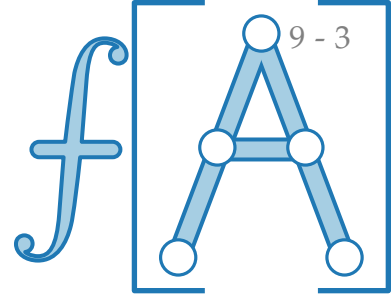
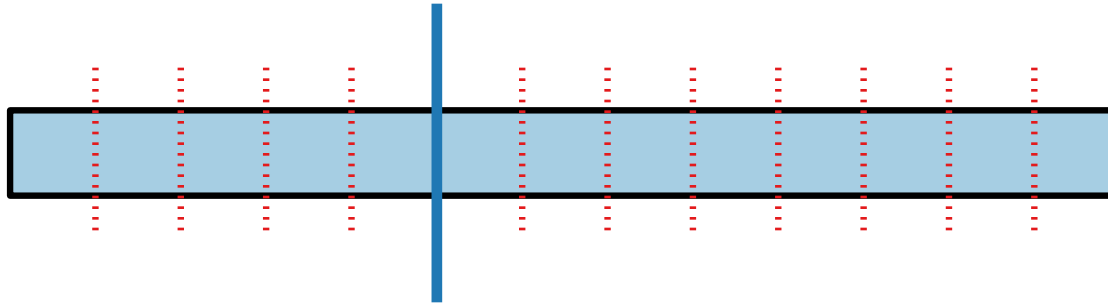
1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



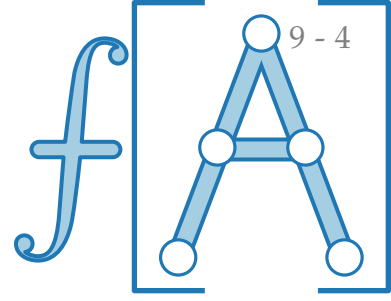
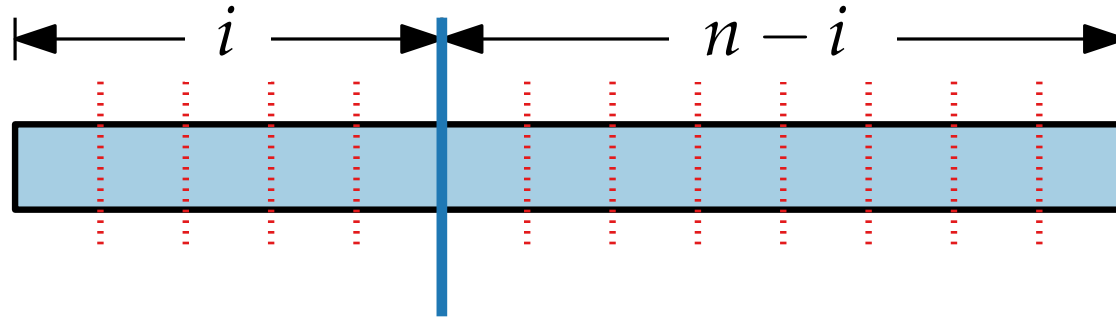
1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



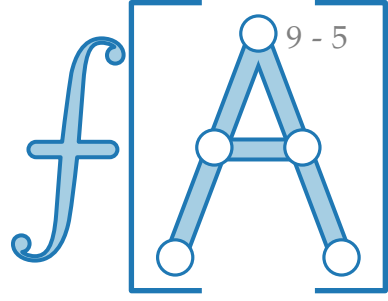
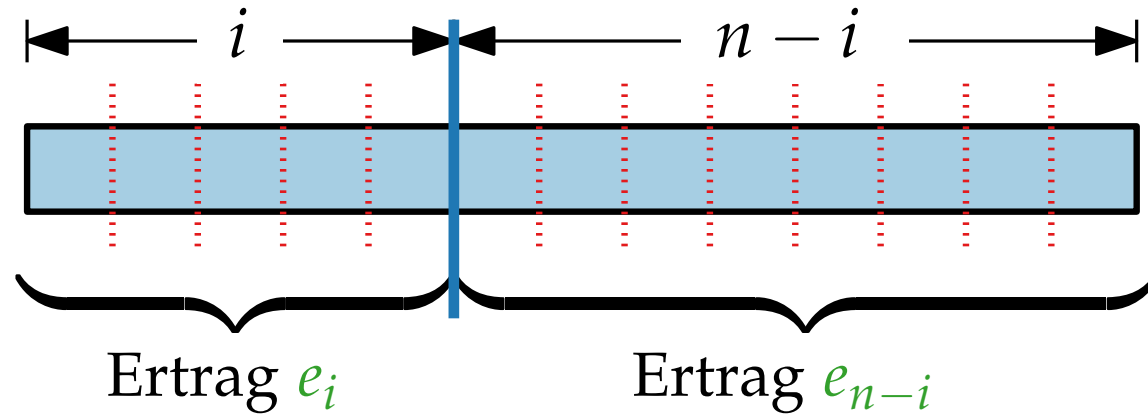
1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .

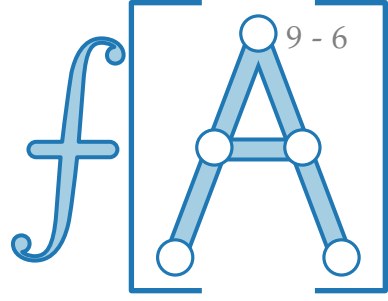


1. Struktur einer optimalen Lösung charakterisieren

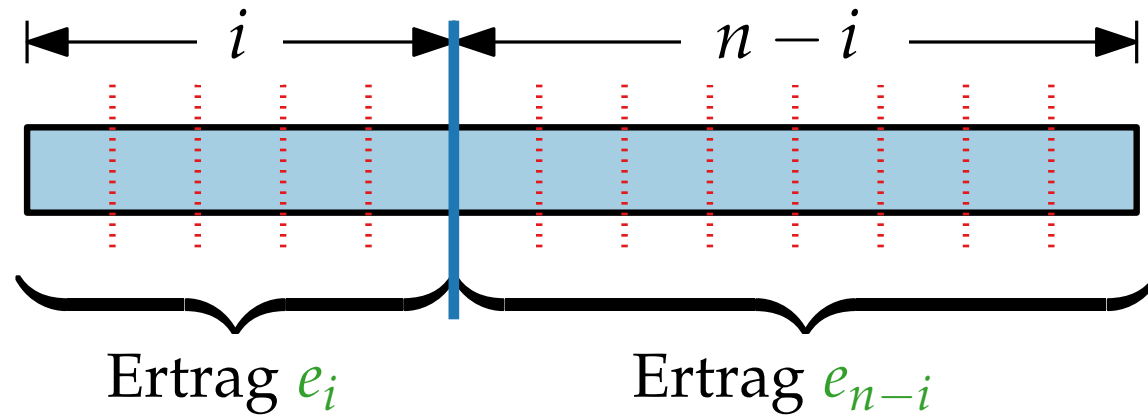
Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



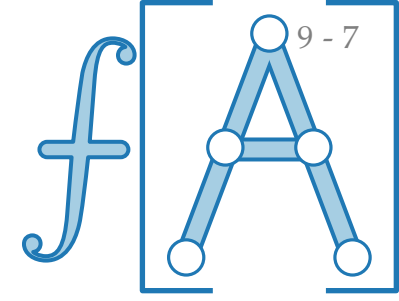
1. Struktur einer optimalen Lösung charakterisieren



Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .

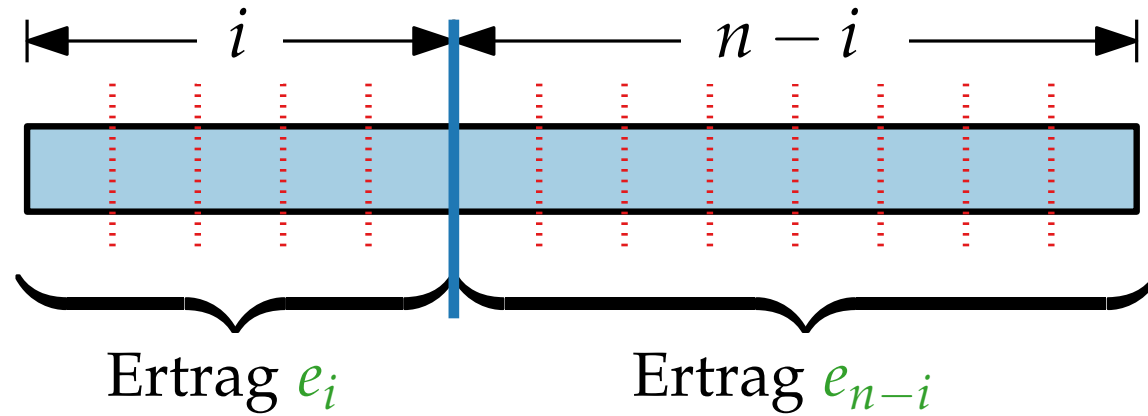


Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.



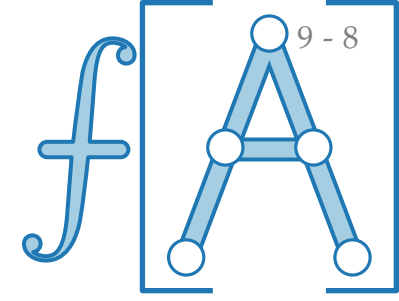
1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



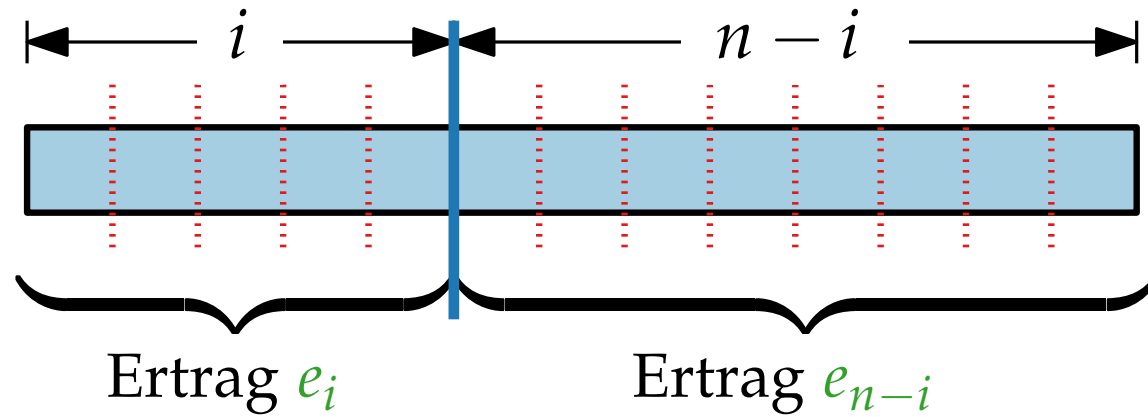
Phänomen
der **optimalen
Teilstruktur!**

Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.



1. Struktur einer optimalen Lösung charakterisieren

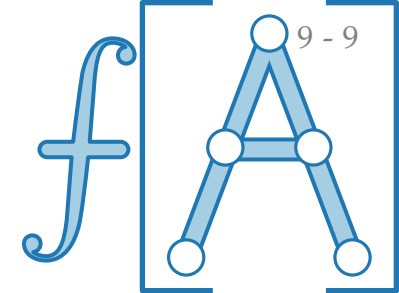
Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

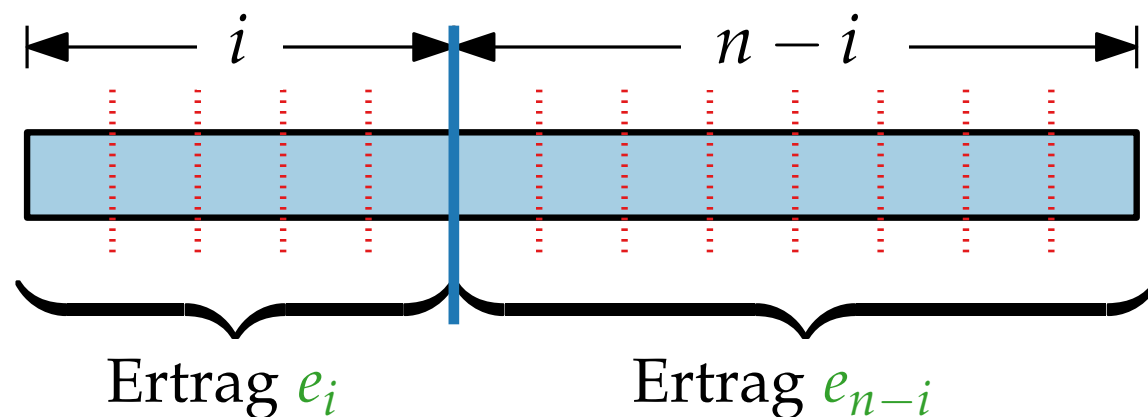
Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

2. Wert einer optimalen Lösung rekursiv definieren



1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

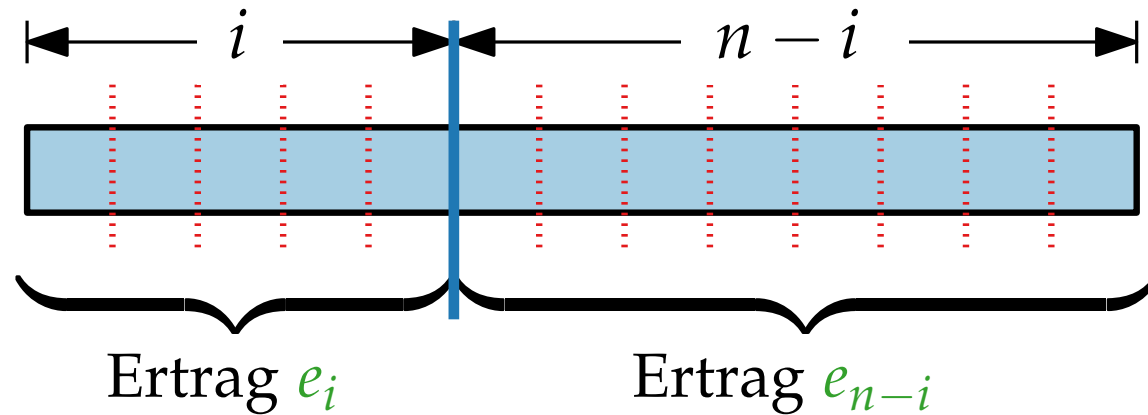
Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

2. Wert einer optimalen Lösung rekursiv definieren

Wissen nicht, *welcher* Schnitt in einer opt. Lösung vorkommt.

1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

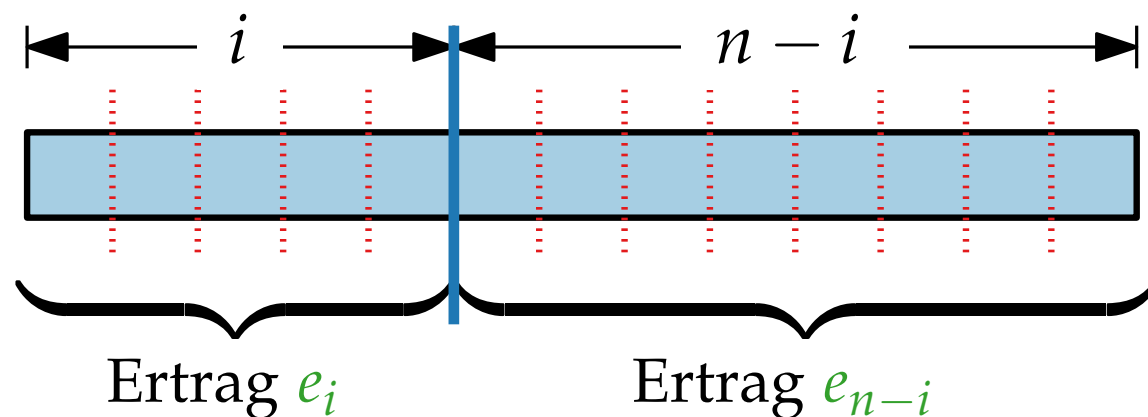
2. Wert einer optimalen Lösung rekursiv definieren

Wissen nicht, *welcher* Schnitt in einer opt. Lösung vorkommt.

Also probieren wir einfach *alle* möglichen Schnitte aus:

1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

2. Wert einer optimalen Lösung rekursiv definieren

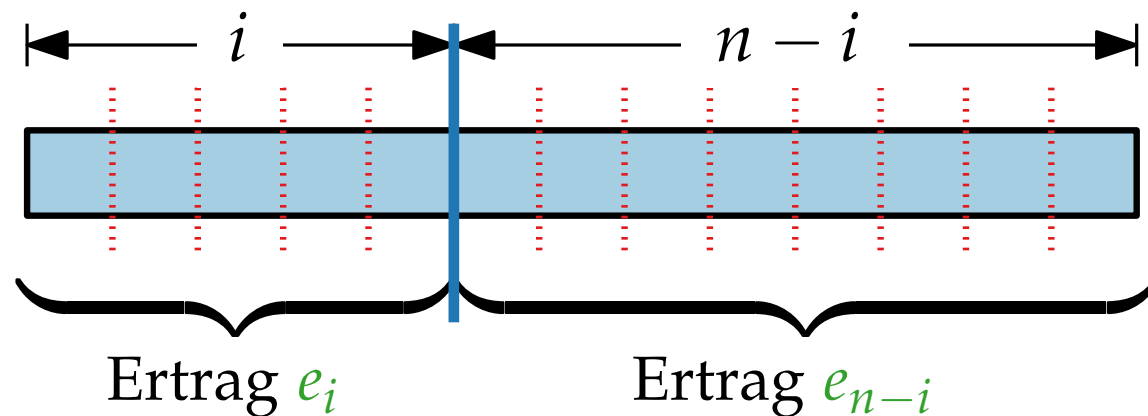
Wissen nicht, *welcher* Schnitt in einer opt. Lösung vorkommt.

Also probieren wir einfach *alle* möglichen Schnitte aus:

$e_n =$

1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

2. Wert einer optimalen Lösung rekursiv definieren

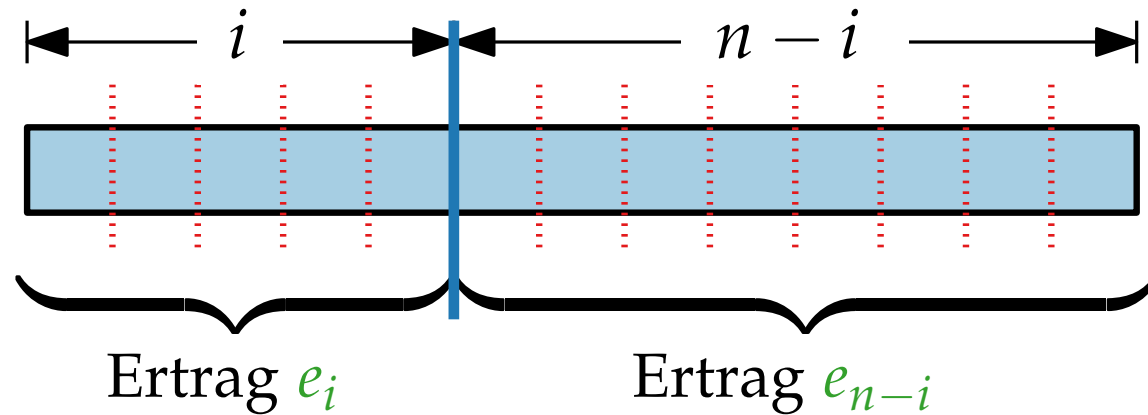
Wissen nicht, *welcher* Schnitt in einer opt. Lösung vorkommt.

Also probieren wir einfach *alle* möglichen Schnitte aus:

$$e_n = \max\{ \quad \quad \quad \}$$

1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

2. Wert einer optimalen Lösung rekursiv definieren

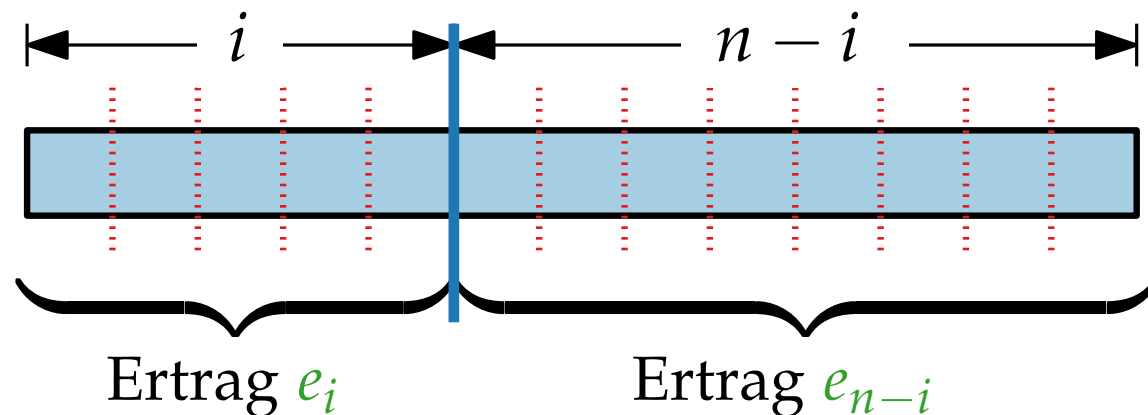
Wissen nicht, *welcher* Schnitt in einer opt. Lösung vorkommt.

Also probieren wir einfach *alle* möglichen Schnitte aus:

$$e_n = \max\{ p_n, \quad \}$$

1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

2. Wert einer optimalen Lösung rekursiv definieren

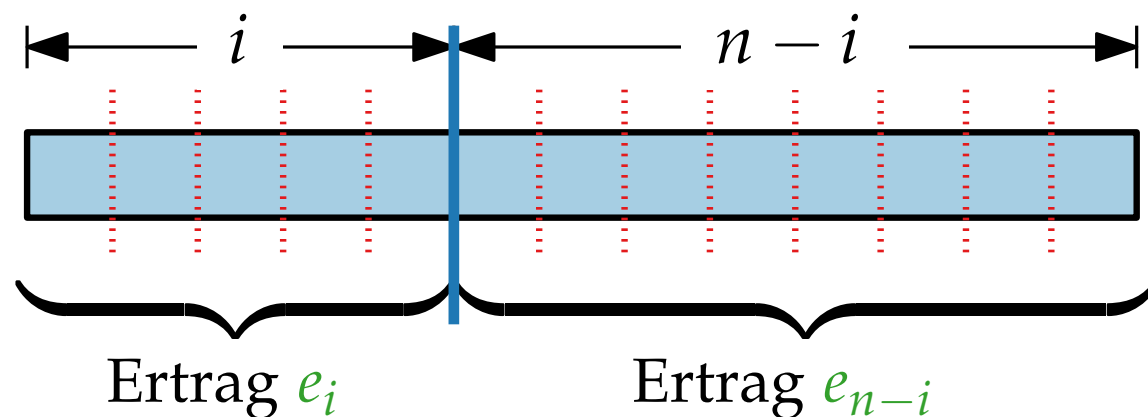
Wissen nicht, *welcher* Schnitt in einer opt. Lösung vorkommt.

Also probieren wir einfach *alle* möglichen Schnitte aus:

$$e_n = \max\{ p_n, e_1 + e_{n-1}, \quad \}$$

1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

2. Wert einer optimalen Lösung rekursiv definieren

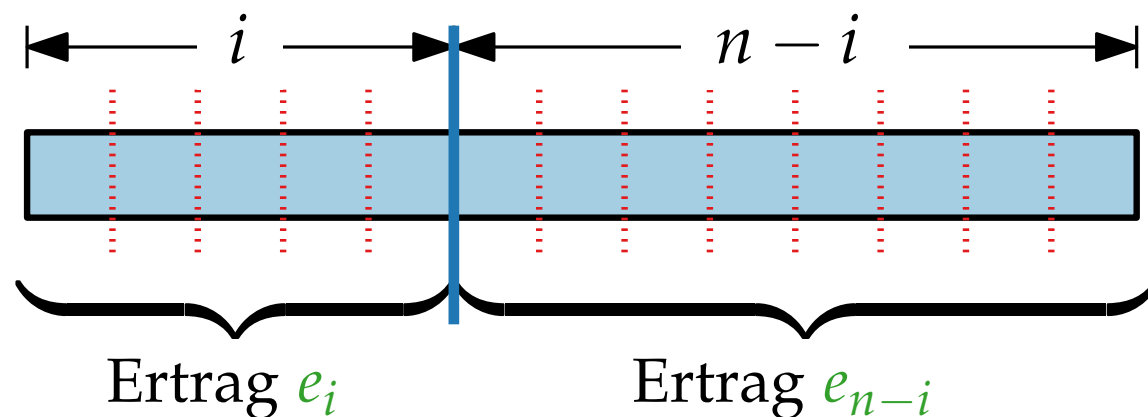
Wissen nicht, *welcher* Schnitt in einer opt. Lösung vorkommt.

Also probieren wir einfach *alle* möglichen Schnitte aus:

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots \}$$

1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

2. Wert einer optimalen Lösung rekursiv definieren

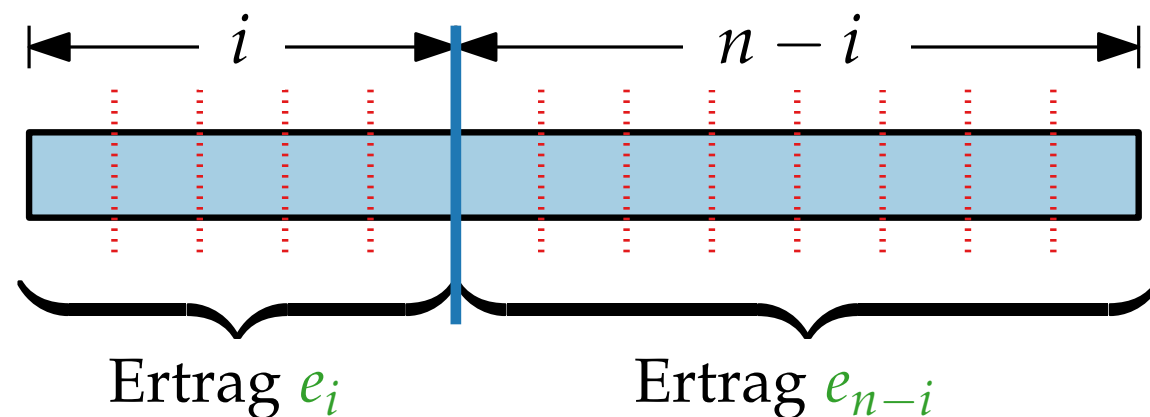
Wissen nicht, *welcher* Schnitt in einer opt. Lösung vorkommt.

Also probieren wir einfach *alle* möglichen Schnitte aus:

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, \}$$

1. Struktur einer optimalen Lösung charakterisieren

Def. Für $i = 1, \dots, n$
sei e_i der maximale Ertrag für einen Stab der Länge i .



Phänomen
der **optimalen
Teilstruktur!**

Beob. Ein Schnitt zerlegt das Problem in **unabhängige** Teilprobleme.

2. Wert einer optimalen Lösung rekursiv definieren

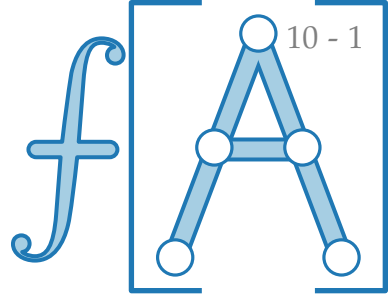
Wissen nicht, *welcher* Schnitt in einer opt. Lösung vorkommt.

Also probieren wir einfach *alle* möglichen Schnitte aus:

$$e_n = \max \{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

2. Wert einer optimalen Lösung rekursiv definieren

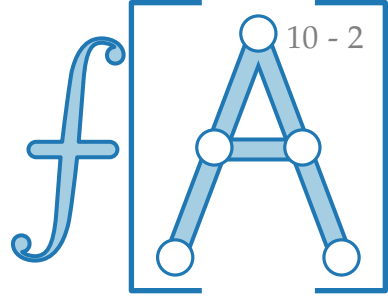
$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$



2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

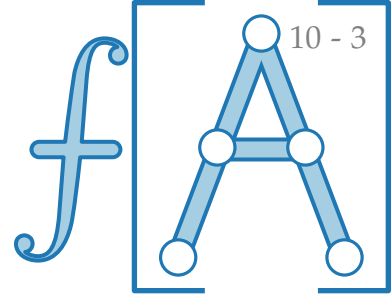
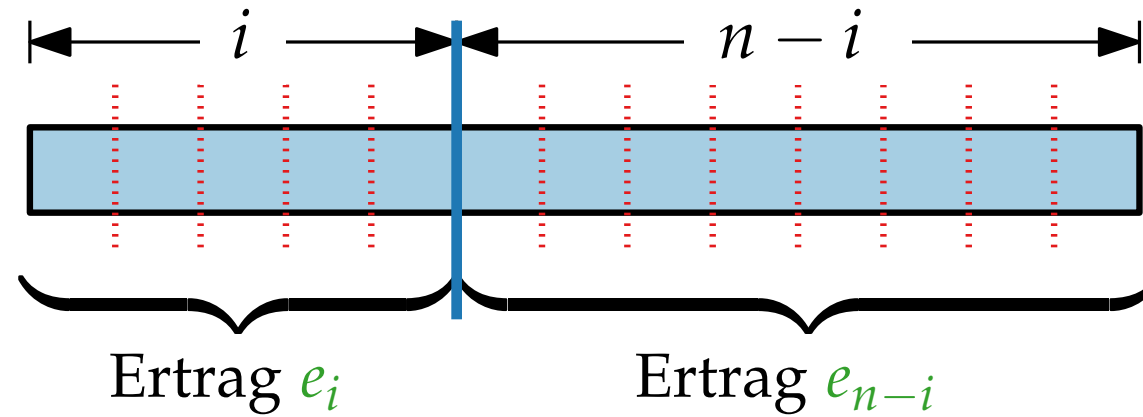
Kleine Verbesserung:



2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

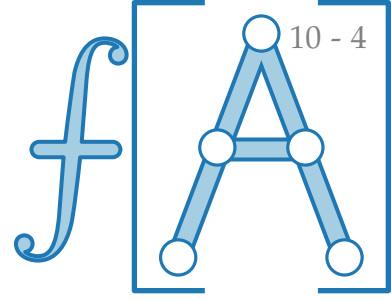
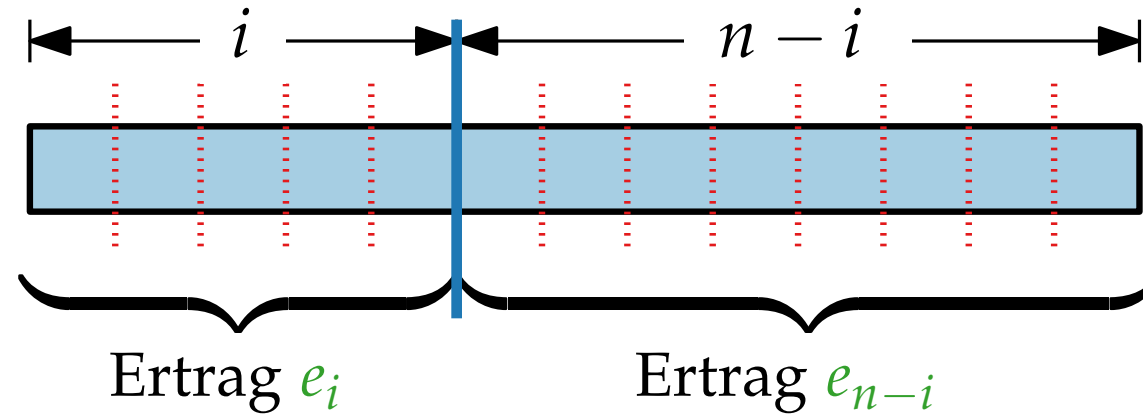


2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiете weitere Schnitte
im linken Teilstück!

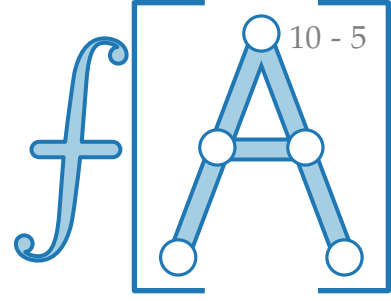
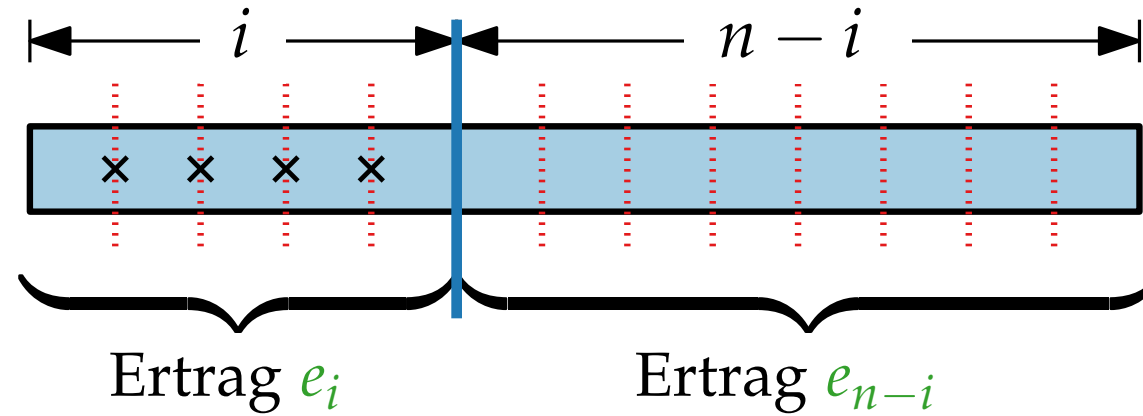


2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiете weitere Schnitte
im linken Teilstück!

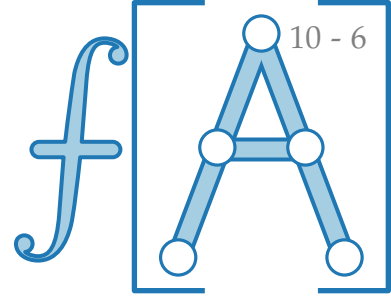
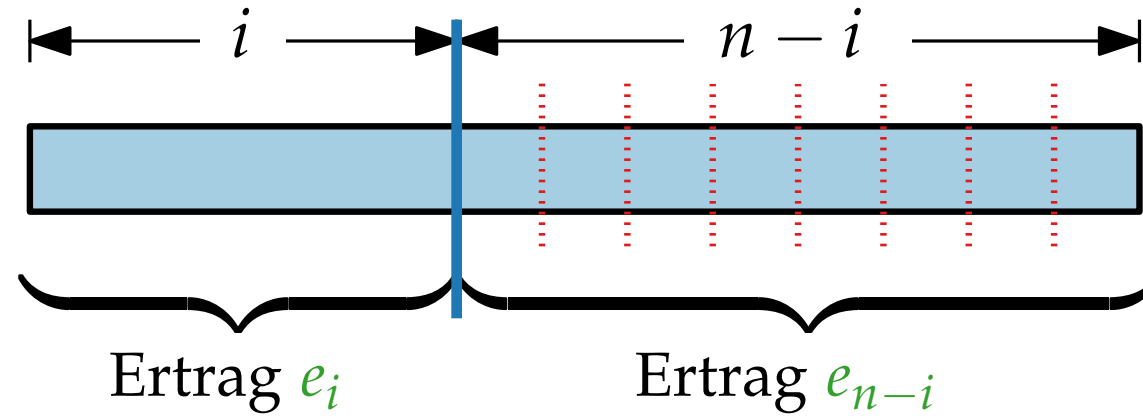


2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiere weitere Schnitte
im linken Teilstück!

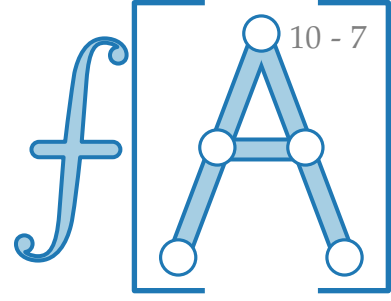
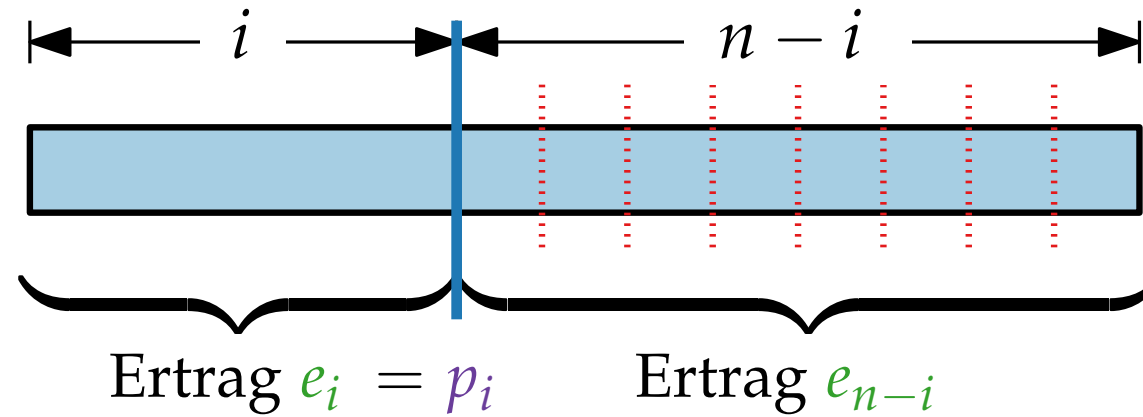


2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiете weitere Schnitte
im linken Teilstück!

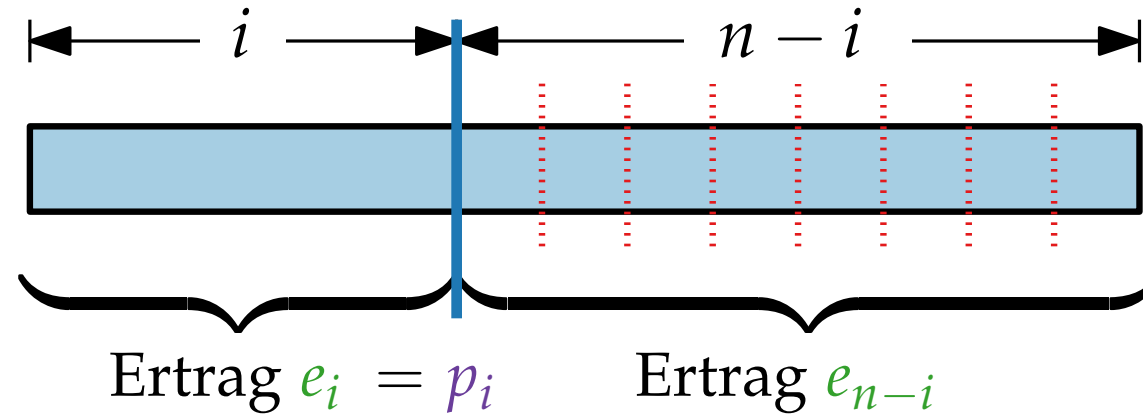


2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

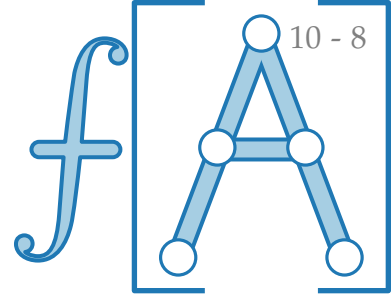
Kleine Verbesserung:

Verbiere weitere Schnitte
im linken Teilstück!



Also gilt:

$$e_n =$$

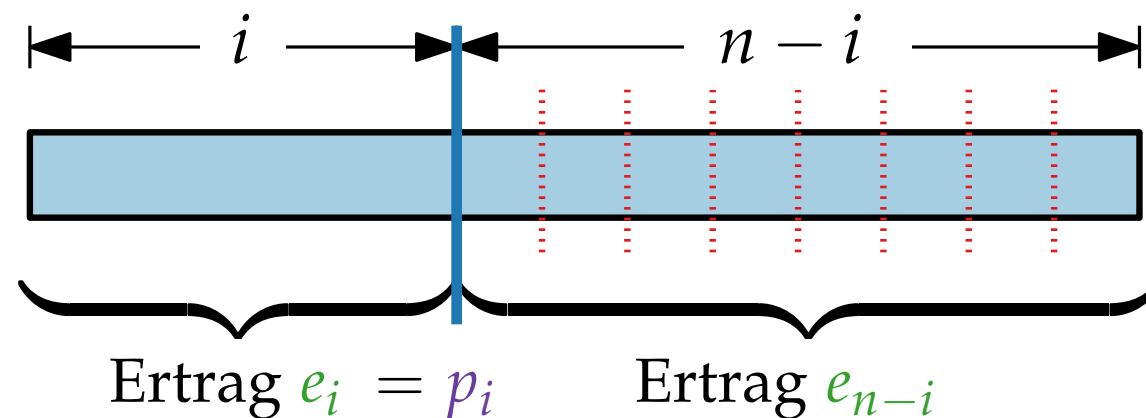


2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiere weitere Schnitte
im linken Teilstück!



Also gilt:

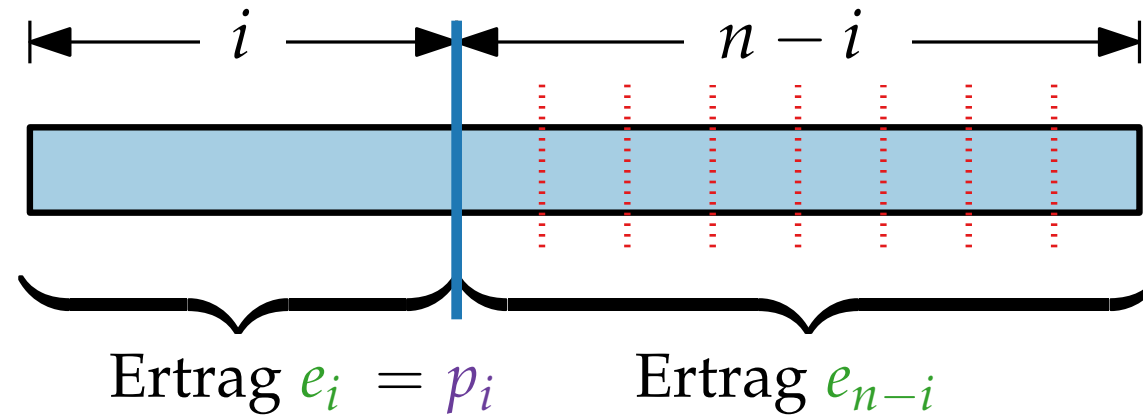
$$e_n = \max\{$$

2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

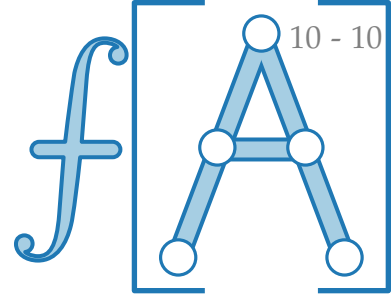
Kleine Verbesserung:

Verbiere weitere Schnitte
im linken Teilstück!



Also gilt:

$$e_n = \max\{ p_n, \quad \quad \quad \}$$

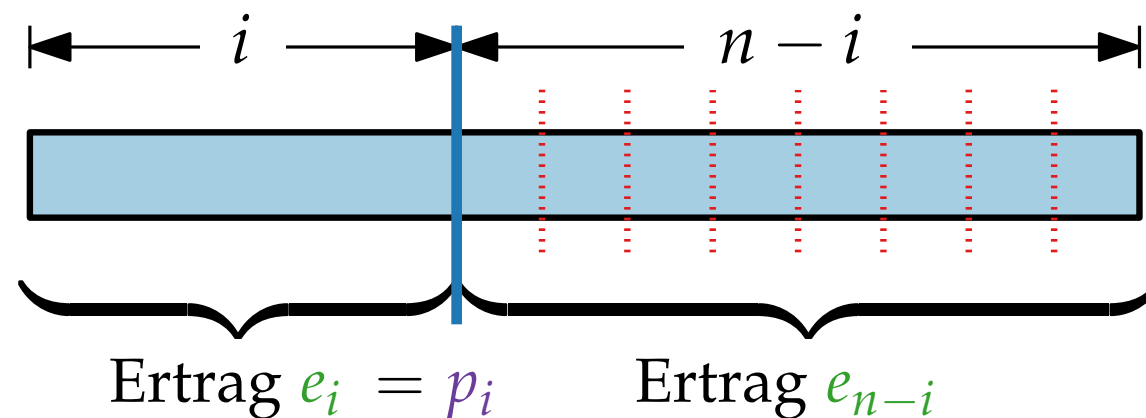


2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiere weitere Schnitte
im linken Teilstück!



Also gilt:

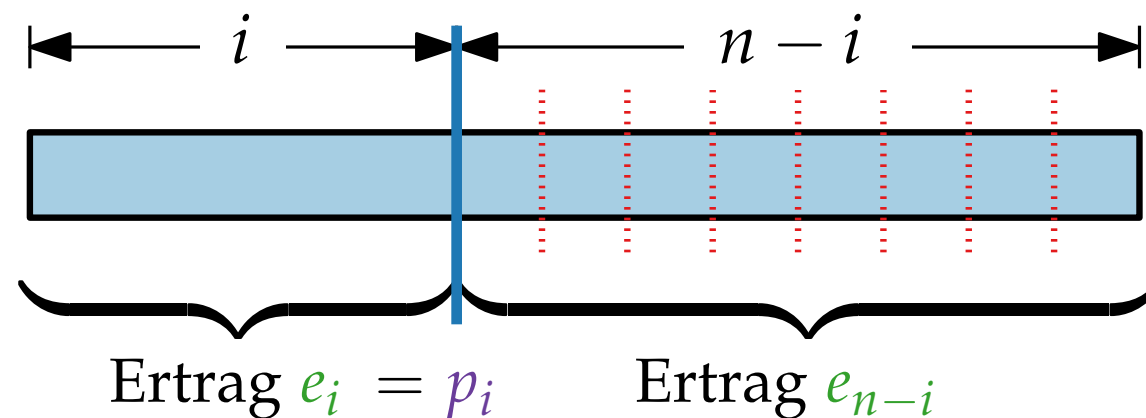
$$e_n = \max\{ p_n, p_1 + e_{n-1}, \dots \}$$

2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiere weitere Schnitte
im linken Teilstück!



Also gilt:

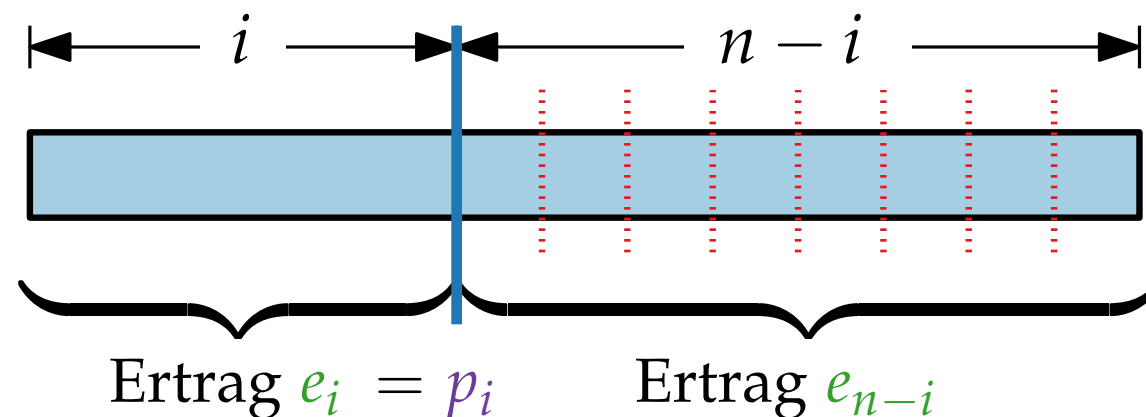
$$e_n = \max\{ p_n, p_1 + e_{n-1}, p_2 + e_{n-2}, \dots, \}$$

2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiee weitere Schnitte
im linken Teilstück!



Also gilt:

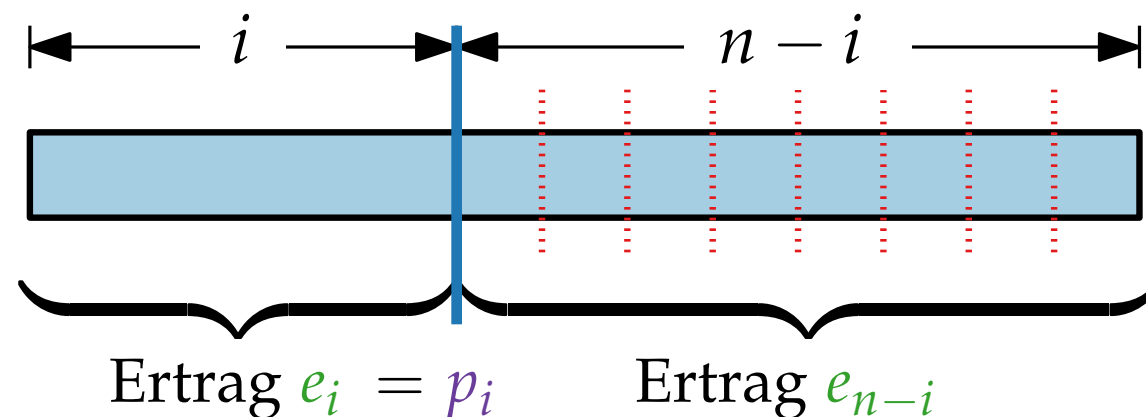
$$e_n = \max\{ p_n, p_1 + e_{n-1}, p_2 + e_{n-2}, \dots, p_{n-1} + e_1 \}$$

2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiere weitere Schnitte
im linken Teilstück!



Also gilt:

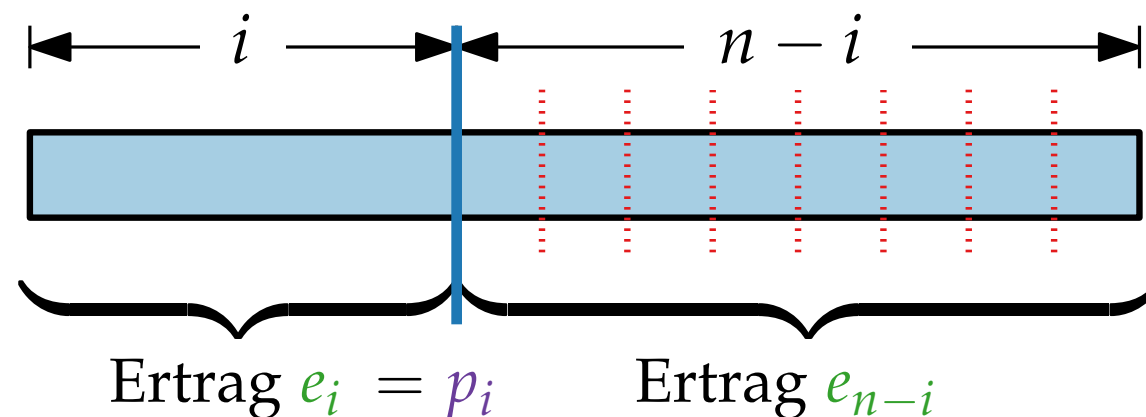
$$\begin{aligned} e_n &= \max\{ p_n, p_1 + e_{n-1}, p_2 + e_{n-2}, \dots, p_{n-1} + e_1 \} \\ &= \max_{1 \leq i \leq n} \{ p_i + e_{n-i} \} \end{aligned}$$

2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiere weitere Schnitte
im linken Teilstück!



Also gilt:

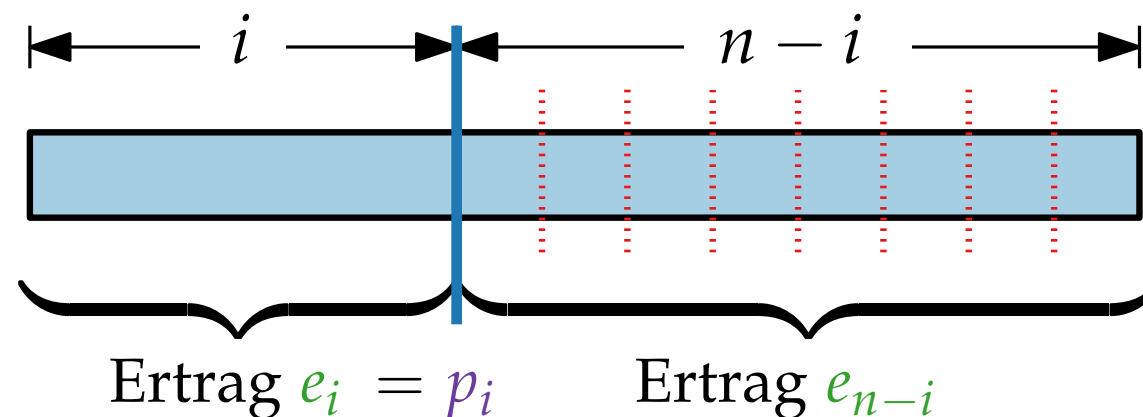
$$\begin{aligned} e_n &= \max\{ p_n, p_1 + e_{n-1}, p_2 + e_{n-2}, \dots, p_{n-1} + e_1 \} \\ &= \max_{1 \leq i \leq n} \{ p_i + e_{n-i} \}, \quad \text{wobei } e_0 := 0. \end{aligned}$$

2. Wert einer optimalen Lösung rekursiv definieren

$$e_n = \max\{ p_n, e_1 + e_{n-1}, e_2 + e_{n-2}, \dots, e_{n-1} + e_1 \}$$

Kleine Verbesserung:

Verbiere weitere Schnitte
im linken Teilstück!



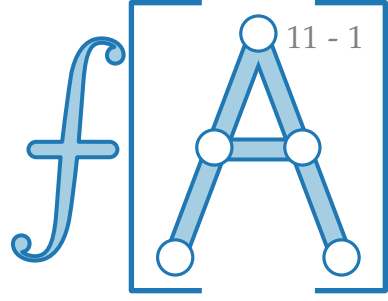
Also gilt:

$$\begin{aligned} e_n &= \max\{ p_n, p_1 + e_{n-1}, p_2 + e_{n-2}, \dots, p_{n-1} + e_1 \} \\ &= \max_{1 \leq i \leq n} \{ p_i + e_{n-i} \}, \quad \text{wobei } e_0 := 0. \end{aligned}$$

Vorteil: Wert einer optimalen Lösung ist Summe aus einer Zahl der Eingabe und *einem* Wert einer optimalen Teillösung.

3. Wert einer optimalen Lösung berechnen: *top-down*

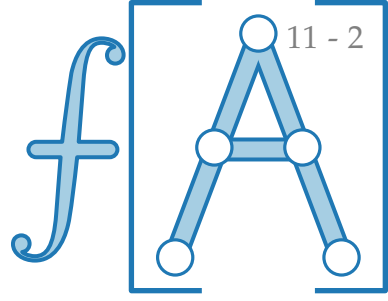
Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.



3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0
```



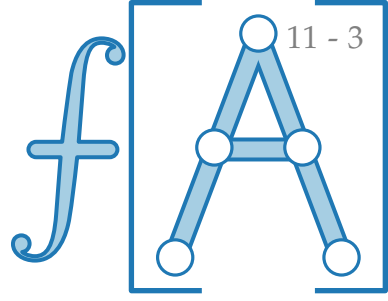
3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

```
    if  $n == 0$  then return 0
```

```
     $q = -\infty$ 
```



3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

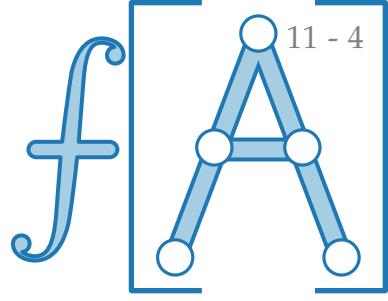
```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

```
    if  $n == 0$  then return 0
```

```
     $q = -\infty$ 
```

```
    for  $i = 1$  to  $n$  do
```

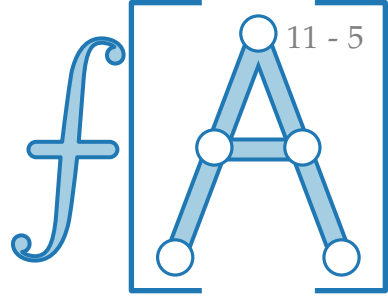
```
        |
```



3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

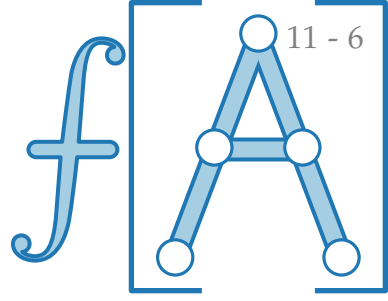
```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0  
   $q = -\infty$   
  for  $i = 1$  to  $n$  do  
     $q = \max\{q, p[i] + \text{STANGENZERLEGUNG}(p, n - i)\}$ 
```



3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0  
   $q = -\infty$   
  for  $i = 1$  to  $n$  do  
     $q = \max\{q, p[i] + \text{STANGENZERLEGUNG}(p, n - i)\}$   
  return  $q$ 
```

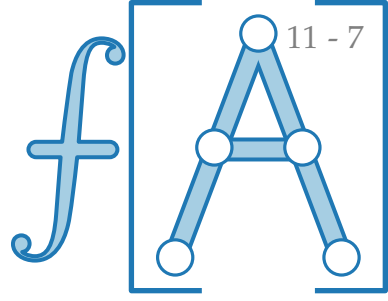


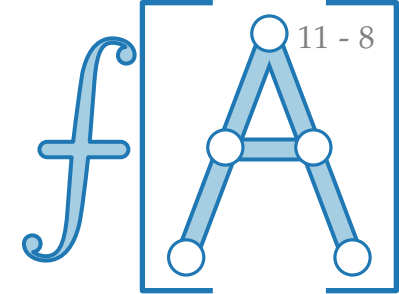
3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0  
   $q = -\infty$   
  for  $i = 1$  to  $n$  do  
     $q = \max\{q, p[i] + \text{STANGENZERLEGUNG}(p, n - i)\}$   
  return  $q$ 
```

Laufzeit.



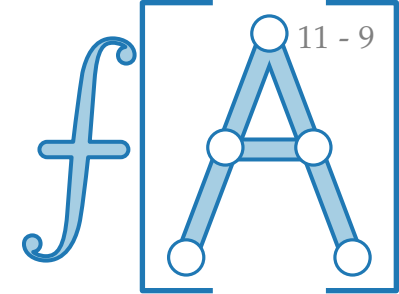


3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0  
   $q = -\infty$   
  for  $i = 1$  to  $n$  do  
     $q = \max\{q, p[i] + \text{STANGENZERLEGUNG}(p, n - i)\}$   
  return  $q$ 
```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$



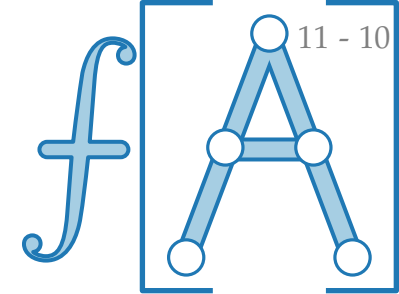
3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0  
   $q = -\infty$   
  for  $i = 1$  to  $n$  do  
     $q = \max\{q, p[i] + \text{STANGENZERLEGUNG}(p, n - i)\}$   
  return  $q$ 
```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$\Rightarrow A(0) =$



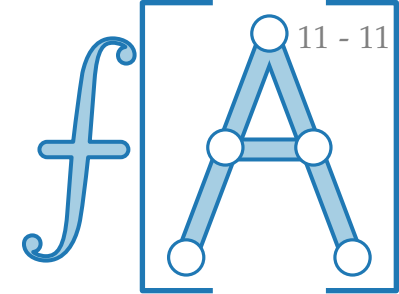
3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0  
   $q = -\infty$   
  for  $i = 1$  to  $n$  do  
     $q = \max\{q, p[i] + \text{STANGENZERLEGUNG}(p, n - i)\}$   
  return  $q$ 
```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$$\Rightarrow A(0) = 1$$



3. Wert einer optimalen Lösung berechnen: *top-down*

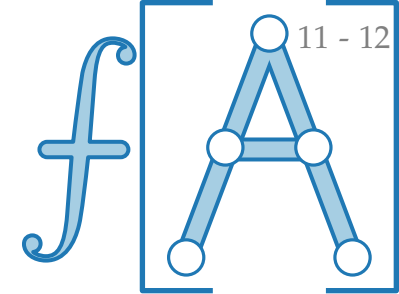
Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0  
   $q = -\infty$   
  for  $i = 1$  to  $n$  do  
     $q = \max\{q, p[i] + \text{STANGENZERLEGUNG}(p, n - i)\}$   
  return  $q$ 
```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$$\Rightarrow A(0) = 1$$

$$\text{und } A(n) =$$



3. Wert einer optimalen Lösung berechnen: *top-down*

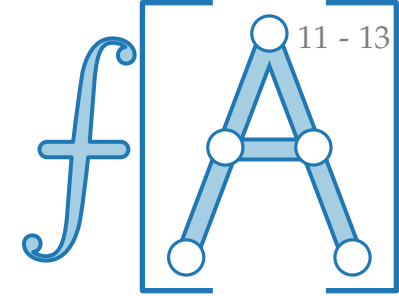
Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0  
   $q = -\infty$   
  for  $i = 1$  to  $n$  do  
     $q = \max\{q, p[i] + \text{STANGENZERLEGUNG}(p, n - i)\}$   
  return  $q$ 
```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$$\Rightarrow A(0) = 1$$

$$\text{und } A(n) = 1 + \sum_{i=1}^n A(n-i)$$



3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```

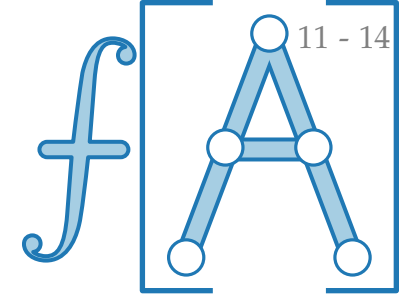
STANGENZERLEGUNG(int[] p, int n = p.length)
  if n == 0 then return 0
  q = -∞
  for i = 1 to n do
    q = max{q, p[i] + STANGENZERLEGUNG(p, n - i)}
  return q

```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$$\Rightarrow A(0) = 1$$

$$\text{und } A(n) = 1 + \sum_{i=1}^n A(n-i)$$



3. Wert einer optimalen Lösung berechnen: *top-down*

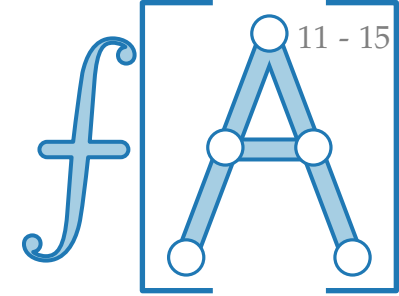
Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```
STANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
  if  $n == 0$  then return 0  
   $q = -\infty$   
  for  $i = 1$  to  $n$  do  
     $q = \max\{q, p[i] + \text{STANGENZERLEGUNG}(p, n - i)\}$   
  return  $q$ 
```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$$\Rightarrow A(0) = 1$$

$$\text{und } A(n) = 1 + \sum_{i=1}^n A(n-i) = 1 + \sum_{j=0}^{n-1} A(j)$$



3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```

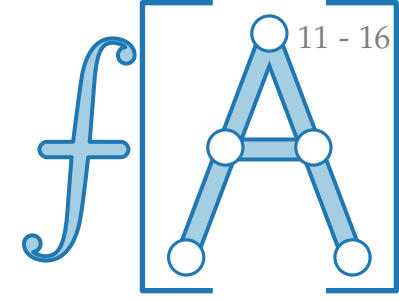
STANGENZERLEGUNG(int[] p, int n = p.length)
  if n == 0 then return 0
  q = -∞
  for i = 1 to n do
    q = max{q, p[i] + STANGENZERLEGUNG(p, n - i)}
  return q

```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$$\Rightarrow A(0) = 1$$

$$\text{und } A(n) = 1 + \sum_{i=1}^n A(n-i) = 1 + \sum_{j=0}^{n-1} A(j) = 2^n - 1$$



3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```

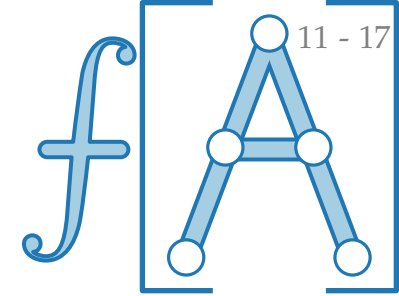
STANGENZERLEGUNG(int[] p, int n = p.length)
  if n == 0 then return 0
  q = -∞
  for i = 1 to n do
    q = max{q, p[i] + STANGENZERLEGUNG(p, n - i)}
  return q

```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$$\Rightarrow A(0) = 1$$

$$\text{und } A(n) = 1 + \sum_{i=1}^n A(n-i) = 1 + \sum_{j=0}^{n-1} A(j) = 2^n - 1 \quad \text{☹️}$$



3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```

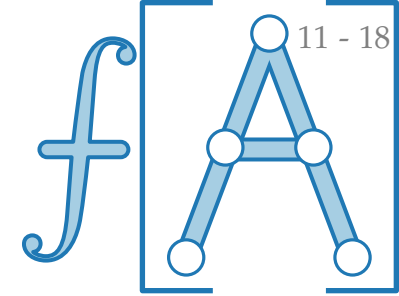
STANGENZERLEGUNG(int[] p, int n = p.length)
  if n == 0 then return 0
  q = -∞
  for i = 1 to n do
    q = max{q, p[i] + STANGENZERLEGUNG(p, n - i)}
  return q

```

Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$$\Rightarrow A(0) = 1$$

$$\text{und } A(n) = 1 + \sum_{i=1}^n A(n-i) = 1 + \sum_{j=0}^{n-1} A(j) \stackrel{\text{Beweis?!}}{=} 2^n - 1 \quad \text{☹️}$$



3. Wert einer optimalen Lösung berechnen: *top-down*

Wir wissen: $e_n = \max_{1 \leq i \leq n} \{p_i + e_{n-i}\}$, wobei $e_0 := 0$.

```

STANGENZERLEGUNG(int[] p, int n = p.length)
  if n == 0 then return 0
  q = -∞
  for i = 1 to n do
    q = max{q, p[i] + STANGENZERLEGUNG(p, n - i)}
  return q

```

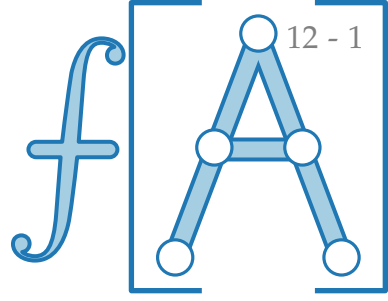
Laufzeit. Sei $A(n)$ die Gesamtzahl
von Aufrufen von $\text{STANGENZERLEGUNG}(p, \cdot)$
beim Ausführen von $\text{STANGENZERLEGUNG}(p, n)$

$$\Rightarrow A(0) = 1$$

$$\text{und } A(n) = 1 + \sum_{i=1}^n A(n-i) = 1 + \underbrace{\sum_{j=0}^{n-1} A(j)}_{1+2+4+8+\dots+2^{n-1}} \stackrel{\text{Beweis?!}}{=} 2^n - 1 \quad \text{☹️}$$

3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

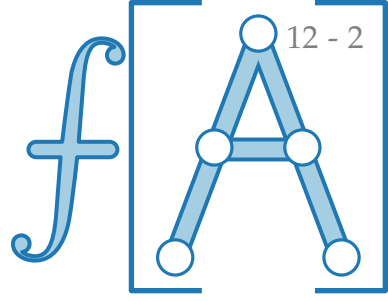
Zeit-Speicher-Tausch (engl. *time-memory trade-off*)



3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

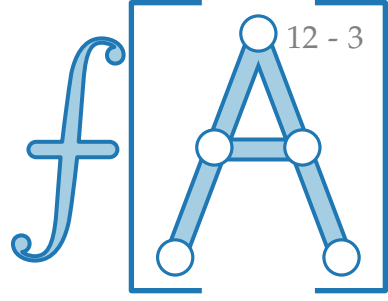
```
MEMOSTANGENZERLEGUNG(int[] p, int n = p.length)
```



3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[] p, int n = p.length)  
    e = new int[0...n]
```



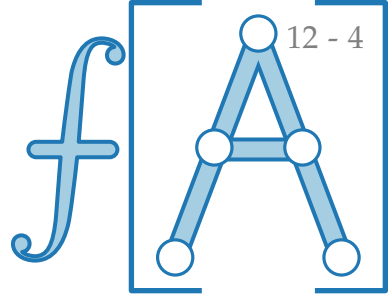
3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[] p, int n = p.length)
```

```
    e = new int[0...n]
```

```
    e[0] = 0
```



3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

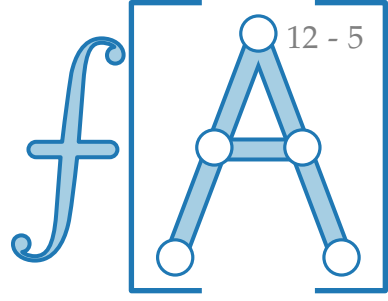
```
MEMOSTANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

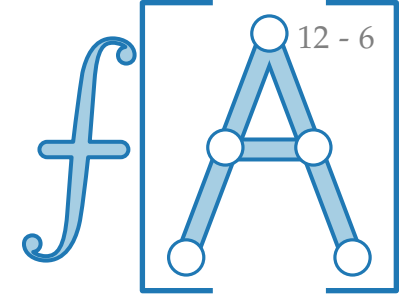
```
   $e = \text{new int}[0 \dots n]$ 
```

```
   $e[0] = 0$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $e[i] = -\infty$ 
```





3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )  
   $e = \text{new int}[0 \dots n]$   
   $e[0] = 0$   
  for  $i = 1$  to  $n$  do  
     $e[i] = -\infty$   
  return HAUPTSTANGENZERLEGUNG( $p, n, e$ )
```

3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[] p, int n = p.length)
```

```
  e = new int[0...n]
```

```
  e[0] = 0
```

```
  for i = 1 to n do
```

```
    e[i] =  $-\infty$ 
```

```
  return HAUPTSTANGENZERLEGUNG(p, n, e)
```

```
HAUPTSTANGENZERLEGUNG(int[] p, int n, int[] e)
```

3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

```
   $e = \text{new int}[0 \dots n]$ 
```

```
   $e[0] = 0$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $e[i] = -\infty$ 
```

```
  return HAUPTSTANGENZERLEGUNG( $p, n, e$ )
```

```
HAUPTSTANGENZERLEGUNG(int[]  $p$ , int  $n$ , int[]  $e$ )
```

```
  if  $e[n] > -\infty$  then return  $e[n]$ 
```

3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

```
   $e = \text{new int}[0 \dots n]$ 
```

```
   $e[0] = 0$ 
```

```
  for  $i = 1$  to  $n$  do
```

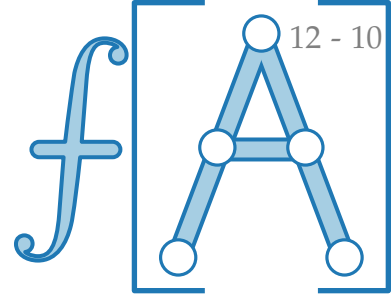
```
     $e[i] = -\infty$ 
```

```
  return HAUPTSTANGENZERLEGUNG( $p, n, e$ )
```

```
HAUPTSTANGENZERLEGUNG(int[]  $p$ , int  $n$ , int[]  $e$ )
```

```
  if  $e[n] > -\infty$  then return  $e[n]$ 
```

```
   $q = -\infty$ 
```

3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

```
   $e = \text{new int}[0 \dots n]$ 
```

```
   $e[0] = 0$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $e[i] = -\infty$ 
```

```
  return HAUPTSTANGENZERLEGUNG( $p, n, e$ )
```

```
HAUPTSTANGENZERLEGUNG(int[]  $p$ , int  $n$ , int[]  $e$ )
```

```
  if  $e[n] > -\infty$  then return  $e[n]$ 
```

```
   $q = -\infty$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $q = \max\{q, p[i] + \text{HAUPTSTANGENZERLEGUNG}(p, n - i, e)\}$ 
```

3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

```
   $e = \text{new int}[0 \dots n]$ 
```

```
   $e[0] = 0$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $e[i] = -\infty$ 
```

```
  return HAUPTSTANGENZERLEGUNG( $p, n, e$ )
```

```
HAUPTSTANGENZERLEGUNG(int[]  $p$ , int  $n$ , int[]  $e$ )
```

```
  if  $e[n] > -\infty$  then return  $e[n]$ 
```

```
   $q = -\infty$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $q = \max\{q, p[i] + \text{HAUPTSTANGENZERLEGUNG}(p, n - i, e)\}$ 
```

```
   $e[n] = q$ ; return  $q$ 
```

3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

```
   $e = \text{new int}[0 \dots n]$ 
```

```
   $e[0] = 0$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $e[i] = -\infty$ 
```

```
  return HAUPTSTANGENZERLEGUNG( $p, n, e$ )
```

```
HAUPTSTANGENZERLEGUNG(int[]  $p$ , int  $n$ , int[]  $e$ )
```

```
  if  $e[n] > -\infty$  then return  $e[n]$ 
```

```
   $q = -\infty$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $q = \max\{q, p[i] + \text{HAUPTSTANGENZERLEGUNG}(p, n - i, e)\}$ 
```

```
   $e[n] = q$ ; return  $q$ 
```

Laufzeit?

3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

```
   $e = \text{new int}[0 \dots n]$ 
```

```
   $e[0] = 0$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $e[i] = -\infty$ 
```

```
  return HAUPTSTANGENZERLEGUNG( $p, n, e$ )
```

```
HAUPTSTANGENZERLEGUNG(int[]  $p$ , int  $n$ , int[]  $e$ )
```

```
  if  $e[n] > -\infty$  then return  $e[n]$ 
```

```
   $q = -\infty$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $q = \max\{q, p[i] + \text{HAUPTSTANGENZERLEGUNG}(p, n - i, e)\}$ 
```

```
   $e[n] = q$ ; return  $q$ 
```

Laufzeit?

■ Wie letzte Folie?

3. Wert einer optimalen Lösung berechnen: *mit Tabelle*

Zeit-Speicher-Tausch (engl. *time-memory trade-off*)

```
MEMOSTANGENZERLEGUNG(int[]  $p$ , int  $n = p.length$ )
```

```
   $e = \text{new int}[0 \dots n]$ 
```

```
   $e[0] = 0$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $e[i] = -\infty$ 
```

```
  return HAUPTSTANGENZERLEGUNG( $p, n, e$ )
```

```
HAUPTSTANGENZERLEGUNG(int[]  $p$ , int  $n$ , int[]  $e$ )
```

```
  if  $e[n] > -\infty$  then return  $e[n]$ 
```

```
   $q = -\infty$ 
```

```
  for  $i = 1$  to  $n$  do
```

```
     $q = \max\{q, p[i] + \text{HAUPTSTANGENZERLEGUNG}(p, n - i, e)\}$ 
```

```
   $e[n] = q$ ; return  $q$ 
```

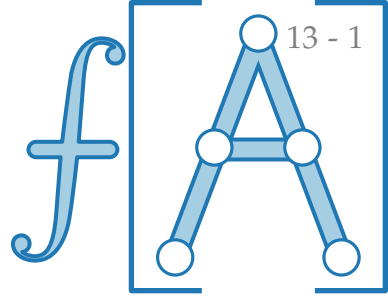
Laufzeit?

■ Wie letzte Folie?

■ Asymptotisch schneller?

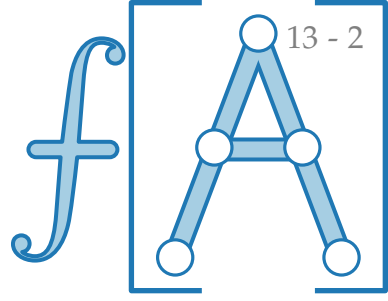
3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] *p*, int *n*)



3. Wert einer optimalen Lösung berechnen: *bottom-up*

```
BOTTOMUPSTANGENZERLEGUNG(int[] p, int n)  
    e = new int[0...n]
```

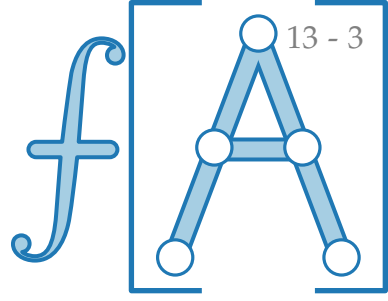


3. Wert einer optimalen Lösung berechnen: *bottom-up*

```
BOTTOMUPSTANGENZERLEGUNG(int[]  $p$ , int  $n$ )
```

```
     $e$  = new int[0... $n$ ]
```

```
     $e[0]$  = 0
```



3. Wert einer optimalen Lösung berechnen: *bottom-up*

```
BOTTOMUPSTANGENZERLEGUNG(int[]  $p$ , int  $n$ )
```

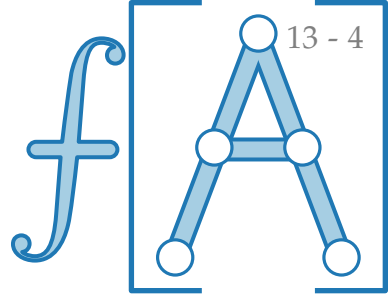
```
     $e$  = new int[0... $n$ ]
```

```
     $e[0]$  = 0
```

```
    for  $j$  = 1 to  $n$  do
```

```
        |
```

```
         $1$ ]
```



3. Wert einer optimalen Lösung berechnen: *bottom-up*

```
BOTTOMUPSTANGENZERLEGUNG(int[]  $p$ , int  $n$ )
```

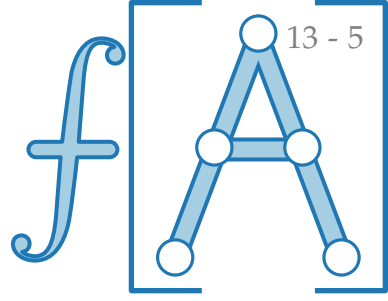
```
   $e$  = new int[0... $n$ ]
```

```
   $e[0]$  = 0
```

```
  for  $j$  = 1 to  $n$  do
```

```
     $q$  =  $-\infty$ 
```

```
    ]
```



3. Wert einer optimalen Lösung berechnen: *bottom-up*

```
BOTTOMUPSTANGENZERLEGUNG(int[]  $p$ , int  $n$ )
```

```
   $e$  = new int[0... $n$ ]
```

```
   $e[0]$  = 0
```

```
  for  $j$  = 1 to  $n$  do
```

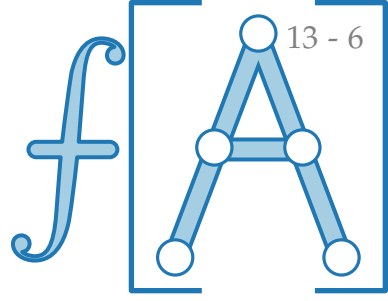
```
     $q$  =  $-\infty$ 
```

```
    for  $i$  = 1 to  $j$  do
```

```
      |
```

```
    |
```

```
       $i$ ]
```



3. Wert einer optimalen Lösung berechnen: *bottom-up*

```
BOTTOMUPSTANGENZERLEGUNG(int[]  $p$ , int  $n$ )
```

```
   $e$  = new int[0... $n$ ]
```

```
   $e[0] = 0$ 
```

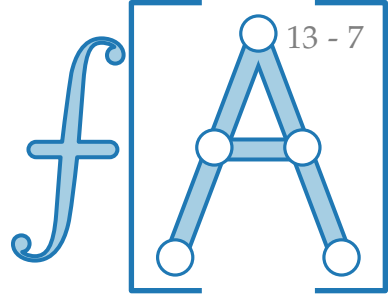
```
  for  $j = 1$  to  $n$  do
```

```
     $q = -\infty$ 
```

```
    for  $i = 1$  to  $j$  do
```

```
       $q = \max\{q, p[i] + e[j - i]\}$ 
```

```
  ]
```



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

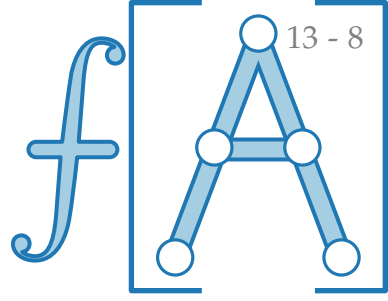
for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$
 1



3. Wert einer optimalen Lösung berechnen: *bottom-up*

```
BOTTOMUPSTANGENZERLEGUNG(int[]  $p$ , int  $n$ )
```

```
   $e$  = new int[0... $n$ ]
```

```
   $e[0]$  = 0
```

```
  for  $j$  = 1 to  $n$  do
```

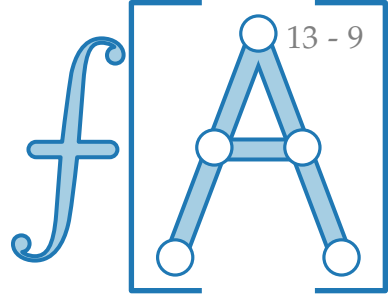
```
     $q$  =  $-\infty$ 
```

```
    for  $i$  = 1 to  $j$  do
```

```
       $q$  =  $\max\{q, p[i] + e[j - i]\}$ 
```

```
     $e[j]$  =  $q$ 
```

```
  return  $e[n]$ 
```



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

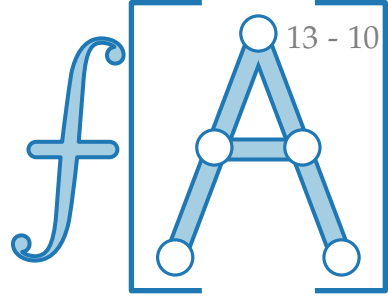
 for $i = 1$ to j do

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

return $e[n]$

Neu: *kein*
rekursiver
Aufruf!

④

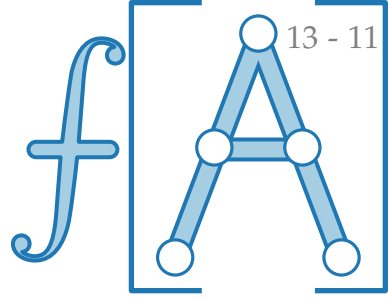
③

②

①

①

Graph der
Teilinstanzen



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

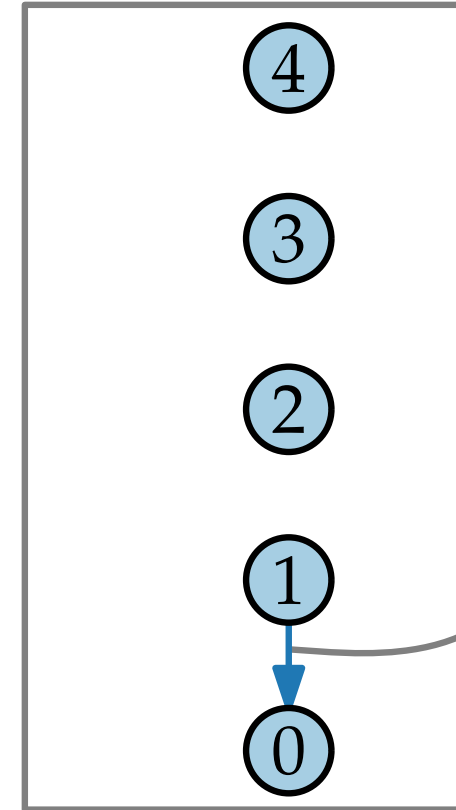
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

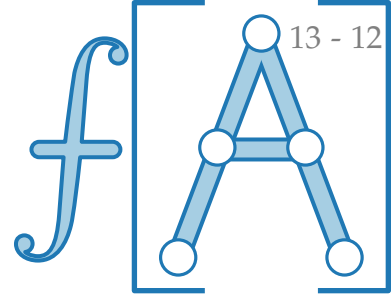
return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

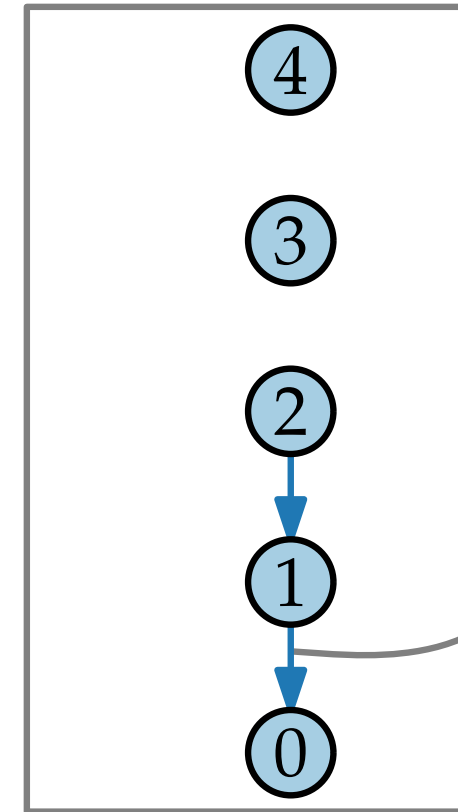
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

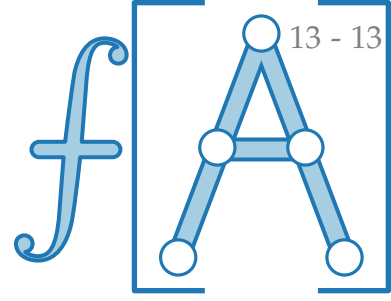
return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

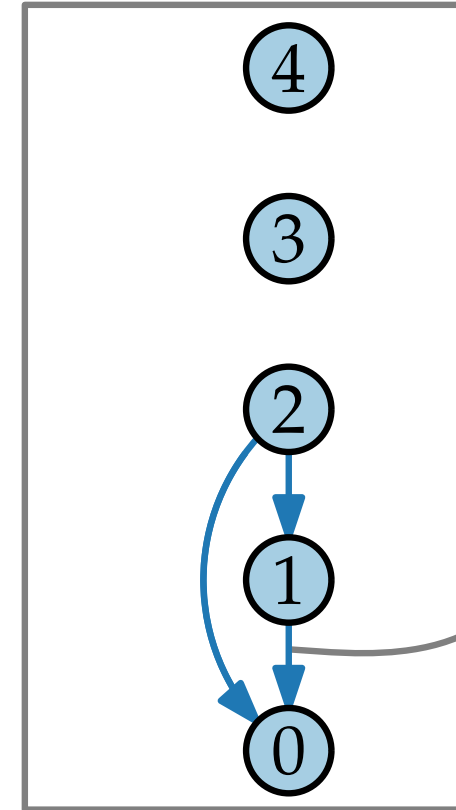
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .

3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

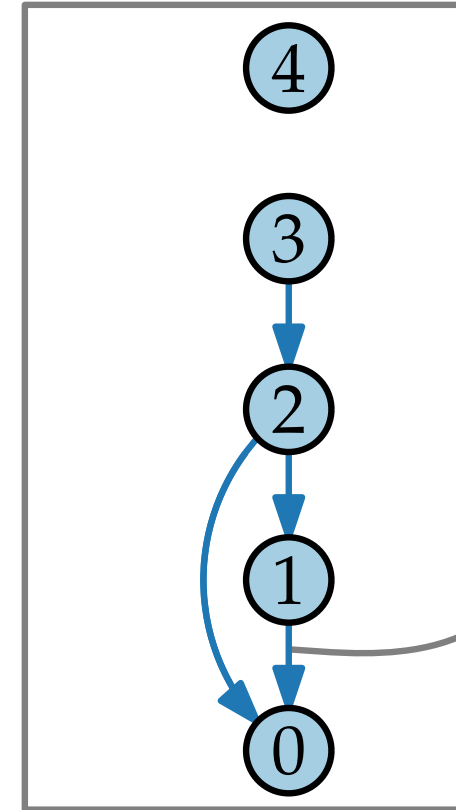
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

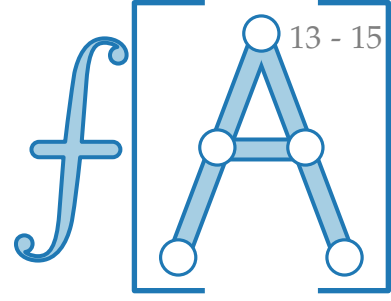
return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

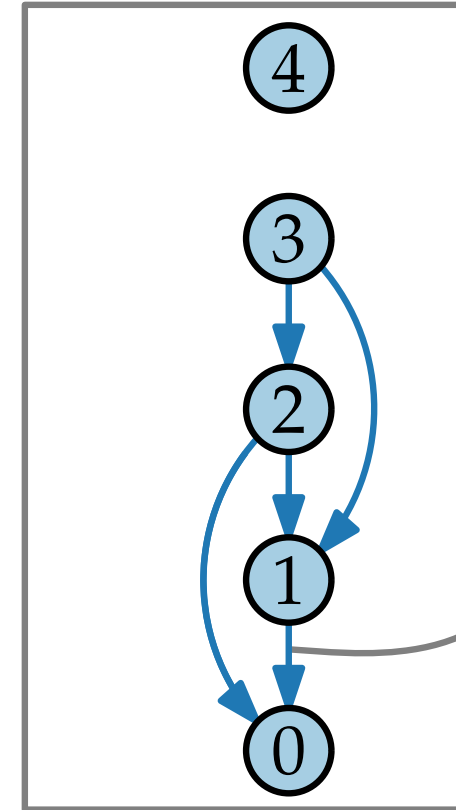
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

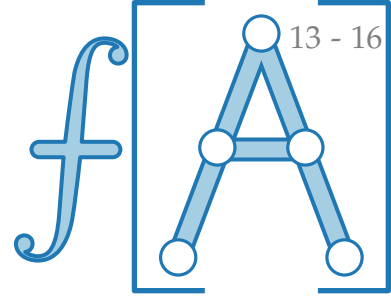
return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

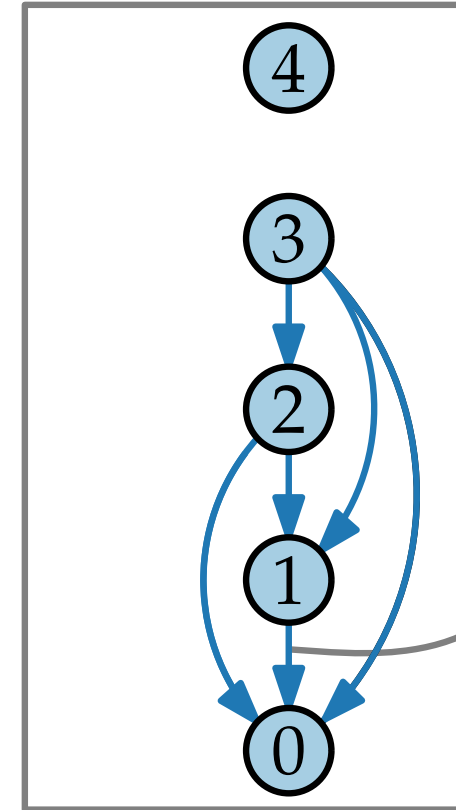
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

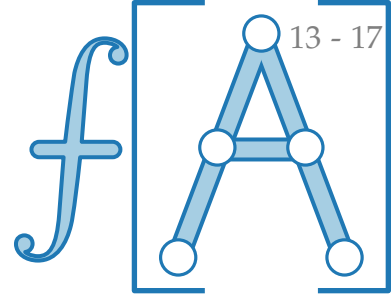
return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

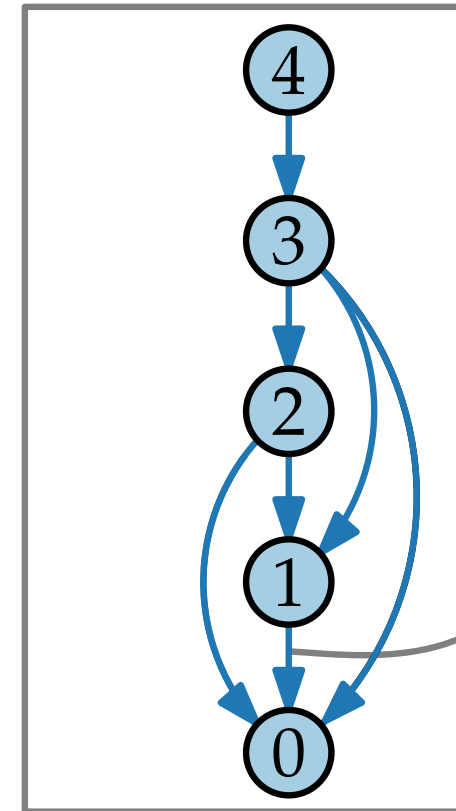
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .

3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

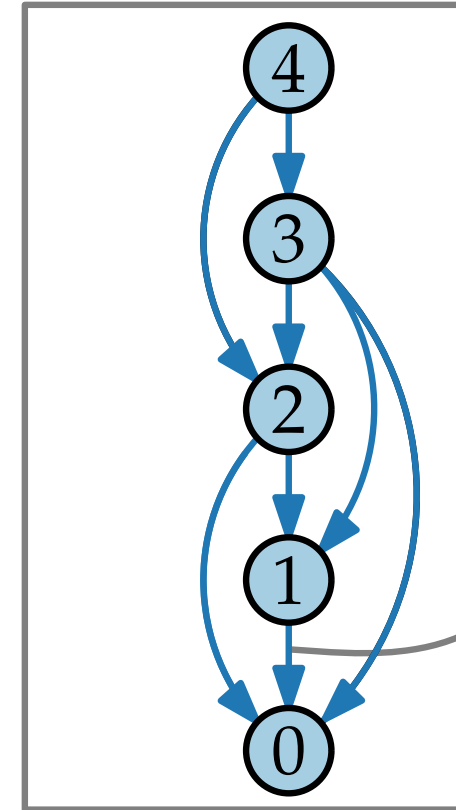
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

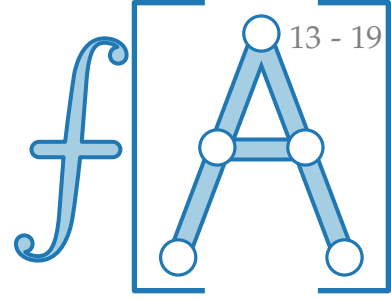
return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .



3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

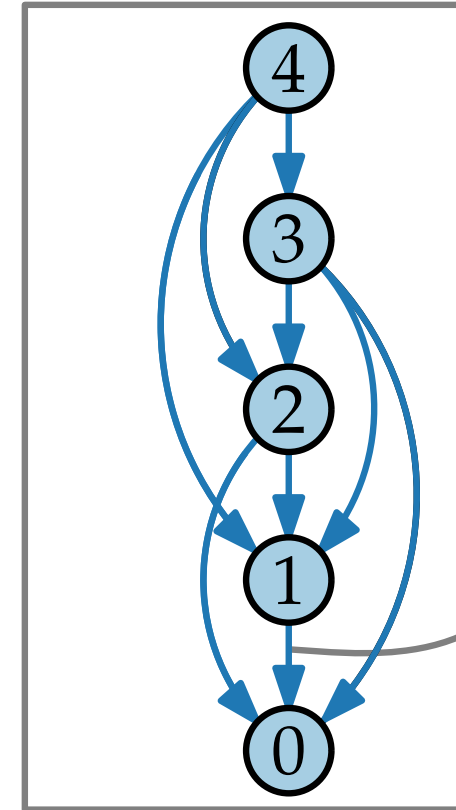
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .

3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

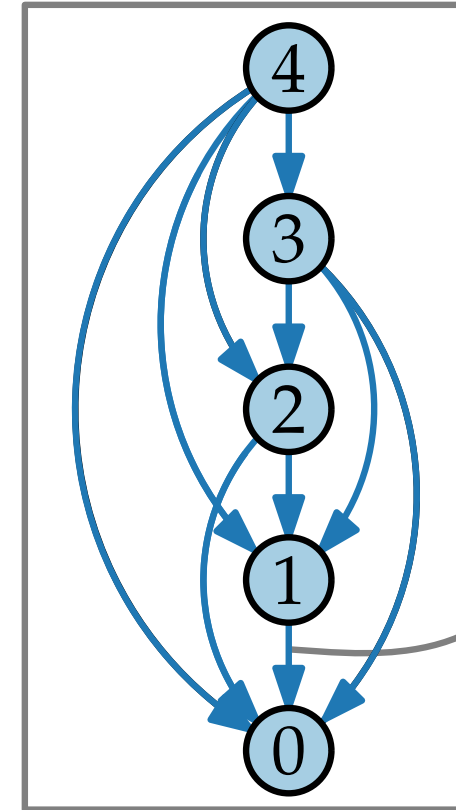
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .

3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

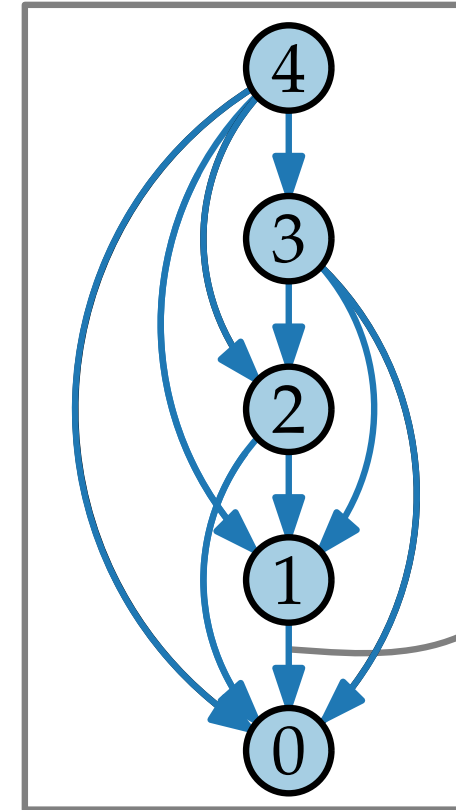
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .

Beob. Die Anzahl der Kanten im Graphen ist proportional zur Laufzeit des DP (Anz. Additionen).

3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

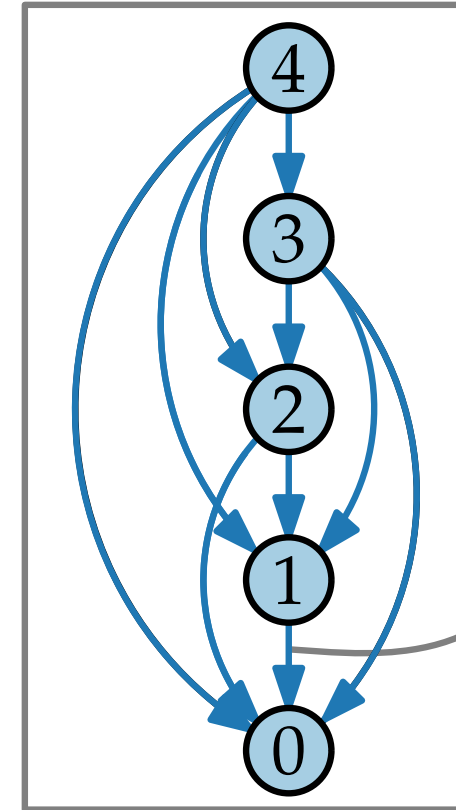
 for $i = 1$ to j do

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .

Beob. Die Anzahl der Kanten im Graphen ist proportional zur Laufzeit des DP (Anz. Additionen).

Satz. BOTTOMUPSZERL() und MEMOSZERL() laufen in $\mathcal{O}(\quad)$ Zeit.

3. Wert einer optimalen Lösung berechnen: *bottom-up*

BOTTOMUPSTANGENZERLEGUNG(int[] p , int n)

$e = \text{new int}[0 \dots n]$

$e[0] = 0$

for $j = 1$ **to** n **do**

$q = -\infty$

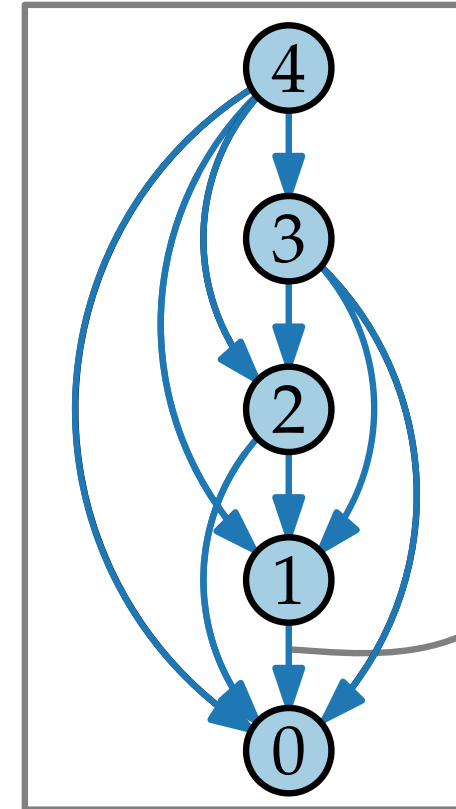
for $i = 1$ **to** j **do**

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

return $e[n]$

Neu: *kein*
rekursiver
Aufruf!



Graph der
Teilinstanzen

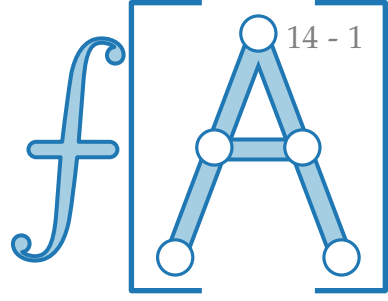
Kante (j, i) bedeutet:
Teilinstanz j benutzt
Wert einer opt.
Lösung von
Teilinstanz i .

Beob. Die Anzahl der Kanten im Graphen ist proportional zur Laufzeit des DP (Anz. Additionen).

Satz. BOTTOMUPSZERL() und MEMOSZERL() laufen in $\mathcal{O}(n^2)$ Zeit.

4. Optimale Lösung aus berechneten Informationen konstruieren

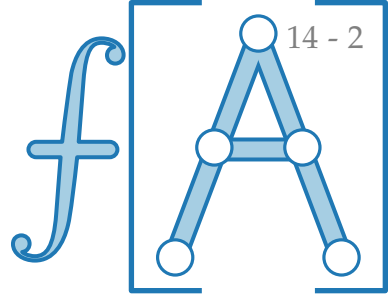
ERWEITERTEBOTTOMUPZERLEGUNG(int[] *p*, int[] *e*, int *n*)



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int n)

$e[0] = 0$



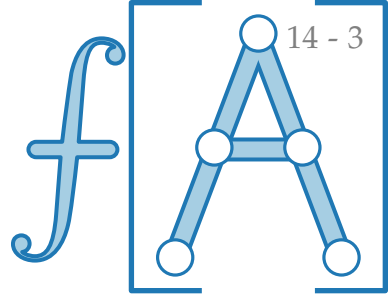
4. Optimale Lösung aus berechneten Informationen konstruieren

```
ERWEITERTEBOTTOMUPZERLEGUNG(int[] p, int[] e, int n)
```

```
  e[0] = 0
```

```
  for j = 1 to n do
```

```
    |
```



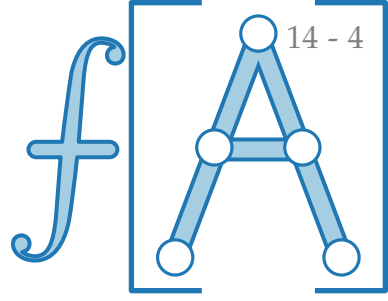
4. Optimale Lösung aus berechneten Informationen konstruieren

```
ERWEITERTEBOTTOMUPZERLEGUNG(int[]  $p$ , int[]  $e$ , int  $n$ )
```

```
   $e[0] = 0$ 
```

```
  for  $j = 1$  to  $n$  do
```

```
     $q = -\infty$ 
```



$$e[0] = 0$$
$$q = -\infty$$

```
for  $i = 1$  to  $j$  do
```

4. Optimale Lösung aus berechneten Informationen konstruieren

```
ERWEITERTEBOTTOMUPZERLEGUNG(int[]  $p$ , int[]  $e$ , int  $n$ )
```

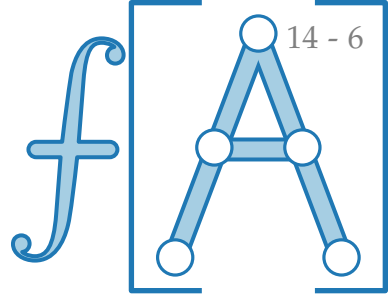
```
   $e[0] = 0$ 
```

```
  for  $j = 1$  to  $n$  do
```

```
     $q = -\infty$ 
```

```
    for  $i = 1$  to  $j$  do
```

```
       $q = \max\{q, p[i] + e[j - i]\}$ 
```



4. Optimale Lösung aus berechneten Informationen konstruieren

```
ERWEITERTEBOTTOMUPZERLEGUNG(int[]  $p$ , int[]  $e$ , int  $n$ )
```

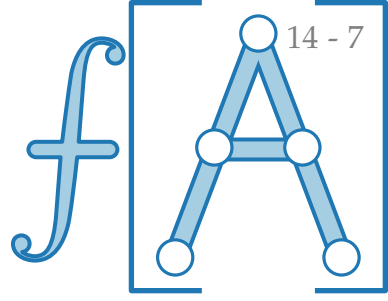
```
 $e[0] = 0$ 
```

```
for  $j = 1$  to  $n$  do
```

```
     $q = -\infty$ 
```

```
    for  $i = 1$  to  $j$  do
```

```
         $q = \max\{q, p[i] + e[j - i]\}$ 
```



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int n)

$e[0] = 0$

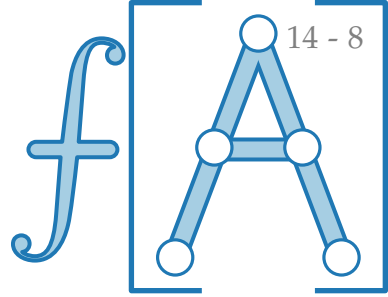
for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = \max\{q, p[i] + e[j - i]\}$



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int n)

$e[0] = 0$

for $j = 1$ to n do

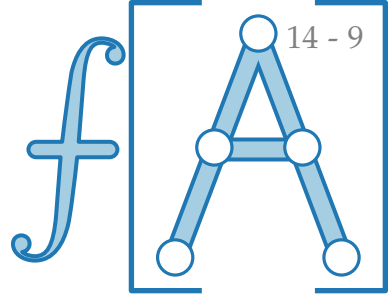
$q = -\infty$

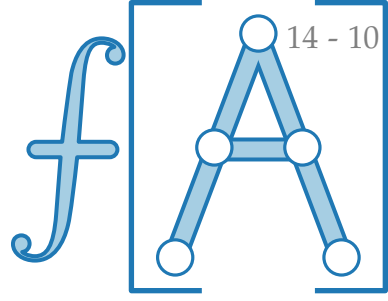
 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$q = \max\{q, p[i] + e[j - i]\}$





4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

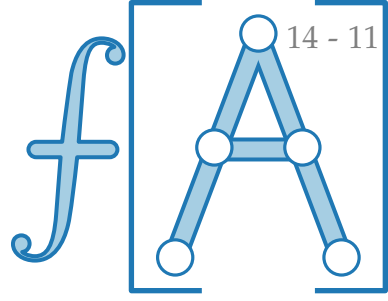
 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$q = \max\{q, p[i] + e[j - i]\}$

// merke Länge des i . Teilstücks



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

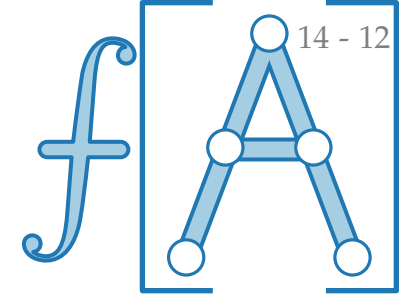
 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

// merke Länge des i . Teilstücks



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

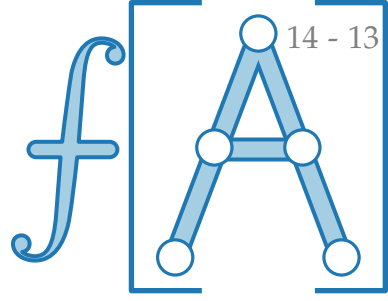
 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

// merke Länge des i . Teilstücks



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

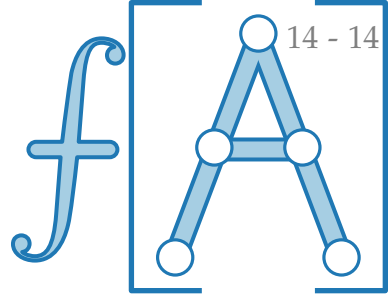
$q = p[i] + e[j - i]$

$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

// merke Länge des i . Teilstücks



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

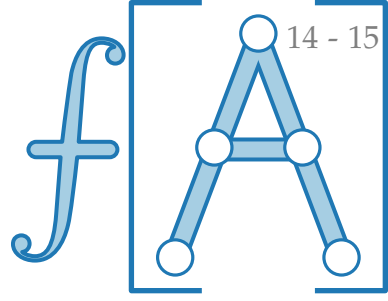
$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

// merke Länge des i . Teilstücks

GIBZERLEGUNGAUS(int[] p , int n)



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$\ell[j] = i$

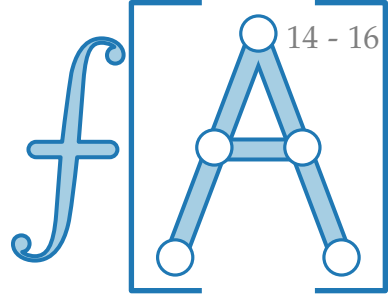
$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

// merke Länge des i . Teilstücks

GIBZERLEGUNGAUS(int[] p , int n)

$\ell = \text{new int}[0 \dots n]$; $e = \text{new int}[0 \dots n]$



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

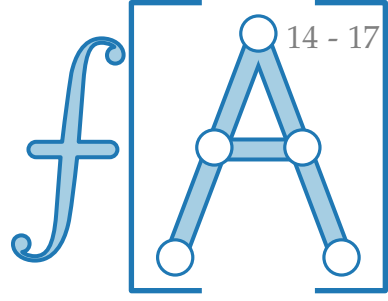
$e[j] = q$

// merke Länge des i . Teilstücks

GIBZERLEGUNGAUS(int[] p , int n)

$\ell = \text{new int}[0 \dots n]$; $e = \text{new int}[0 \dots n]$

ERWEITERTEBOTTOMUPZERLEGUNG(p, e, ℓ, n)



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

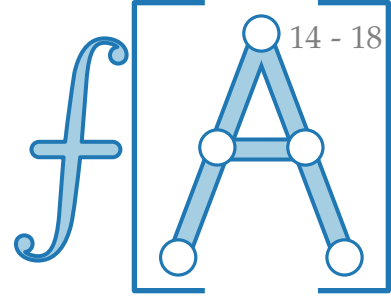
// merke Länge des i . Teilstücks

GIBZERLEGUNGAUS(int[] p , int n)

$\ell = \text{new int}[0 \dots n]$; $e = \text{new int}[0 \dots n]$

ERWEITERTEBOTTOMUPZERLEGUNG(p, e, ℓ, n)

while $n > 0$ do



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

// merke Länge des i . Teilstücks

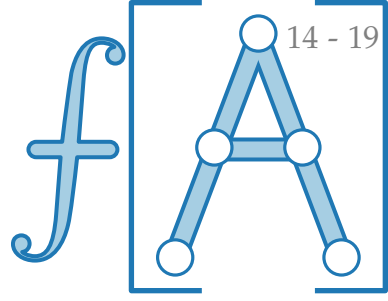
GIBZERLEGUNGAUS(int[] p , int n)

$\ell = \text{new int}[0 \dots n]$; $e = \text{new int}[0 \dots n]$

ERWEITERTEBOTTOMUPZERLEGUNG(p, e, ℓ, n)

while $n > 0$ do

 print $\ell[n]$; $n = n - \ell[n]$



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

// merke Länge des i . Teilstücks

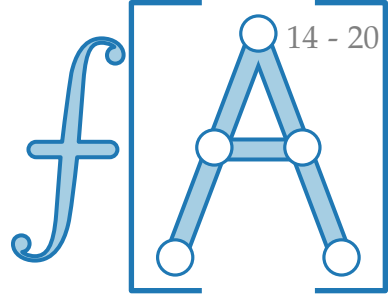
GIBZERLEGUNGAUS(int[] p , int n)

$\ell = \text{new int}[0 \dots n]$; $e = \text{new int}[0 \dots n]$

ERWEITERTEBOTTOMUPZERLEGUNG(p, e, ℓ, n)

while $n > 0$ do

 print $\ell[n]$; $n = n - \ell[n]$ // gib wiederholt Länge des i . Teilstücks aus



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

// merke Länge des i . Teilstücks



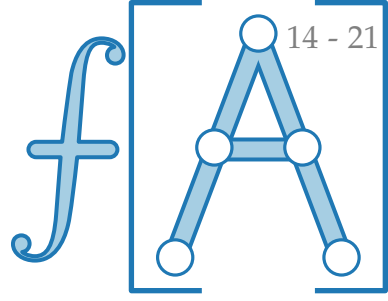
GIBZERLEGUNGAUS(int[] p , int n)

$\ell = \text{new int}[0 \dots n]$; $e = \text{new int}[0 \dots n]$

ERWEITERTEBOTTOMUPZERLEGUNG(p, e, ℓ, n)

while $n > 0$ do

 print $\ell[n]$; $n = n - \ell[n]$ // gib wiederholt Länge des i . Teilstücks aus



4. Optimale Lösung aus berechneten Informationen konstruieren

ERWEITERTEBOTTOMUPZERLEGUNG(int[] p , int[] e , int[] ℓ , int n)

$e[0] = 0$

for $j = 1$ to n do

$q = -\infty$

 for $i = 1$ to j do

 if $q < p[i] + e[j - i]$ then

$q = p[i] + e[j - i]$

$\ell[j] = i$

$q = \max\{q, p[i] + e[j - i]\}$

$e[j] = q$

// merke Länge des i . Teilstücks



GIBZERLEGUNGAUS(int[] p , int n)

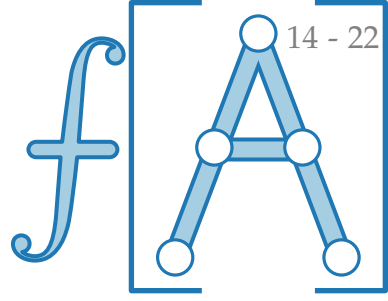
$\ell = \text{new int}[0 \dots n]$; $e = \text{new int}[0 \dots n]$

ERWEITERTEBOTTOMUPZERLEGUNG(p, e, ℓ, n)

while $n > 0$ do

 print $\ell[n]$; $n = n - \ell[n]$ // gib wiederholt Länge des i . Teilstücks aus

4. Optimale Lösung aus berechneten Informationen konstruieren



```
ERWEITERTEBOTTOMUPZERLEGUNG(int[]  $p$ , int[]  $e$ , int[]  $\ell$ , int  $n$ )
```

```
 $e[0] = 0$ 
```

```
for  $j = 1$  to  $n$  do
```

```
     $q = -\infty$ 
```

```
    for  $i = 1$  to  $j$  do
```

```
        if  $q < p[i] + e[j - i]$  then
```

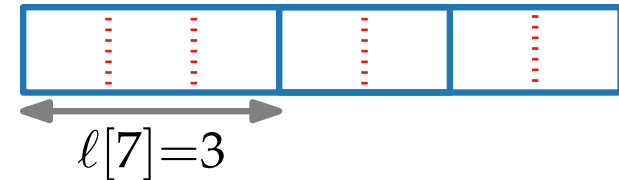
```
             $q = p[i] + e[j - i]$ 
```

```
             $\ell[j] = i$ 
```

```
        }  $q = \max\{q, p[i] + e[j - i]\}$ 
```

```
     $e[j] = q$ 
```

// merke Länge des i . Teilstücks



```
GIBZERLEGUNGAUS(int[]  $p$ , int  $n$ )
```

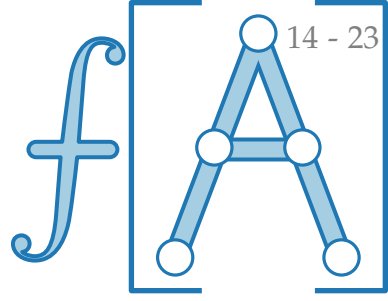
```
 $\ell = \text{new int}[0 \dots n]$ ;  $e = \text{new int}[0 \dots n]$ 
```

```
ERWEITERTEBOTTOMUPZERLEGUNG( $p, e, \ell, n$ )
```

```
while  $n > 0$  do
```

```
    print  $\ell[n]$ ;  $n = n - \ell[n]$  // gib wiederholt Länge des  $i$ . Teilstücks aus
```

4. Optimale Lösung aus berechneten Informationen konstruieren



```
ERWEITERTEBOTTOMUPZERLEGUNG(int[]  $p$ , int[]  $e$ , int[]  $\ell$ , int  $n$ )
```

```
 $e[0] = 0$ 
```

```
for  $j = 1$  to  $n$  do
```

```
     $q = -\infty$ 
```

```
    for  $i = 1$  to  $j$  do
```

```
        if  $q < p[i] + e[j - i]$  then
```

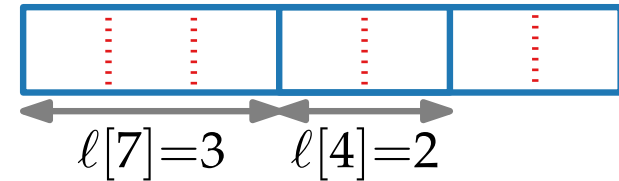
```
             $q = p[i] + e[j - i]$ 
```

```
             $\ell[j] = i$ 
```

```
        }  $q = \max\{q, p[i] + e[j - i]\}$ 
```

```
     $e[j] = q$ 
```

```
// merke Länge des  $i$ . Teilstücks
```



```
GIBZERLEGUNGAUS(int[]  $p$ , int  $n$ )
```

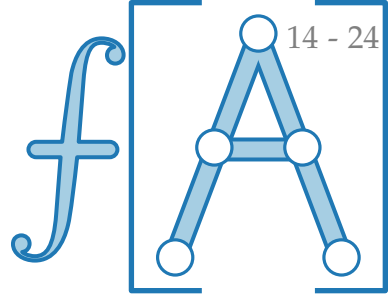
```
 $\ell = \text{new int}[0 \dots n]$ ;  $e = \text{new int}[0 \dots n]$ 
```

```
ERWEITERTEBOTTOMUPZERLEGUNG( $p, e, \ell, n$ )
```

```
while  $n > 0$  do
```

```
    print  $\ell[n]$ ;  $n = n - \ell[n]$  // gib wiederholt Länge des  $i$ . Teilstücks aus
```

4. Optimale Lösung aus berechneten Informationen konstruieren



```
ERWEITERTEBOTTOMUPZERLEGUNG(int[]  $p$ , int[]  $e$ , int[]  $\ell$ , int  $n$ )
```

```
 $e[0] = 0$ 
```

```
for  $j = 1$  to  $n$  do
```

```
     $q = -\infty$ 
```

```
    for  $i = 1$  to  $j$  do
```

```
        if  $q < p[i] + e[j - i]$  then
```

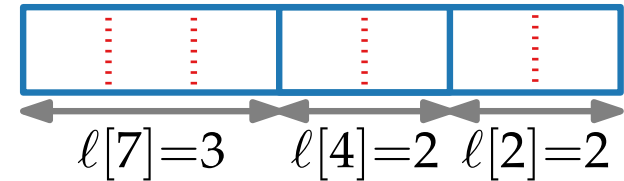
```
             $q = p[i] + e[j - i]$ 
```

```
             $\ell[j] = i$ 
```

```
        }  $q = \max\{q, p[i] + e[j - i]\}$ 
```

```
     $e[j] = q$ 
```

```
// merke Länge des  $i$ . Teilstücks
```



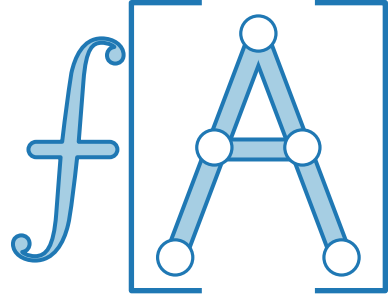
```
GIBZERLEGUNGAUS(int[]  $p$ , int  $n$ )
```

```
 $\ell = \text{new int}[0 \dots n]$ ;  $e = \text{new int}[0 \dots n]$ 
```

```
ERWEITERTEBOTTOMUPZERLEGUNG( $p, e, \ell, n$ )
```

```
while  $n > 0$  do
```

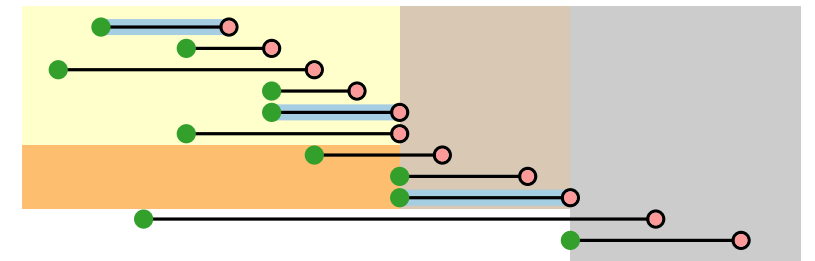
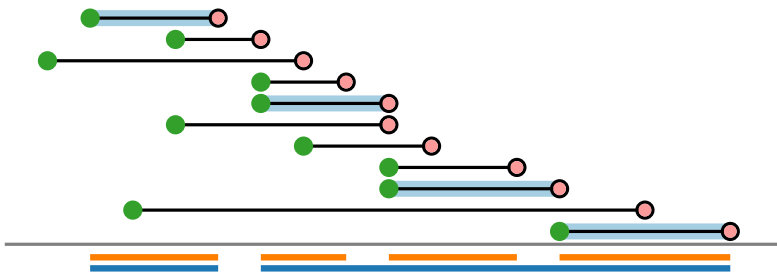
```
    print  $\ell[n]$ ;  $n = n - \ell[n]$  // gib wiederholt Länge des  $i$ . Teilstücks aus
```



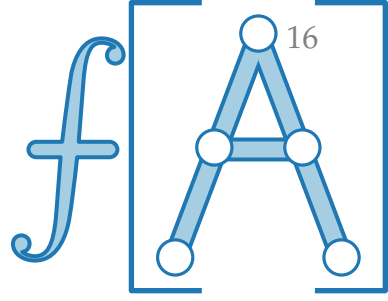
Fortgeschrittene Algorithmen

2. Vorlesung Dynamisches Programmieren

Teil III: Ablaufplanung

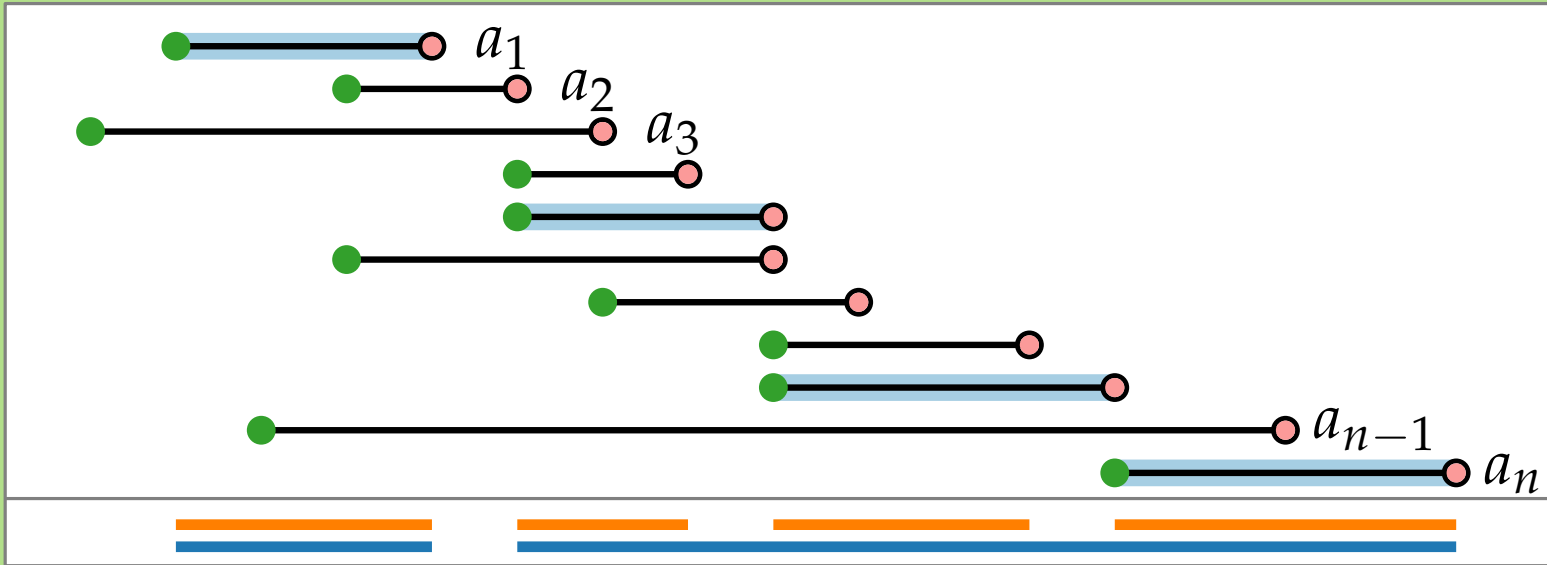


Erinnerung – Ablaufplanung'



Problem. ABLAUFPLANUNG'

Geg. Menge $A = \{a_1, \dots, a_n\}$ von **halboffenen Intervallen**, wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.

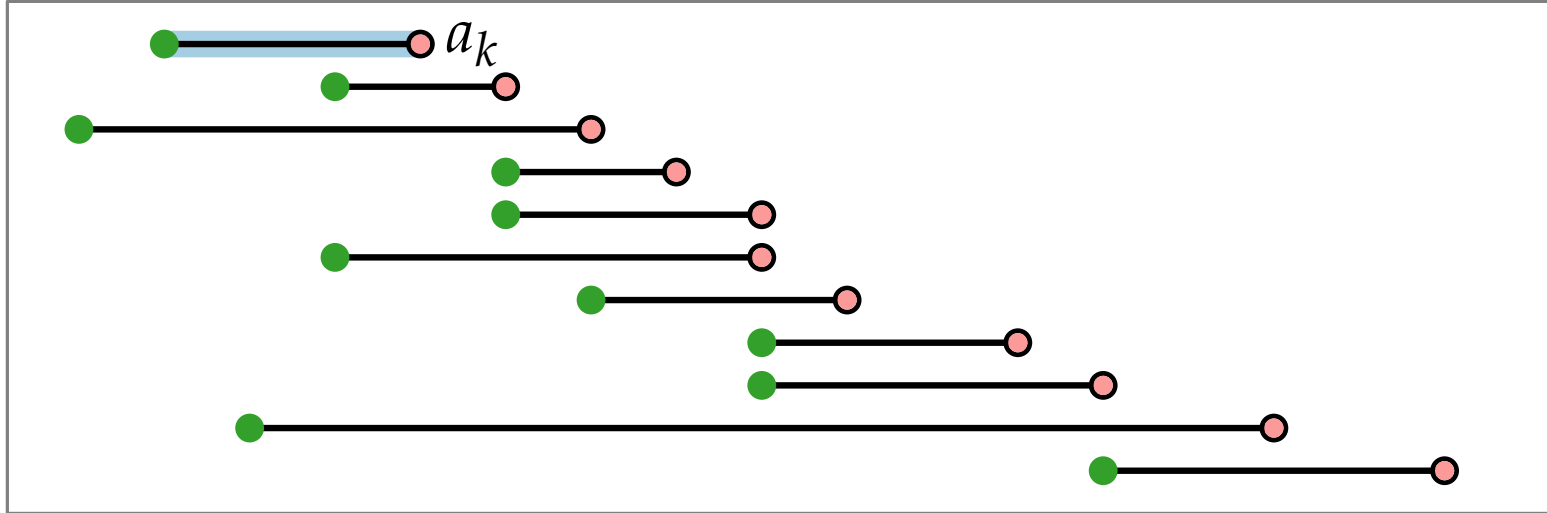


Ges. eine Menge $A' \subseteq A$ paarweise disjunkter Intervalle, deren **Gesamtlänge $\ell(A')$ maximal** ist.

Grund. Intervalle $\hat{=}$ Prozesse, die die gleiche Ressource nutzen; der Gesamtertrag ist proportional zur Auslastung.

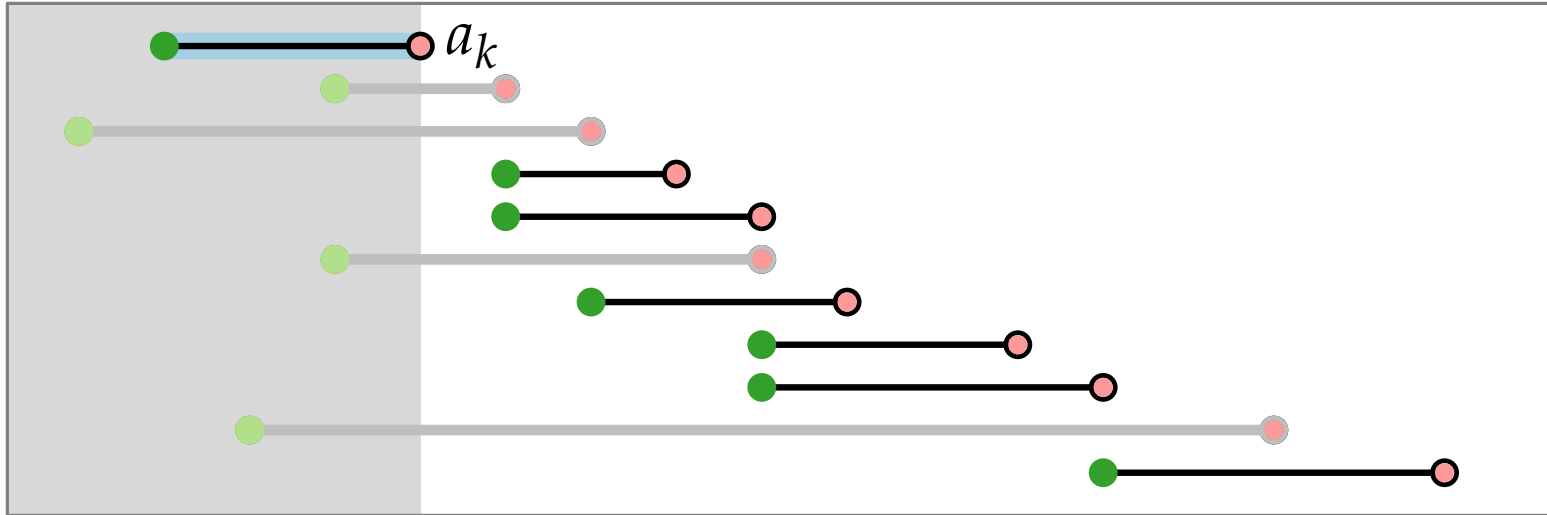
Ein exakter Algorithmus...

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



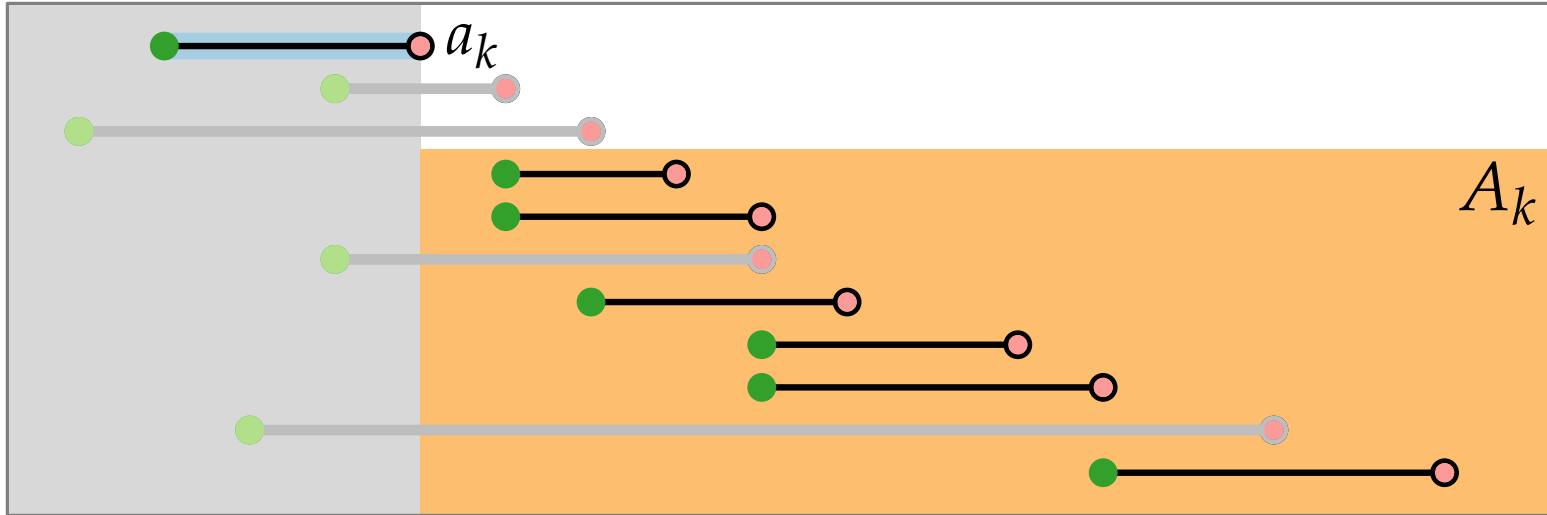
Ein exakter Algorithmus...

Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)

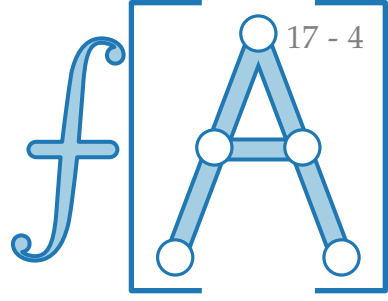


Ein exakter Algorithmus...

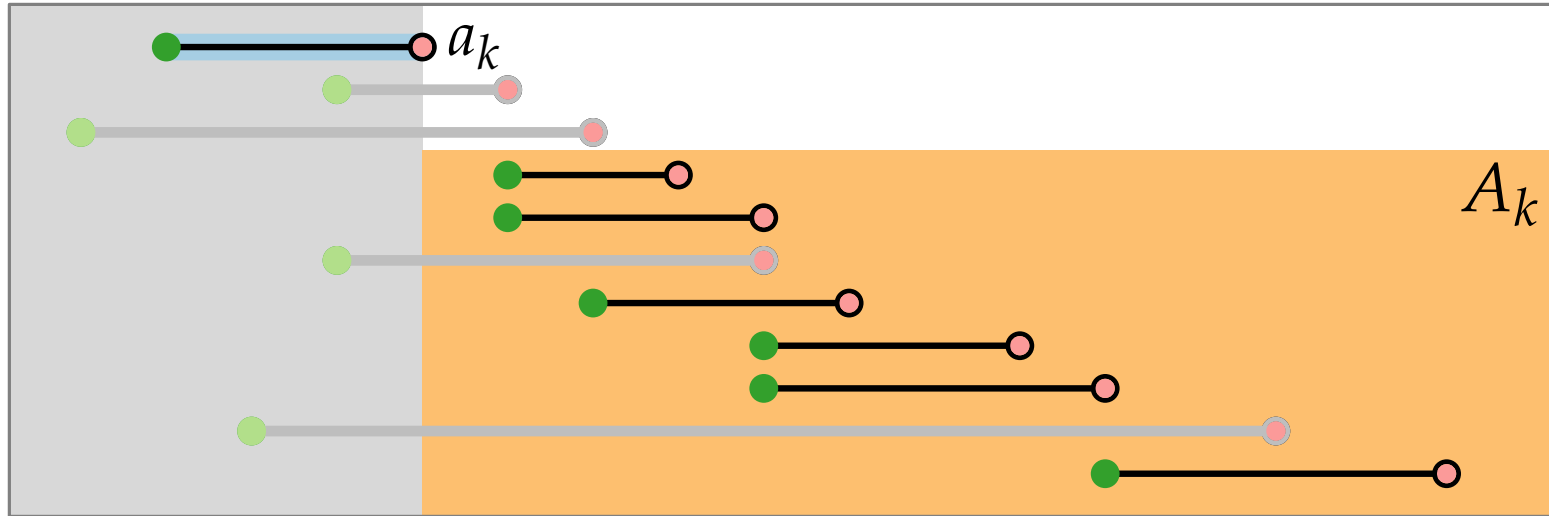
Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



Ein exakter Algorithmus...

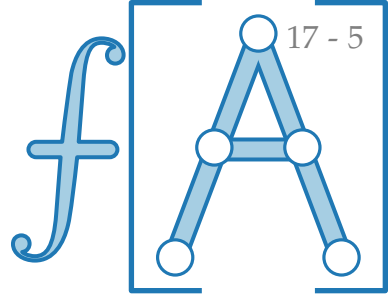


Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)

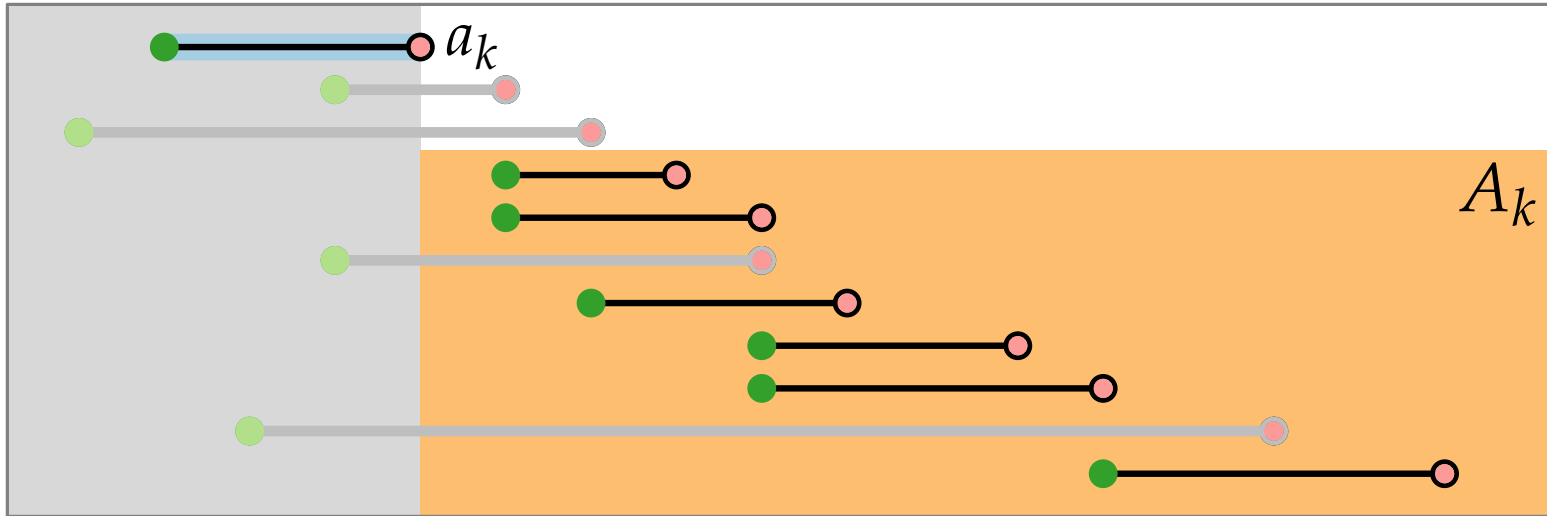


Eine optimale Lösung für A_k besteht aus:

Ein exakter Algorithmus...



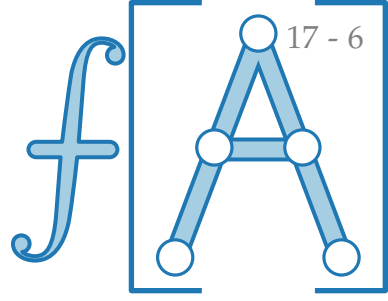
Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



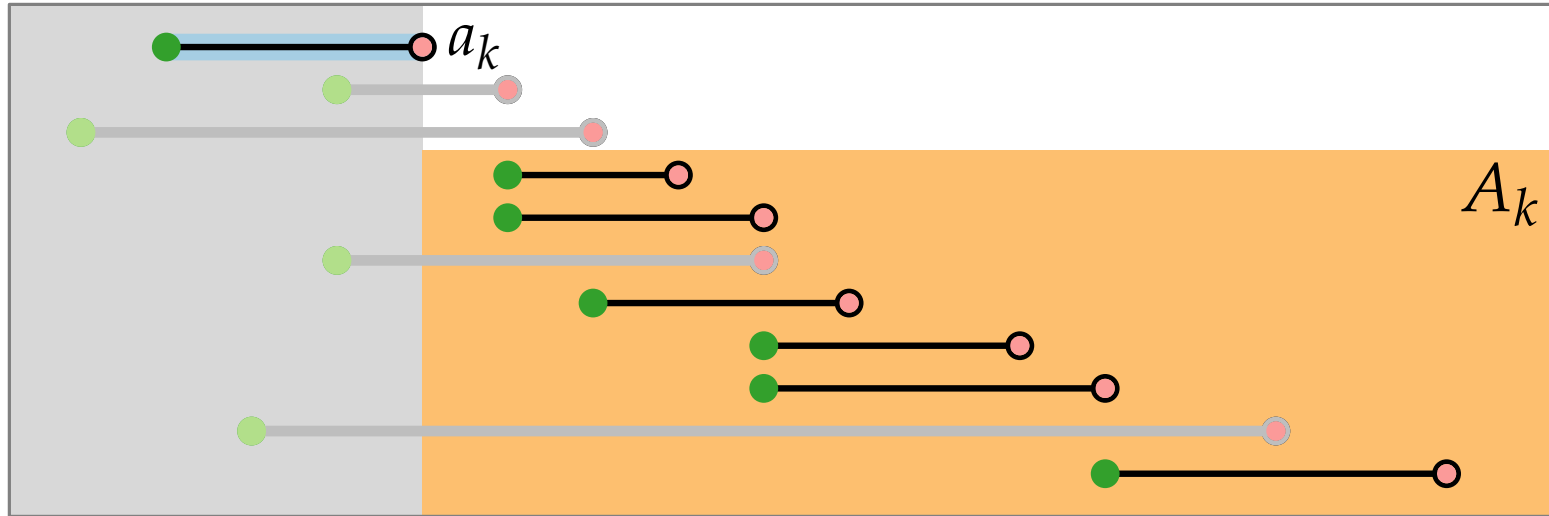
Eine optimale Lösung für A_k besteht aus:

- **einem** ersten Intervall a_m und

Ein exakter Algorithmus...



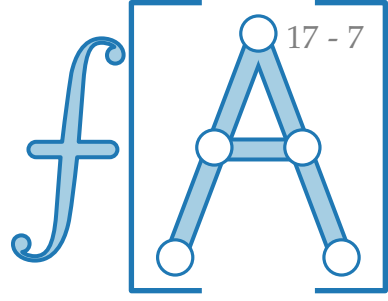
Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



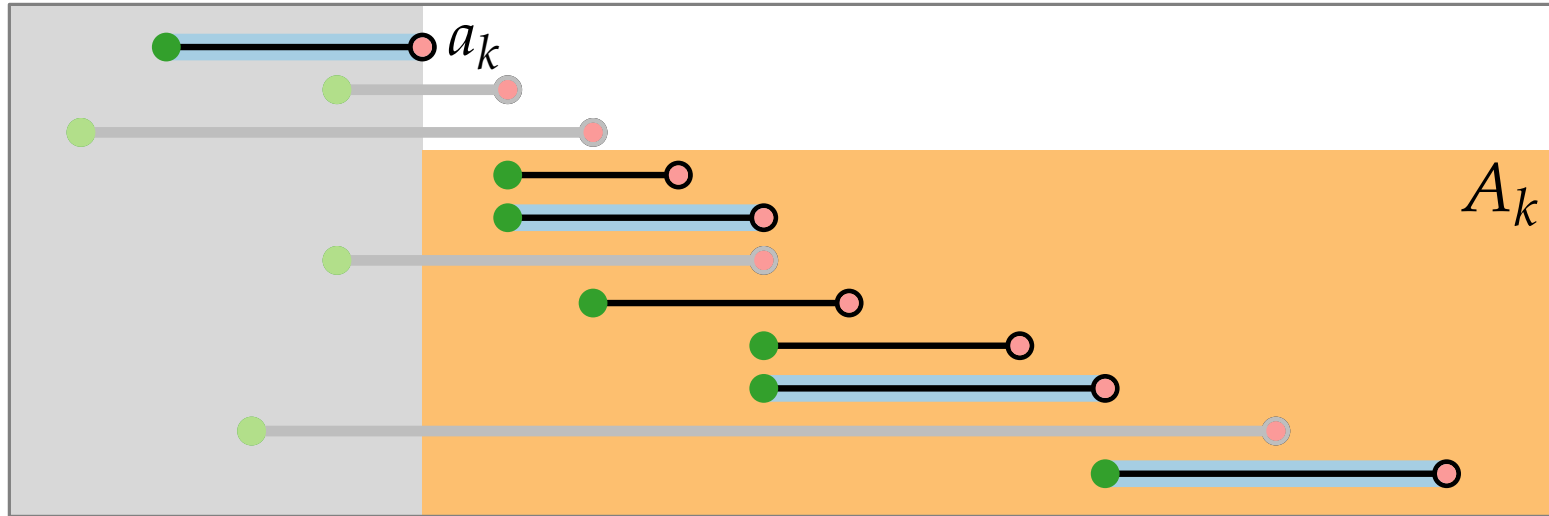
Eine optimale Lösung für A_k besteht aus:

- **einem** ersten Intervall a_m und
- einer optimalen Lösung für A_m .

Ein exakter Algorithmus...



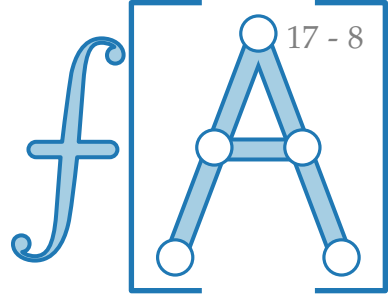
Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



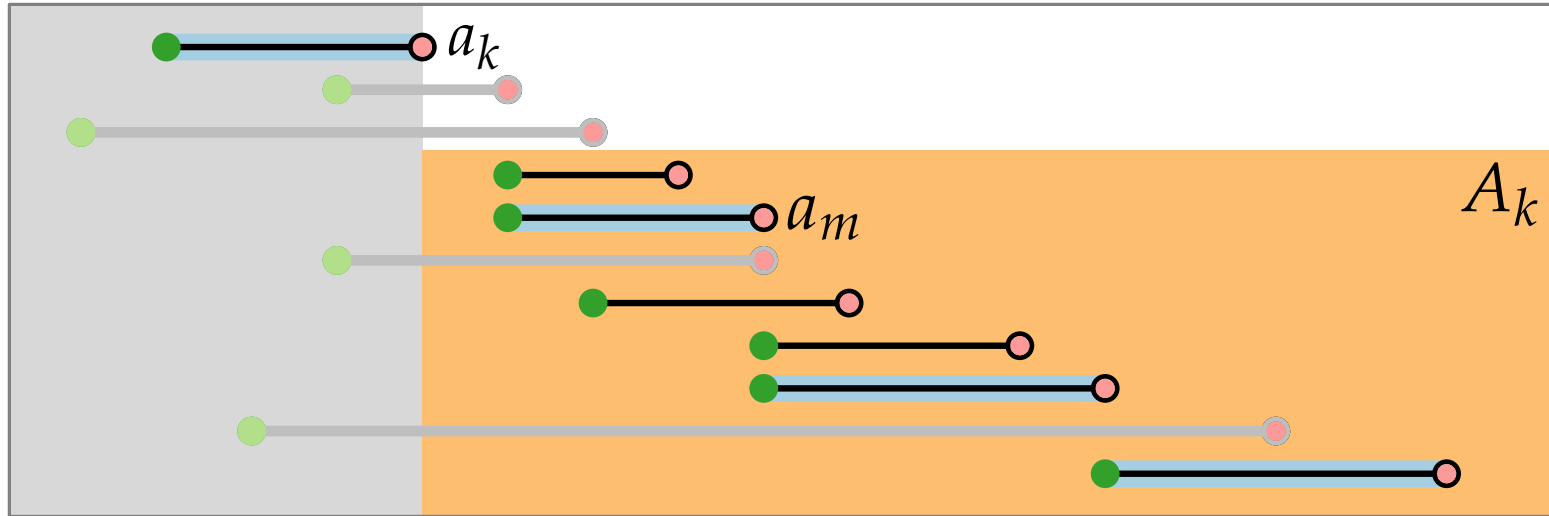
Eine optimale Lösung für A_k besteht aus:

- **einem** ersten Intervall a_m und
- einer optimalen Lösung für A_m .

Ein exakter Algorithmus...



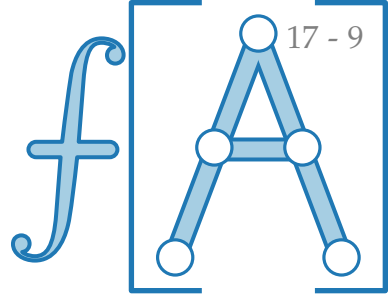
Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



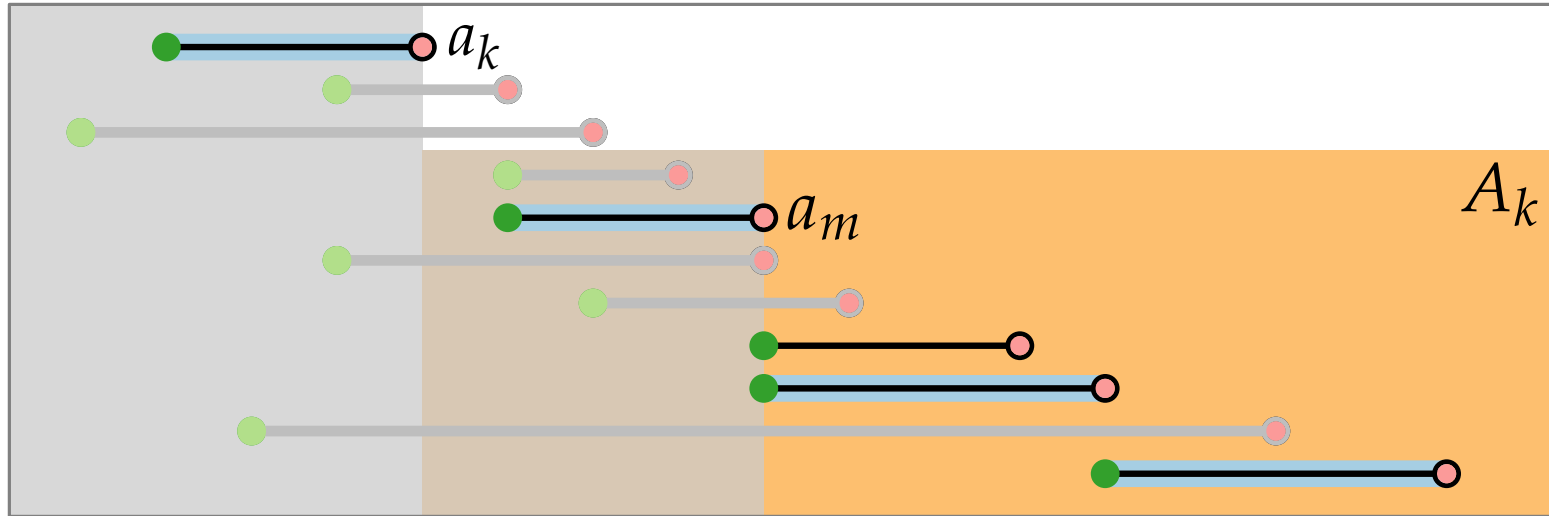
Eine optimale Lösung für A_k besteht aus:

- **einem** ersten Intervall a_m und
- einer optimalen Lösung für A_m .

Ein exakter Algorithmus...



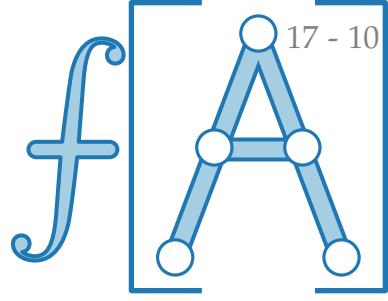
Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



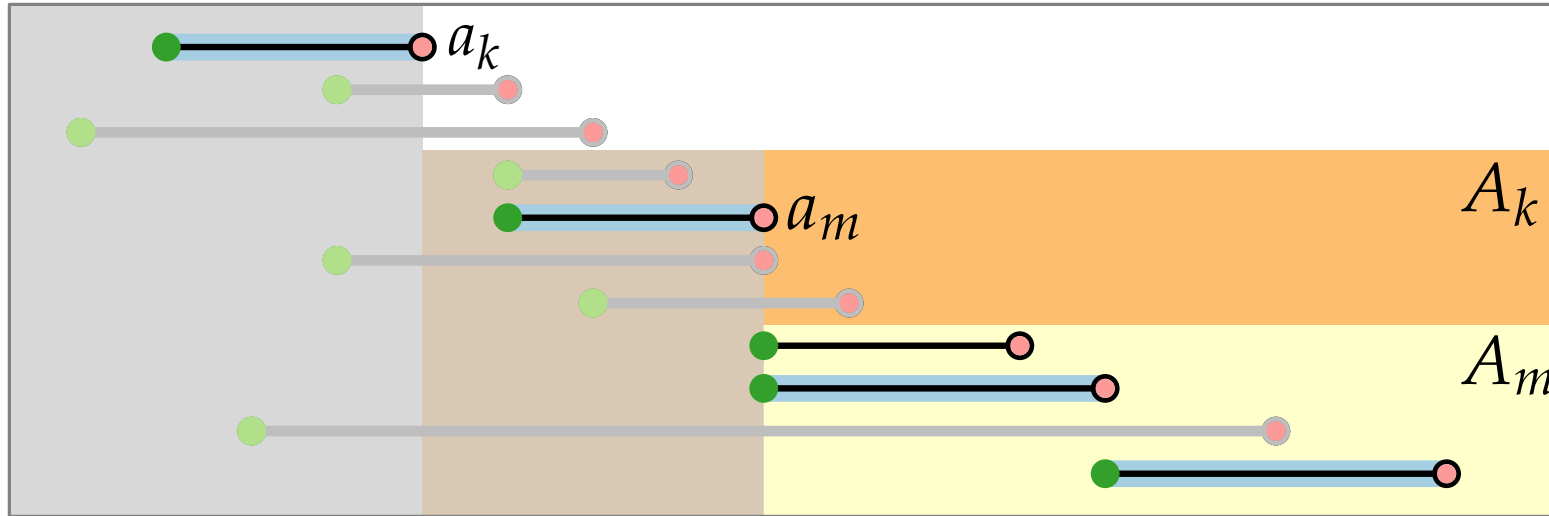
Eine optimale Lösung für A_k besteht aus:

- **einem** ersten Intervall a_m und
- einer optimalen Lösung für A_m .

Ein exakter Algorithmus...



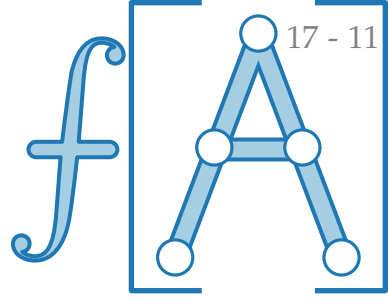
Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



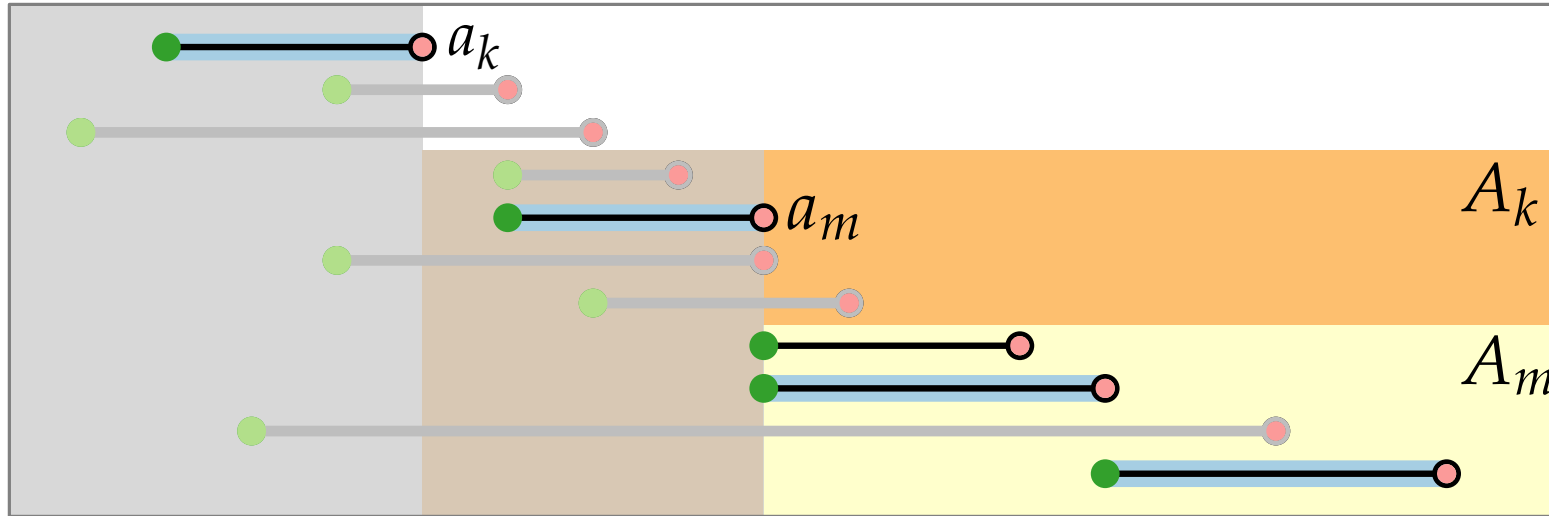
Eine optimale Lösung für A_k besteht aus:

- **einem** ersten Intervall a_m und
- einer optimalen Lösung für A_m .

Ein exakter Algorithmus...



Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)

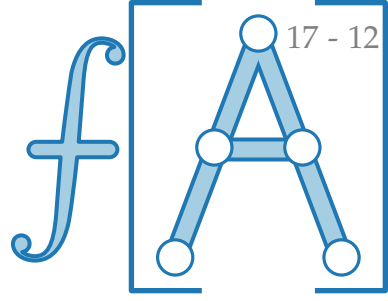


Eine optimale Lösung für A_k besteht aus:

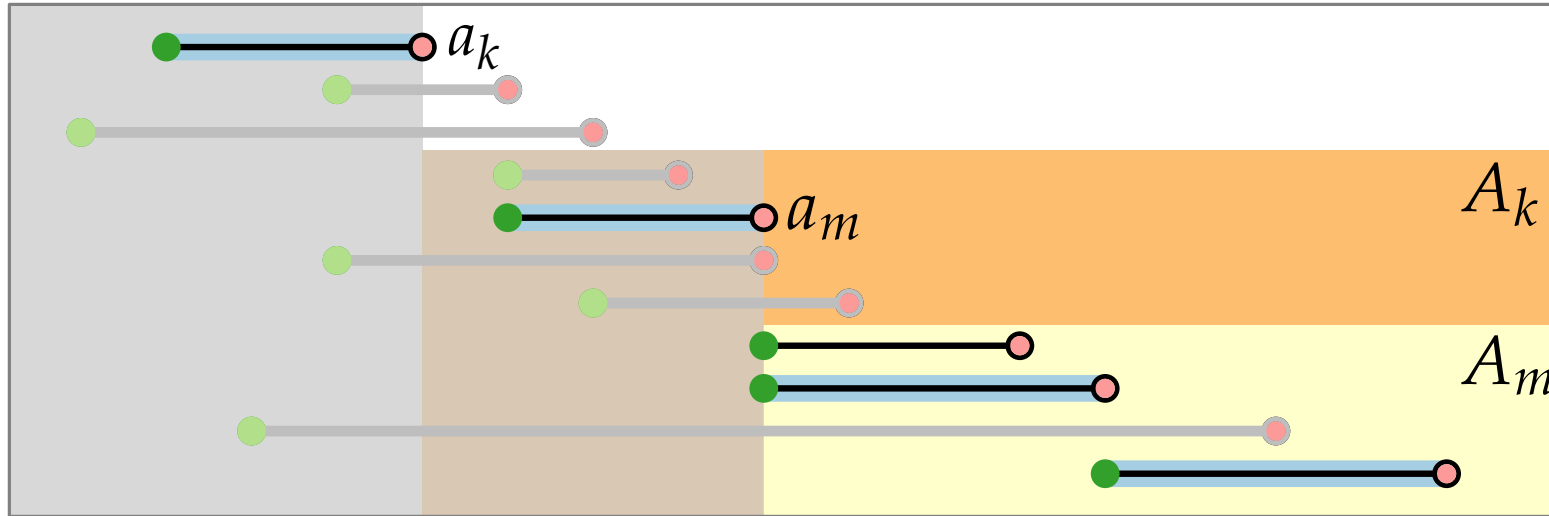
- **einem** ersten Intervall a_m und
- einer optimalen Lösung für A_m .

} optimale Teilstruktur!

Ein exakter Algorithmus...



Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



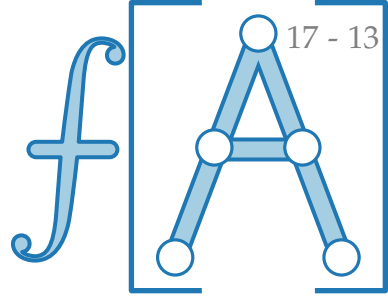
Eine optimale Lösung für A_k besteht aus:

- **einem** ersten Intervall a_m und
- einer optimalen Lösung für A_m .

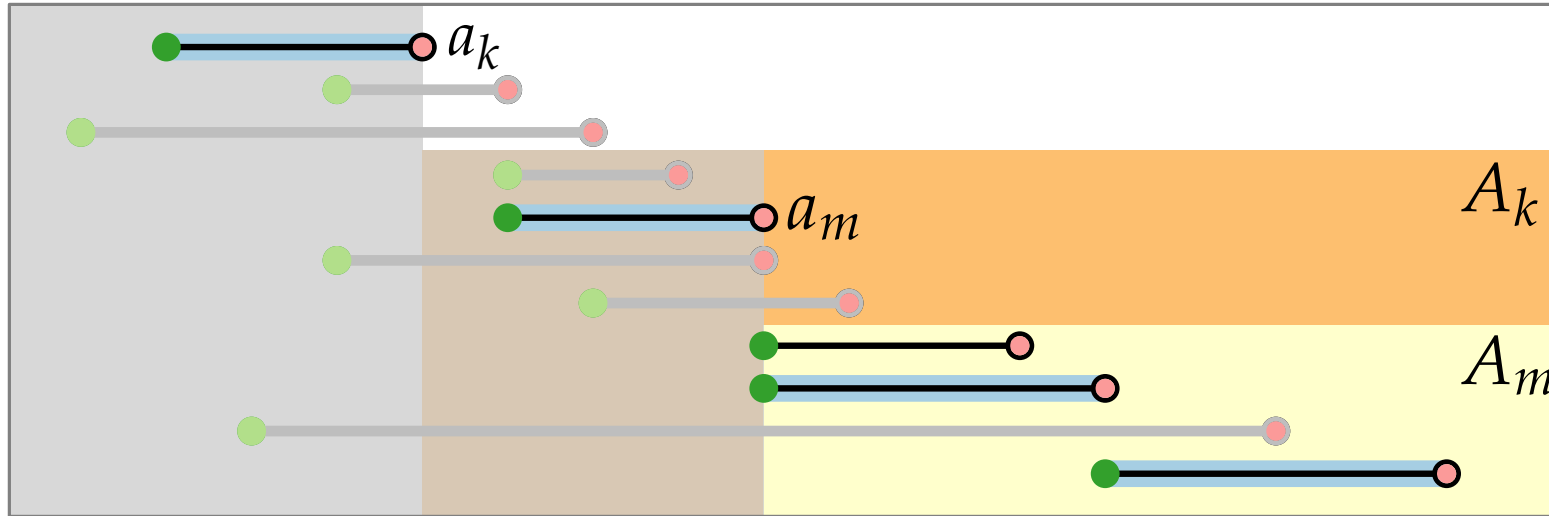
} optimale Teilstruktur!

Also gilt für den Wert c_k einer optimalen Lösung für A_k :

Ein exakter Algorithmus...



Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



Eine optimale Lösung für A_k besteht aus:

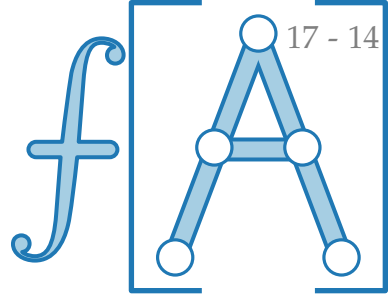
- **einem** ersten Intervall a_m und
- einer optimalen Lösung für A_m .

} optimale Teilstruktur!

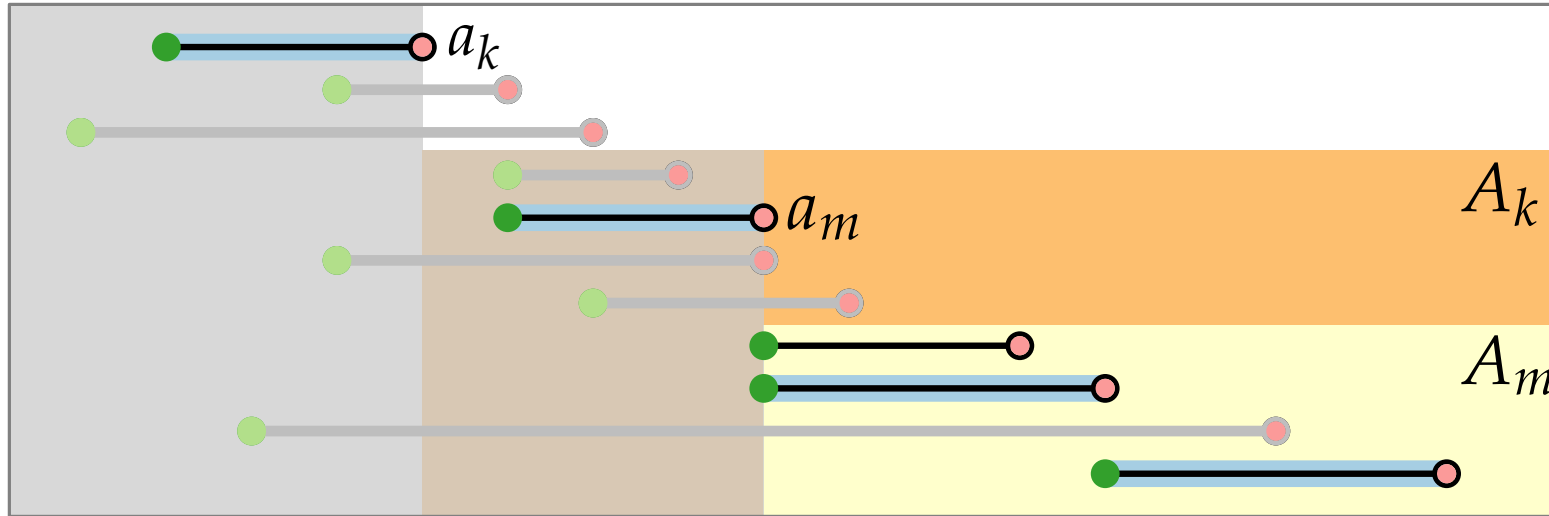
Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m$$

Ein exakter Algorithmus...



Sei $A_k = \{a_i \in A : s_i \geq e_k\}$ die Menge der Aktivitäten, die nach Ablauf von a_k beginnen. (Setze $A_0 = A$.)



Eine optimale Lösung für A_k besteht aus:

- **einem** ersten Intervall a_m und
- einer optimalen Lösung für A_m .

} optimale Teilstruktur!

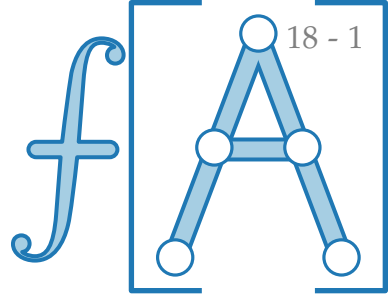
Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m)$$

...ein Dynamisches Programm!

Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m)$$



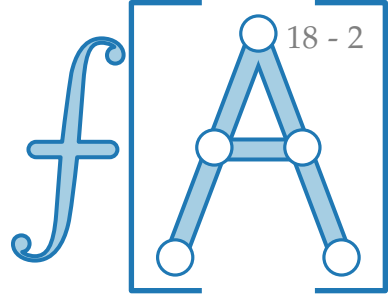
...ein Dynamisches Programm!

Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m)$$

Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.



...ein Dynamisches Programm!

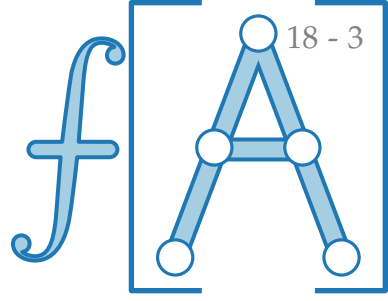
Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m)$$

Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es $c_0, \dots, c_n$ zu berechnen.



...ein Dynamisches Programm!

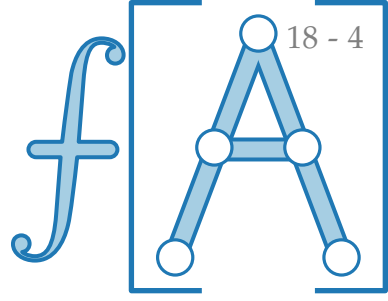
Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m)$$

Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es $c_0, \dots, c_n$ zu berechnen, wobei $c_n =$



...ein Dynamisches Programm!

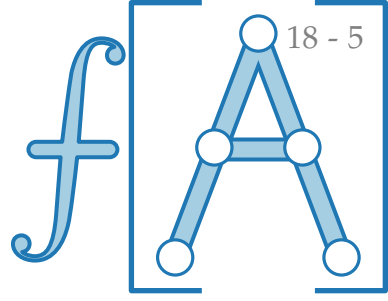
Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m)$$

Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.



...ein Dynamisches Programm!

Also gilt für den Wert c_k einer optimalen Lösung für A_k :

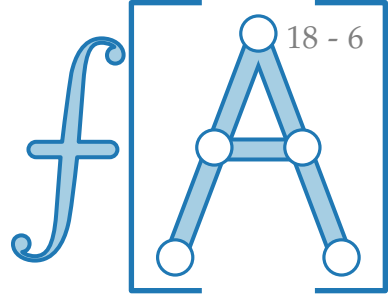
$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m)$$

Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit.



...ein Dynamisches Programm!

Also gilt für den Wert c_k einer optimalen Lösung für A_k :

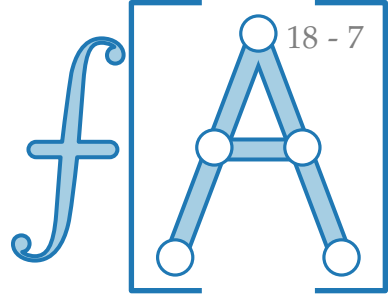
$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m)$$

Erinnern wir uns...

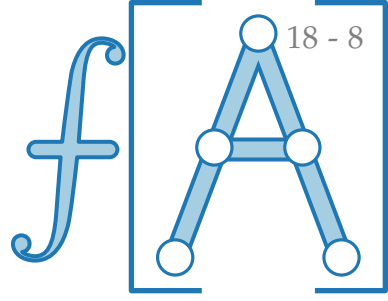
c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es **TABELLE** $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit.



...ein Dynamisches Programm!



Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m) \quad \text{BERECHNUNG EINES TABELLENEINTRAGS}$$

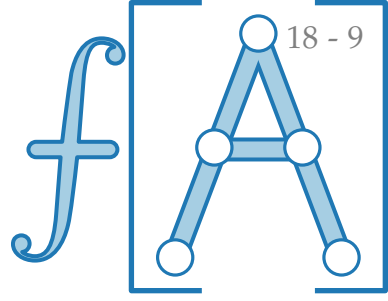
Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es **TABELLE** $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit.

...ein Dynamisches Programm!



Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m) \quad \text{BERECHNUNG EINES TABELLENEINTRAGS}$$

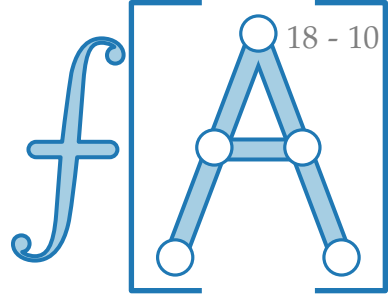
Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es **TABELLE** Größe $\mathcal{O}(n)$ $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit.

...ein Dynamisches Programm!



Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m) \quad \text{BERECHNUNG EINES TABELLENEINTRAGS} \quad \text{in je } \mathcal{O}(n) \text{ Zeit}$$

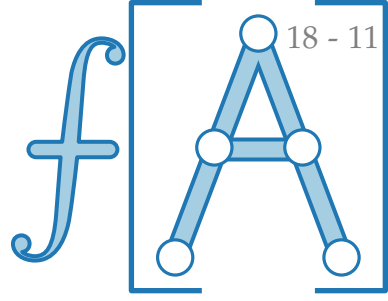
Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es **TABELLE** Größe $\mathcal{O}(n)$ $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit.

...ein Dynamisches Programm!



Also gilt für den Wert c_k einer optimalen Lösung für A_k :

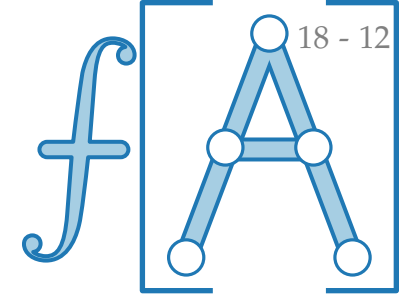
$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m) \quad \text{BERECHNUNG EINES TABELLENEINTRAGS} \quad \text{in je } \mathcal{O}(n) \text{ Zeit}$$

Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es **TABELLE** Größe $\mathcal{O}(n)$ $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit. $\mathcal{O}(n^2)$



...ein Dynamisches Programm!

Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m) \quad \text{BERECHNUNG EINES TABELLENEINTRAGS} \quad \text{in je } \mathcal{O}(n) \text{ Zeit}$$

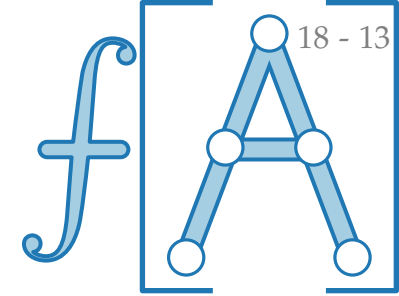
Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es **TABELLE** Größe $\mathcal{O}(n)$ $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit. $\mathcal{O}(n^2)$

Resultate.



...ein Dynamisches Programm!

Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m) \quad \text{BERECHNUNG EINES TABELLENEINTRAGS} \quad \text{in je } \mathcal{O}(n) \text{ Zeit}$$

Erinnern wir uns...

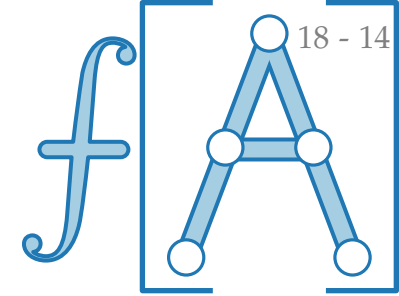
c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es **TABELLE** Größe $\mathcal{O}(n)$ $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit. $\mathcal{O}(n^2)$

Resultate.

- Der 2. Greedy-Alg. findet in $\mathcal{O}(n \log n)$ Zeit eine Lösung, die **mindestens 1/3 des maximalen Ertrags** garantiert.



...ein Dynamisches Programm!

Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m) \quad \text{BERECHNUNG EINES TABELLENEINTRAGS} \quad \text{in je } \mathcal{O}(n) \text{ Zeit}$$

Erinnern wir uns...

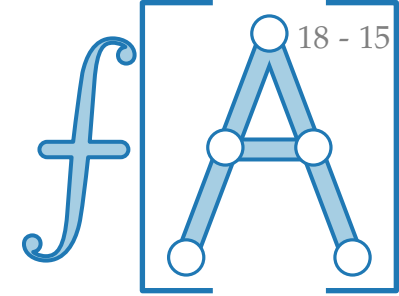
c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

Also genügt es **TABELLE** Größe $\mathcal{O}(n)$ $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit. $\mathcal{O}(n^2)$

Resultate.

- Der 2. Greedy-Alg. findet in $\mathcal{O}(n \log n)$ Zeit eine Lösung, die **mindestens 1/3 des maximalen Ertrags** garantiert.
- Unser DP findet in $\mathcal{O}(n^2)$ Zeit eine Lösung mit **maximalem** Ertrag.



...ein Dynamisches Programm!

Also gilt für den Wert c_k einer optimalen Lösung für A_k :

$$c_k = \max_{a_m \in A_k} c_m + \ell(a_m) \quad \text{BERECHNUNG EINES TABELLENEINTRAGS} \quad \text{in je } \mathcal{O}(n) \text{ Zeit}$$

Erinnern wir uns...

c_0 ist der Wert der optimalen Lösung für $A_0 = A$.

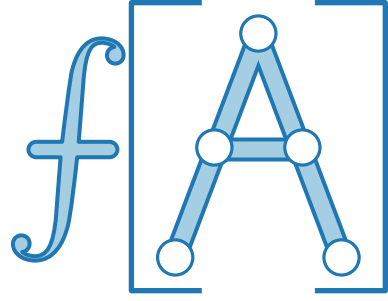
Also genügt es **TABELLE** Größe $\mathcal{O}(n)$ $c_0, \dots, c_n$ zu berechnen, wobei $c_n = 0$.

Laufzeit. $\mathcal{O}(n^2)$

Resultate.

- Der 2. Greedy-Alg. findet in $\mathcal{O}(n \log n)$ Zeit eine Lösung, die **mindestens 1/3 des maximalen Ertrags** garantiert.
- Unser DP findet in $\mathcal{O}(n^2)$ Zeit eine Lösung mit **maximalem** Ertrag.

Trade-Off zwischen Zeit und Qualität!

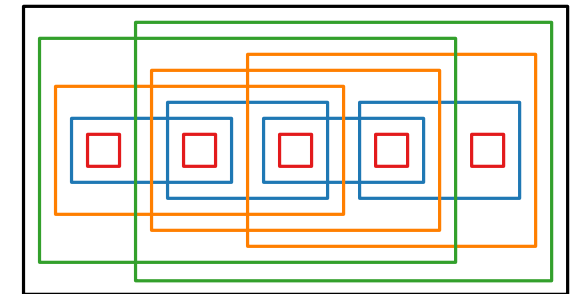
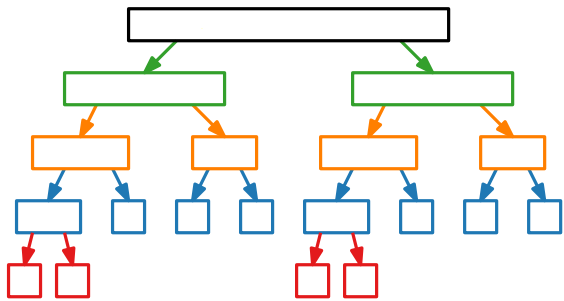


Fortgeschrittene Algorithmen

2. Vorlesung

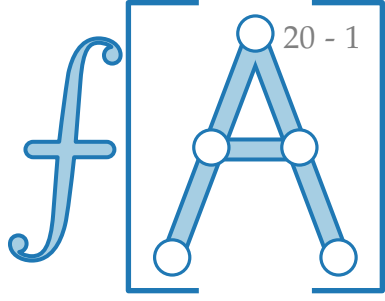
Dynamisches Programmieren

Teil IV: Weitere Beispiele



COINS

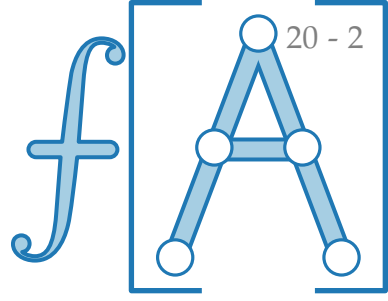
Problem. COINS



COINS

Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

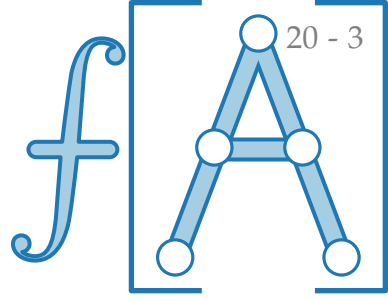


COINS

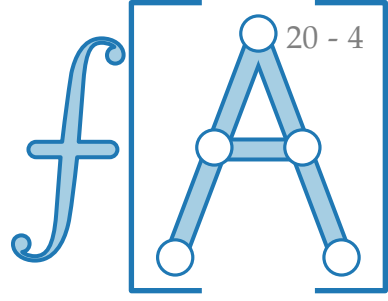
Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen



COINS



Problem. COINS

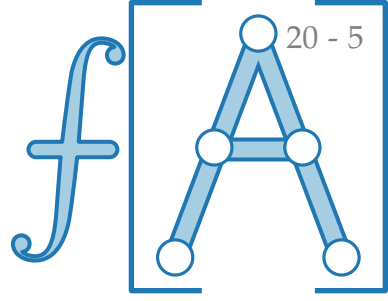
Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

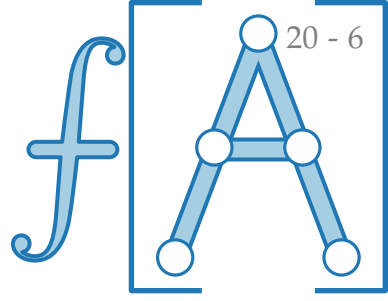
Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

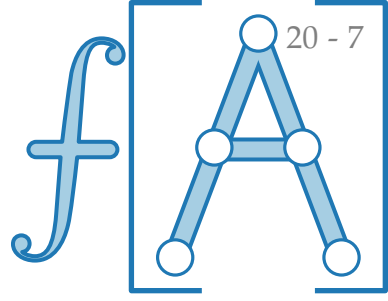
Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$f(0) = 0$

COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

Ansatz mit dynamischer Programmierung:

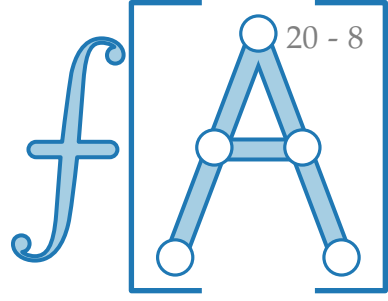
Teilprobleme: kleinere Geldwerte

$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$$f(0) = 0$$

$$f(n) = \min_{m \in M} f(n - m) + 1 \text{ für } n \geq 1$$

COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

Ansatz mit dynamischer Programmierung:

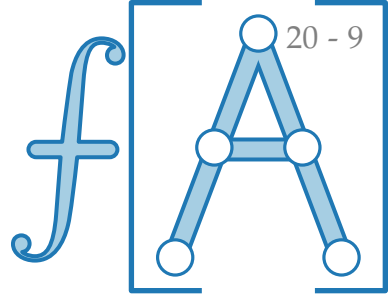
Teilprobleme: kleinere Geldwerte

$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$f(0) = 0$ $f(n) = \infty$ für $n < 0$

$f(n) = \min_{m \in M} f(n - m) + 1$ für $n \geq 1$

COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$$f(0) = 0 \quad f(n) = \infty \text{ für } n < 0$$

$$f(n) = \min_{m \in M} f(n - m) + 1 \text{ für } n \geq 1$$

$$M = \{1, 3, 4\}, G = 9$$

A diagram illustrating the conversion of a cursive letter to an uppercase letter. On the left is a cursive lowercase letter 'f'. To its right is a large blue square frame containing an uppercase letter 'A'. Above the top vertex of the 'A' is the text '20 - 10'.

Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

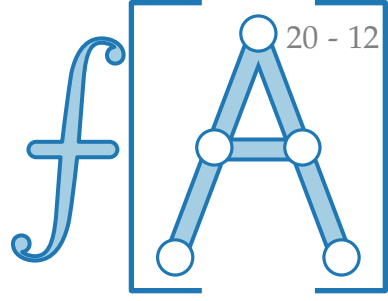
$$f(n) = \text{minimale Anzahl an M\u00fcnzen f\u00fcr Geldbetrag } n$$

$$f(0) = 0 \quad f(n) = \infty \text{ für } n < 0$$

$$f(n) = \min_{m \in M} f(n - m) + 1 \text{ für } n \geq 1$$

[illegible]

COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

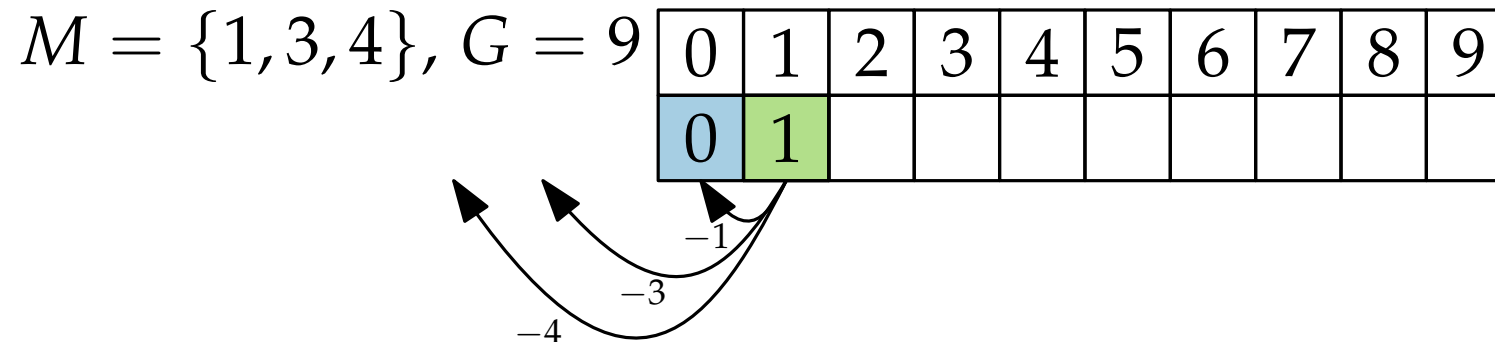
Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

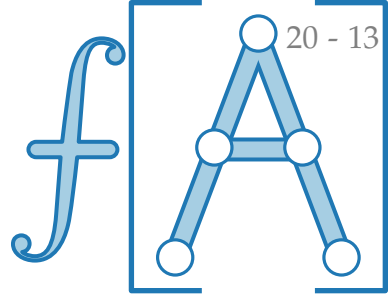
$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$$f(0) = 0 \quad f(n) = \infty \text{ für } n < 0$$

$$f(n) = \min_{m \in M} f(n - m) + 1 \text{ für } n \geq 1$$



COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

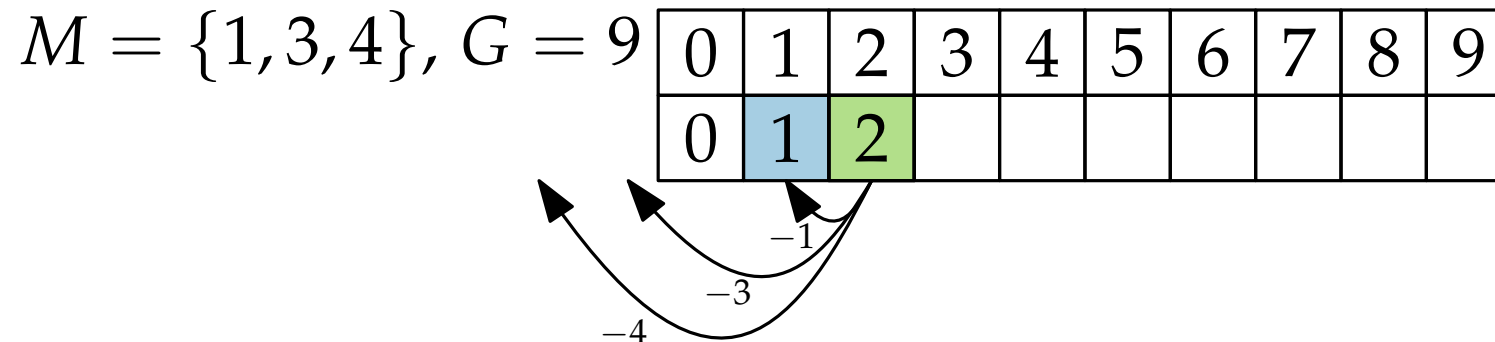
Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

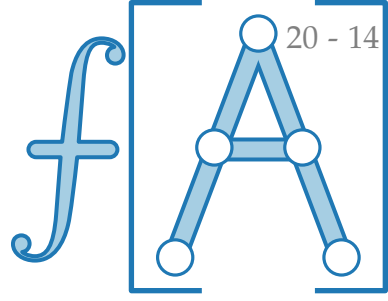
$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$f(0) = 0$ $f(n) = \infty$ für $n < 0$

$f(n) = \min_{m \in M} f(n - m) + 1$ für $n \geq 1$



COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

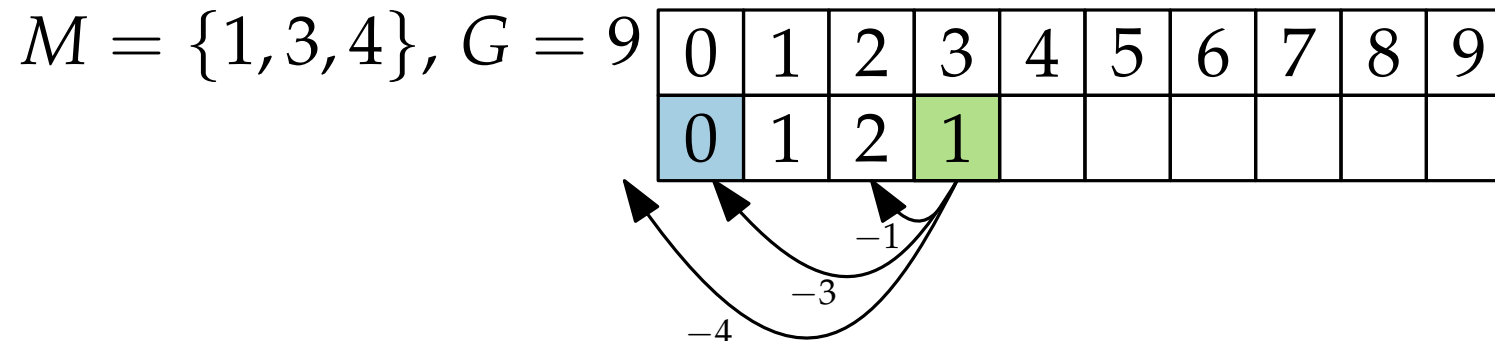
Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

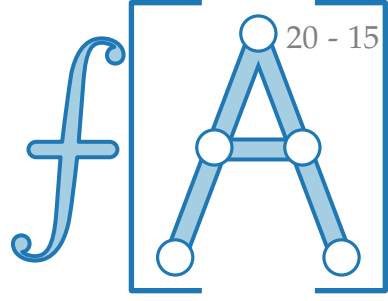
$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$f(0) = 0$ $f(n) = \infty$ für $n < 0$

$f(n) = \min_{m \in M} f(n - m) + 1$ für $n \geq 1$



COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

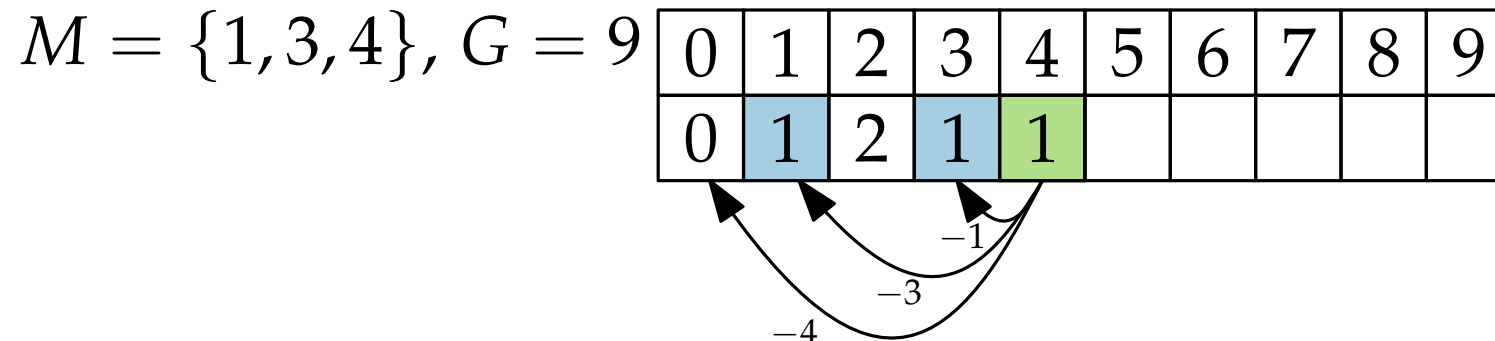
Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

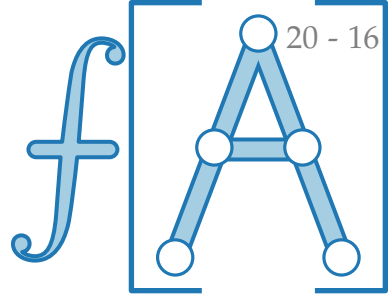
$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$f(0) = 0$ $f(n) = \infty$ für $n < 0$

$f(n) = \min_{m \in M} f(n - m) + 1$ für $n \geq 1$



COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

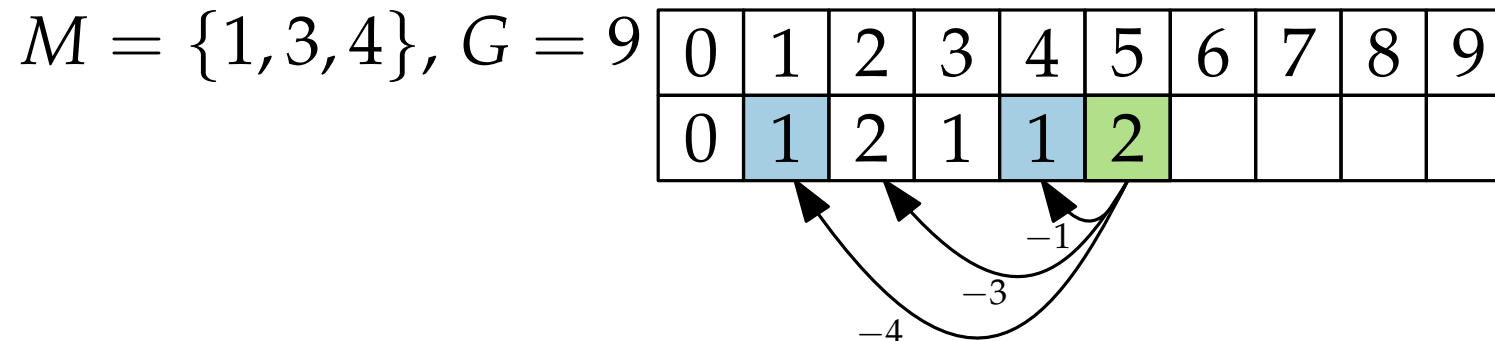
Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

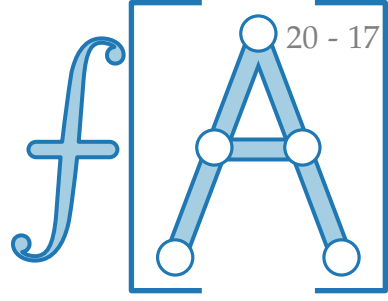
$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$f(0) = 0$ $f(n) = \infty$ für $n < 0$

$f(n) = \min_{m \in M} f(n - m) + 1$ für $n \geq 1$



COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

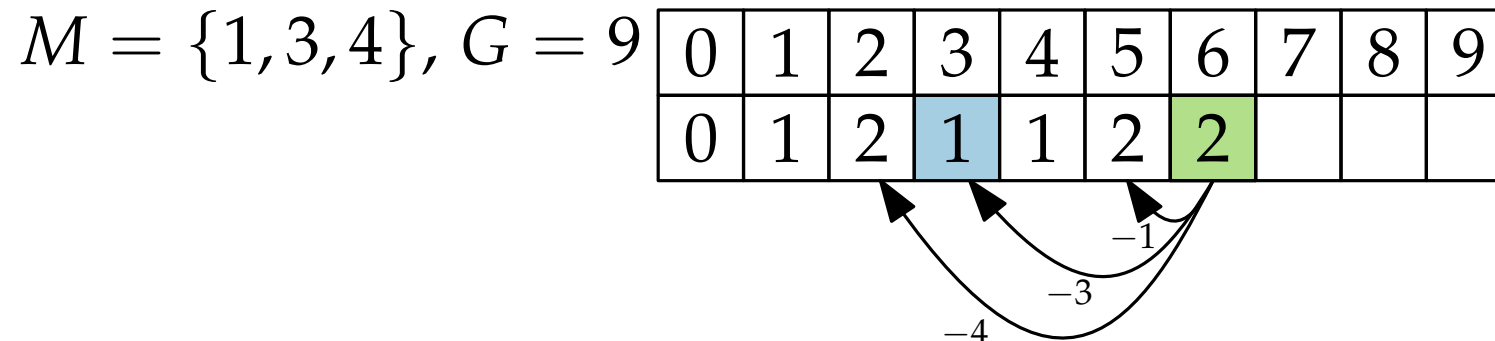
Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

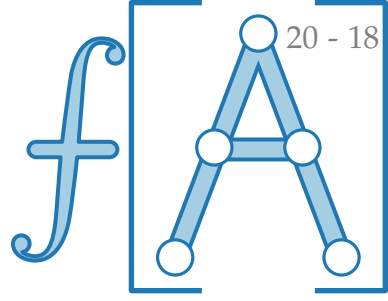
$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$f(0) = 0$ $f(n) = \infty$ für $n < 0$

$f(n) = \min_{m \in M} f(n - m) + 1$ für $n \geq 1$



COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

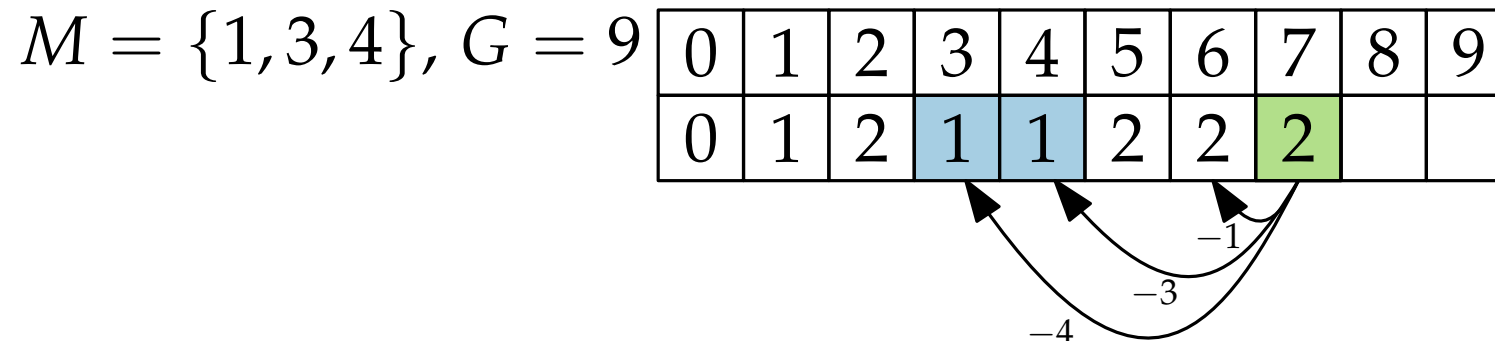
Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

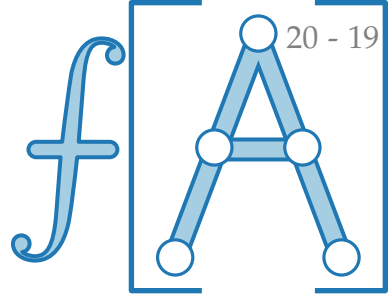
$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

$f(0) = 0$ $f(n) = \infty$ für $n < 0$

$f(n) = \min_{m \in M} f(n - m) + 1$ für $n \geq 1$



COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

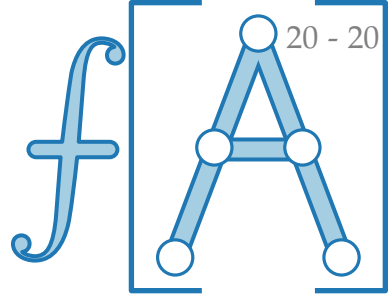
$f(0) = 0$ $f(n) = \infty$ für $n < 0$

$f(n) = \min_{m \in M} f(n - m) + 1$ für $n \geq 1$

$M = \{1, 3, 4\}$, $G = 9$

0	1	2	3	4	5	6	7	8	9
0	1	2	1	1	2	2	2	2	

COINS



Problem. COINS

Geg. Münzwerte $M = \{m_1, \dots, m_k\}$, $G \in \mathbb{N}$

Ges. Minimale Anzahl an Münzen um den Geldbetrag G mit Münzwerten aus M darzustellen

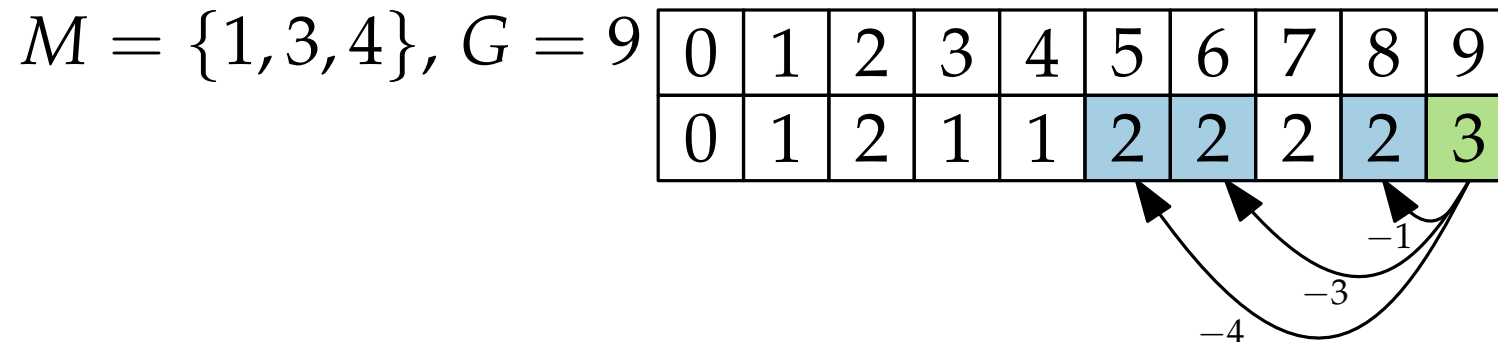
Ansatz mit dynamischer Programmierung:

Teilprobleme: kleinere Geldwerte

$f(n)$ = minimale Anzahl an Münzen für Geldbetrag n

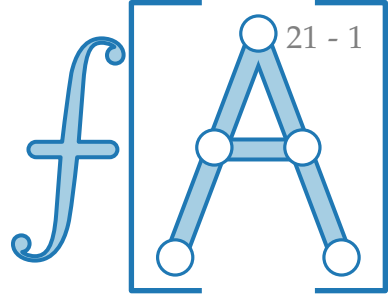
$f(0) = 0$ $f(n) = \infty$ für $n < 0$

$f(n) = \min_{m \in M} f(n - m) + 1$ für $n \geq 1$



ELEVATOR

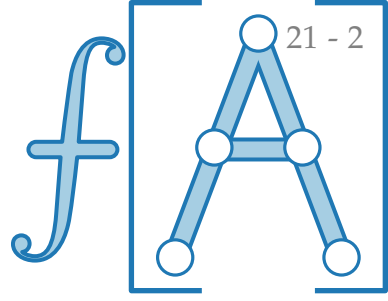
Problem. ELEVATOR
Geg.



ELEVATOR

Problem. ELEVATOR

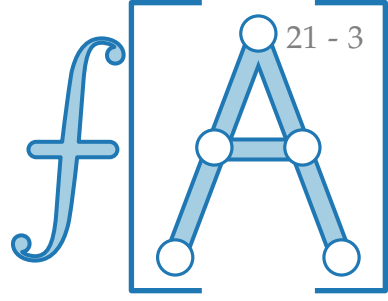
Geg. ■ Anzahl der Stockwerke n



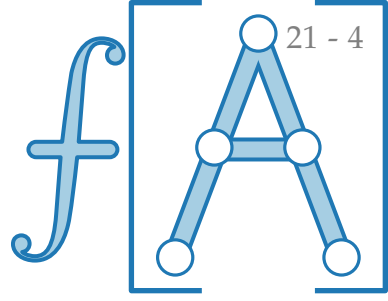
ELEVATOR

Problem. ELEVATOR

- Geg.**
- Anzahl der Stockwerke n
 - Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$



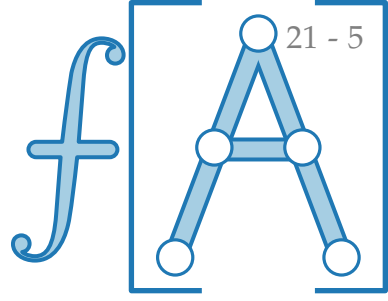
ELEVATOR



Problem. ELEVATOR

- Geg.**
- Anzahl der Stockwerke n
 - Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
 - Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$

ELEVATOR



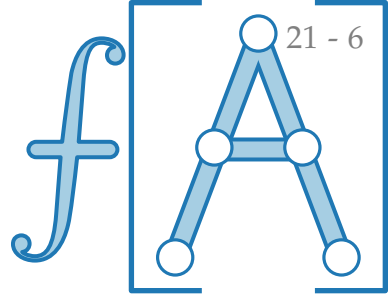
Problem. ELEVATOR

Geg.

- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

(fällt an wenn man von Treppe auf Aufzug wechselt)

ELEVATOR



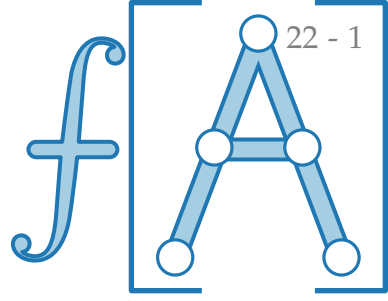
Problem. ELEVATOR

- Geg.**
- Anzahl der Stockwerke n
 - Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
 - Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
 - Wartezeit auf den Aufzug c

(fällt an wenn man von Treppe auf Aufzug wechselt)

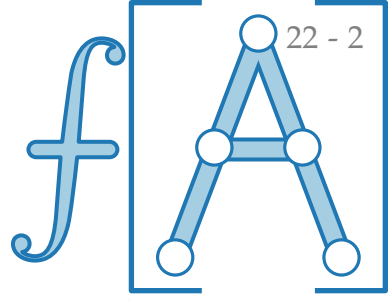
Ges. Wie lange braucht man für jeden Stock vom ersten bis zum n -ten, um ihn zu erreichen?

Problem ELEVATOR: DP-Lösung



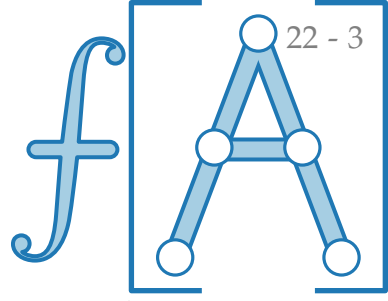
- S_i = schnellste Zeit zu Stockwerk i , wenn man über die Treppe dort ankommt

Problem ELEVATOR: DP-Lösung



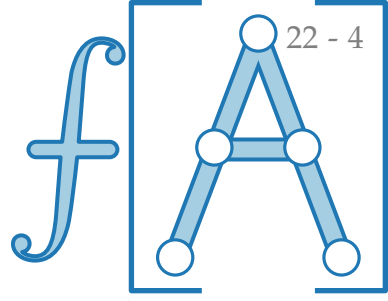
- S_i = schnellste Zeit zu Stockwerk i , wenn man über die Treppe dort ankommt
- E_i = schnellste Zeit zu Stockwerk i , wenn man im Aufzug dort ankommt

Problem ELEVATOR: DP-Lösung



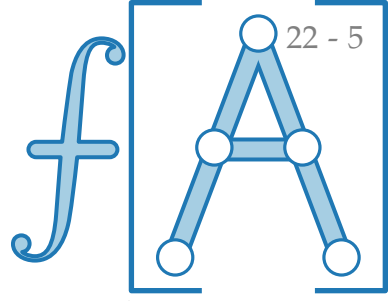
- S_i = schnellste Zeit zu Stockwerk i , wenn man über die Treppe dort ankommt
- E_i = schnellste Zeit zu Stockwerk i , wenn man im Aufzug dort ankommt
- $S_0 = 0, E_0 = c$

Problem ELEVATOR: DP-Lösung



- S_i = schnellste Zeit zu Stockwerk i , wenn man über die Treppe dort ankommt
- E_i = schnellste Zeit zu Stockwerk i , wenn man im Aufzug dort ankommt
- $S_0 = 0, E_0 = c$
- Lösung für Stock i : $\min(S_i, E_i)$

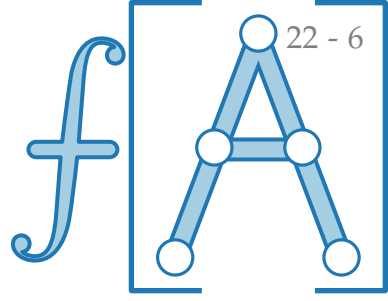
Problem ELEVATOR: DP-Lösung



- S_i = schnellste Zeit zu Stockwerk i , wenn man über die Treppe dort ankommt
- E_i = schnellste Zeit zu Stockwerk i , wenn man im Aufzug dort ankommt
- $S_0 = 0, E_0 = c$
- Lösung für Stock i : $\min(S_i, E_i)$

$$S_i = a_i + \begin{cases} S_{i-1} & \text{weiter über Treppen;} \\ E_{i-1} & \text{Wechsel vom Aufzug auf Treppen.} \end{cases}$$

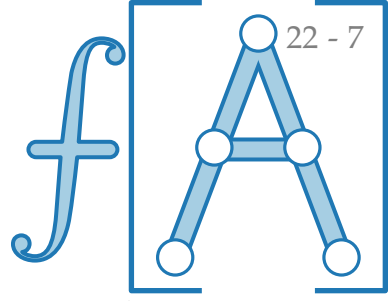
Problem ELEVATOR: DP-Lösung



- S_i = schnellste Zeit zu Stockwerk i , wenn man über die Treppe dort ankommt
- E_i = schnellste Zeit zu Stockwerk i , wenn man im Aufzug dort ankommt
- $S_0 = 0, E_0 = c$
- Lösung für Stock i : $\min(S_i, E_i)$

$$S_i = a_i + \begin{cases} S_{i-1} & \text{weiter über Treppen;} \\ E_{i-1} & \text{Wechsel vom Aufzug auf Treppen.} \end{cases}$$
$$E_i = b_i + \begin{cases} E_{i-1} & \text{weiter mit Aufzug;} \\ S_{i-1} + c & \text{Wechsel von Treppen auf Aufzug.} \end{cases}$$

Problem ELEVATOR: DP-Lösung



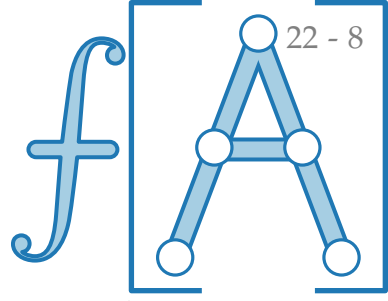
- S_i = schnellste Zeit zu Stockwerk i , wenn man über die Treppe dort ankommt
- E_i = schnellste Zeit zu Stockwerk i , wenn man im Aufzug dort ankommt
- $S_0 = 0, E_0 = c$
- Lösung für Stock i : $\min(S_i, E_i)$

$$S_i = a_i + \begin{cases} S_{i-1} & \text{weiter über Treppen;} \\ E_{i-1} & \text{Wechsel vom Aufzug auf Treppen.} \end{cases}$$
$$E_i = b_i + \begin{cases} E_{i-1} & \text{weiter mit Aufzug;} \\ S_{i-1} + c & \text{Wechsel von Treppen auf Aufzug.} \end{cases}$$

Also:

$$\begin{aligned} S_i &= a_i + \min\{S_{i-1}, E_{i-1}\} \\ E_i &= b_i + \min\{E_{i-1}, S_{i-1} + c\} \end{aligned}$$

Problem ELEVATOR: DP-Lösung



- S_i = schnellste Zeit zu Stockwerk i , wenn man über die Treppe dort ankommt
- E_i = schnellste Zeit zu Stockwerk i , wenn man im Aufzug dort ankommt
- $S_0 = 0, E_0 = c$
- Lösung für Stock i : $\min(S_i, E_i)$

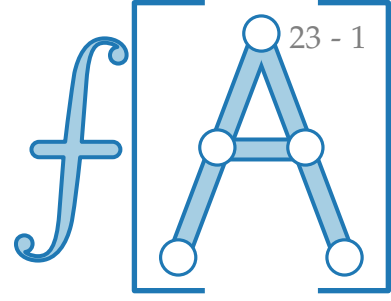
$$S_i = a_i + \begin{cases} S_{i-1} & \text{weiter über Treppen;} \\ E_{i-1} & \text{Wechsel vom Aufzug auf Treppen.} \end{cases}$$
$$E_i = b_i + \begin{cases} E_{i-1} & \text{weiter mit Aufzug;} \\ S_{i-1} + c & \text{Wechsel von Treppen auf Aufzug.} \end{cases}$$

Also:

$$\begin{aligned} S_i &= a_i + \min\{S_{i-1}, E_{i-1}\} \\ E_i &= b_i + \min\{E_{i-1}, S_{i-1} + c\} \end{aligned}$$

➔ Laufzeit $\mathcal{O}(n)$

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

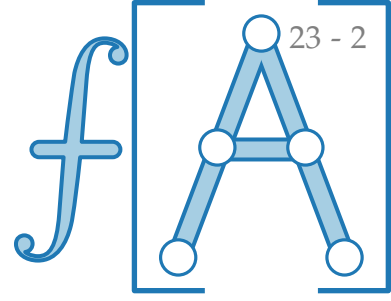
S_i									
E_i									

$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

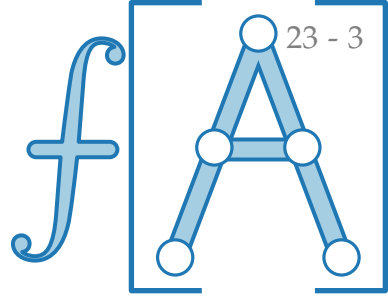
S_i	0								
E_i	4								

$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

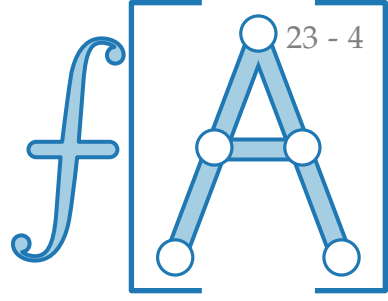
a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$
$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

S_i	0 ← 1								
E_i	4 ← 6								

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

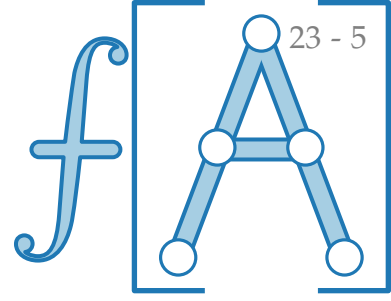
$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

S_i	0	1	3						
E_i	4	6	6						

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

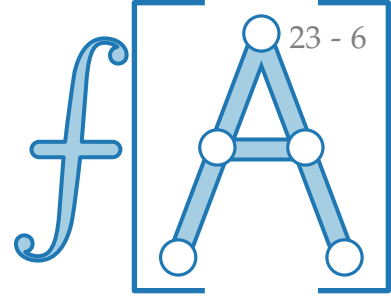
$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

S_i	0	1	3	6					
E_i	4	6	6	7					

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

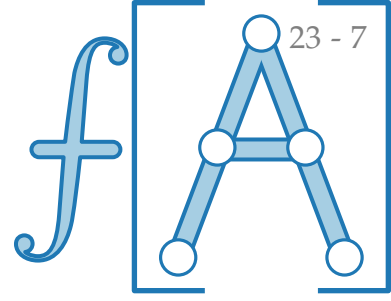
$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

S_i	0	1	3	6	9				
E_i	4	6	6	7	8				

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

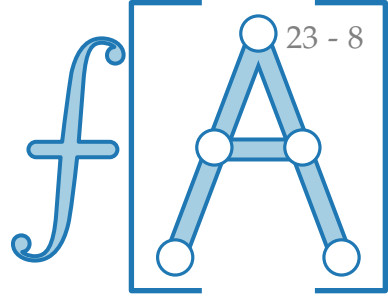
$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

S_i	0	1	3	6	9	9			
E_i	4	6	6	7	8	12			

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

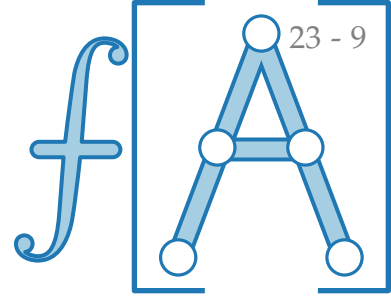
$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

S_i	0	1	3	6	9	9	13		
E_i	4	6	6	7	8	12	13		

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

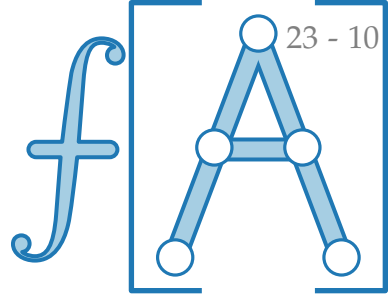
$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

S_i	0	1	3	6	9	9	13	16	
E_i	4	6	6	7	8	12	13	15	

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

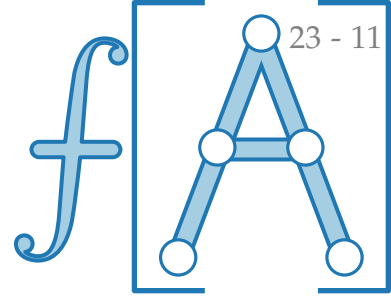
$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

S_i	0	1	3	6	9	9	13	16	18
E_i	4	6	6	7	8	12	13	15	16

ELEVATOR: Beispiel



- Anzahl der Stockwerke n
- Dauer pro Stockwerk über die Treppe $a_1, \dots, a_n$
- Dauer pro Stockwerk über den Aufzug $b_1, \dots, b_n$
- Wartezeit auf den Aufzug c

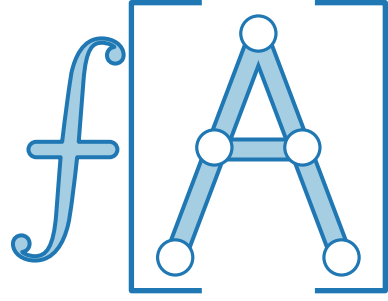
a_i	—	1	2	3	3	1	4	3	3
b_i	—	2	1	1	1	4	1	2	1

$$S_i = a_i + \min\{S_{i-1}, E_{i-1}\}$$

$$E_i = b_i + \min\{E_{i-1}, S_{i-1} + c\}$$

Wartezeit für Aufzug: $c = 4$

S_i	0	1	3	6	9	9	13	16	18
E_i	4	6	6	7	8	12	13	15	16

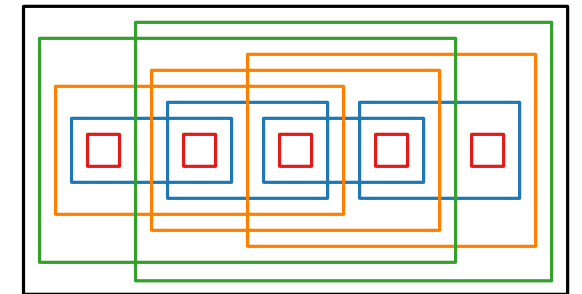
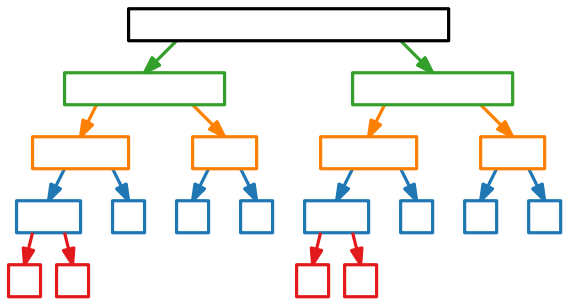


Fortgeschrittene Algorithmen

2. Vorlesung

Dynamisches Programmieren

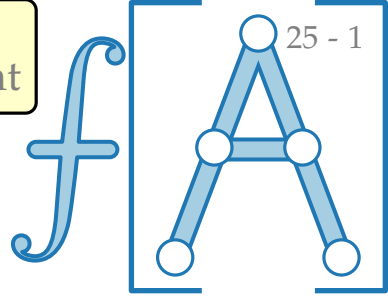
Teil V: Längste Wege



Längste Wege

Problem. LÄNGSTE WEGE

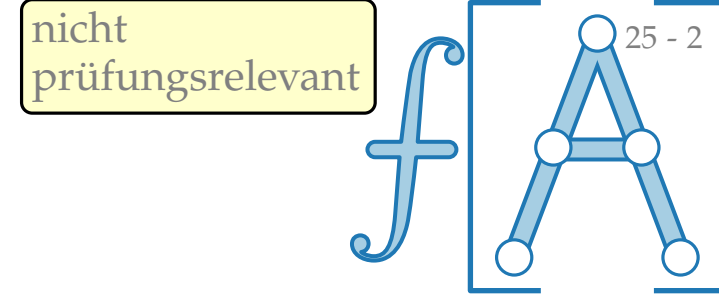
nicht
prüfungsrelevant



Längste Wege

Problem. LÄNGSTEWEGE

Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

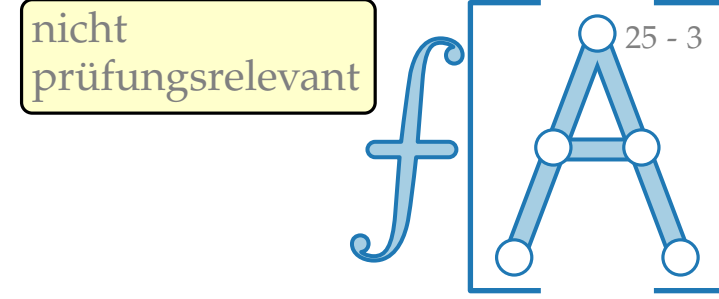


Längste Wege

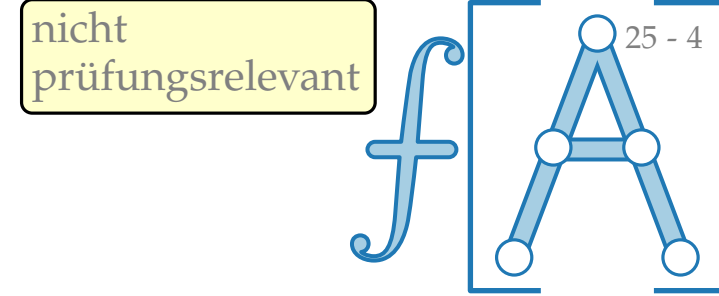
Problem. LÄNGSTEWEGE

Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

Ges. ein längster einfacher s - t -Weg



Längste Wege

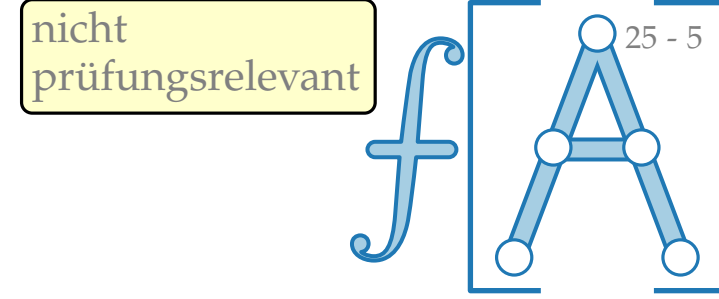


Problem. LÄNGSTE WEGE

Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V$, $s \neq t$ und t von s erreichbar.

Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0 v_1, \dots, v_{k-1} v_k \in E$, $v_i \neq v_j$ (für $i \neq j$) und k maximal.

Längste Wege



Problem. LÄNGSTE WEGE

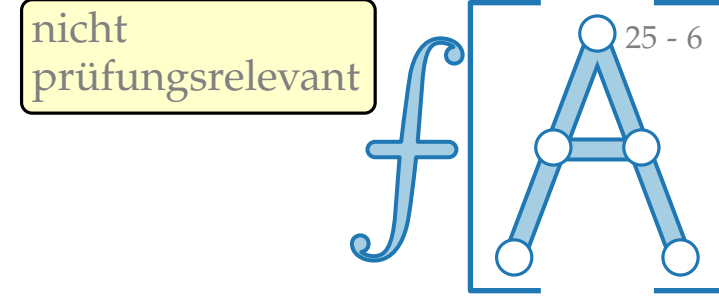
Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0 v_1, \dots, v_{k-1} v_k \in E, v_i \neq v_j$ (für $i \neq j$) und k maximal.

Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)

Längste Wege



Problem. LÄNGSTEWEGE

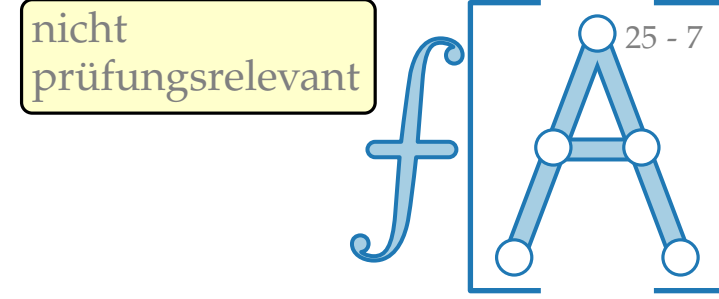
Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0v_1, \dots, v_{k-1}v_k \in E, v_i \neq v_j$ (für $i \neq j$) und k maximal.

Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)

Längste Wege



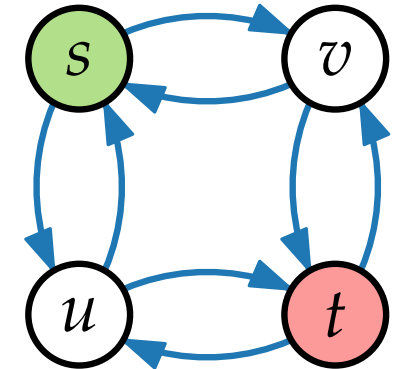
Problem. LÄNGSTE WEGE

Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

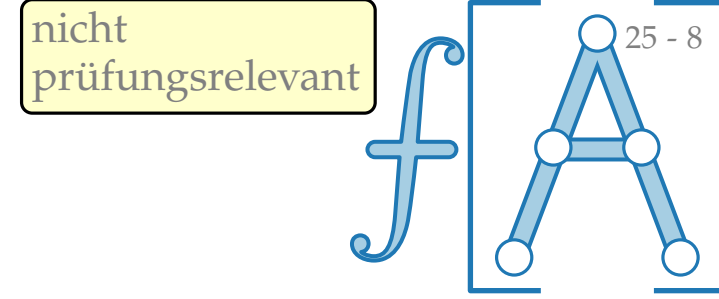
Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0 v_1, \dots, v_{k-1} v_k \in E, v_i \neq v_j$ (für $i \neq j$) und k maximal.

Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)



Längste Wege



Problem. LÄNGSTE WEGE

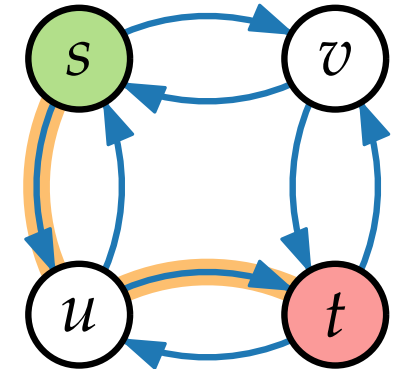
Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0 v_1, \dots, v_{k-1} v_k \in E, v_i \neq v_j$ (für $i \neq j$) und k maximal.

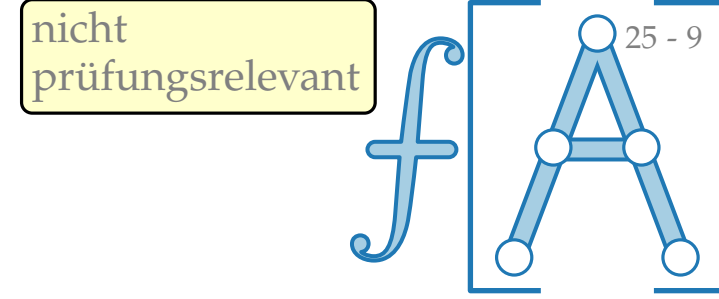
$\langle s, u, t \rangle$ ist ein längster einfacher s - t -Weg.

Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)



Längste Wege



Problem. LÄNGSTE WEGE

Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

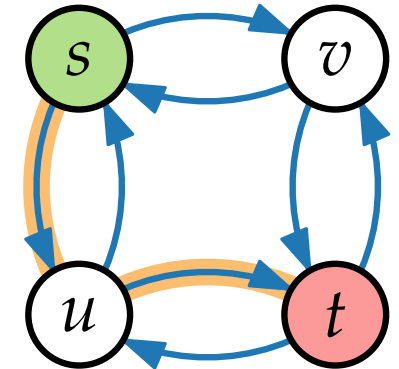
Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0 v_1, \dots, v_{k-1} v_k \in E, v_i \neq v_j$ (für $i \neq j$) und k maximal.

$\langle s, u, t \rangle$ ist ein längster einfacher s - t -Weg.

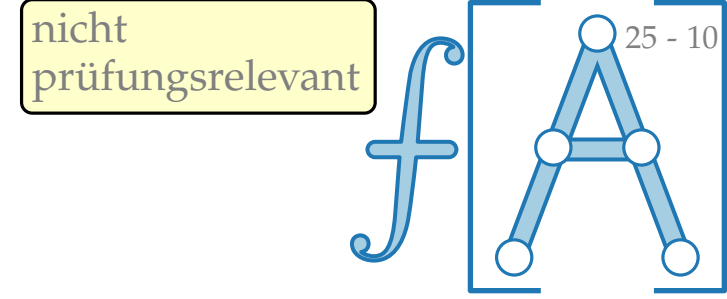
Aber:

Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)



Längste Wege



Problem. LÄNGSTE WEGE

Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0 v_1, \dots, v_{k-1} v_k \in E, v_i \neq v_j$ (für $i \neq j$) und k maximal.

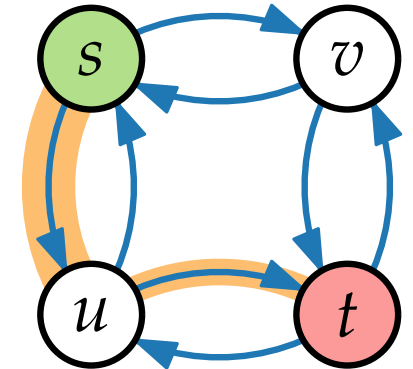
$\langle s, u, t \rangle$ ist ein längster einfacher s - t -Weg.

Aber:

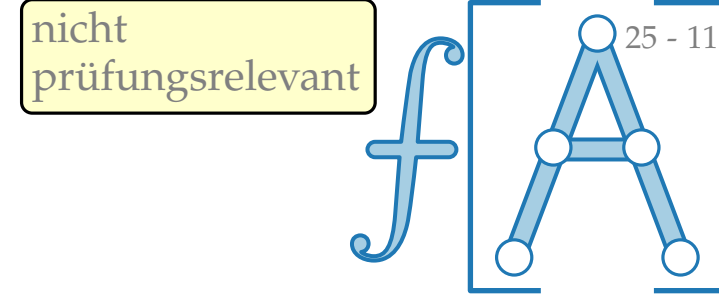
$\langle s, u \rangle$ ist *kein* längster einfacher s - u -Weg;

Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)



Längste Wege



Problem. LÄNGSTE WEGE

Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0 v_1, \dots, v_{k-1} v_k \in E, v_i \neq v_j$ (für $i \neq j$) und k maximal.

$\langle s, u, t \rangle$ ist ein längster einfacher s - t -Weg.

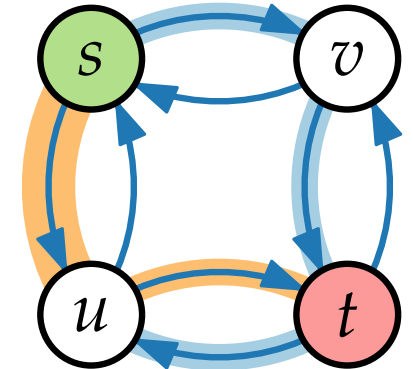
Aber:

$\langle s, u \rangle$ ist *kein* längster einfacher s - u -Weg;

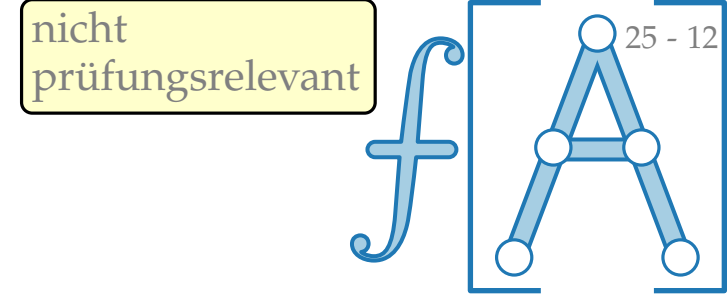
$\langle s, v, t, u \rangle$ ist ein längster einfacher s - u -Weg!

Fahrplan

1. Struktur einer optimalen Lösung charakterisieren
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)



Längste Wege



Problem. LÄNGSTE WEGE

Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0 v_1, \dots, v_{k-1} v_k \in E, v_i \neq v_j$ (für $i \neq j$) und k maximal.

$\langle s, u, t \rangle$ ist ein längster einfacher s - t -Weg.

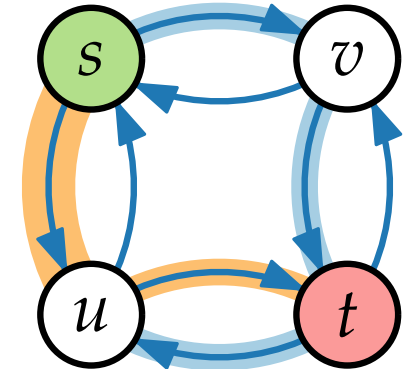
Aber:

$\langle s, u \rangle$ ist *kein* längster einfacher s - u -Weg;

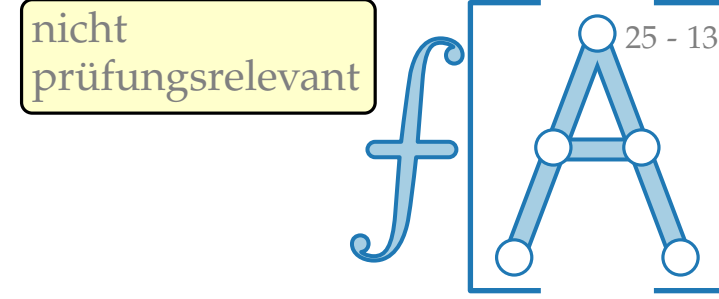
$\langle s, v, t, u \rangle$ ist ein längster einfacher s - u -Weg!

Fahrplan

1. Struktur einer optimalen Lösung charakterisieren ⚡
2. Wert einer optimalen Lösung rekursiv definieren
3. Wert einer optimalen Lösung berechnen (meist bottom-up)



Längste Wege



Problem. LÄNGSTE WEGE

Geg. ungewichteter gerichteter Graph $G = (V, E)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

Ges. ein längster einfacher s - t -Weg,
d.h. eine Folge $\langle s = v_0, v_1, \dots, v_k = t \rangle$ mit
 $v_0 v_1, \dots, v_{k-1} v_k \in E, v_i \neq v_j$ (für $i \neq j$) und k maximal.

$\langle s, u, t \rangle$ ist ein längster einfacher s - t -Weg.

Aber:

$\langle s, u \rangle$ ist *kein* längster einfacher s - u -Weg;

$\langle s, v, t, u \rangle$ ist ein längster einfacher s - u -Weg!

Fahrplan

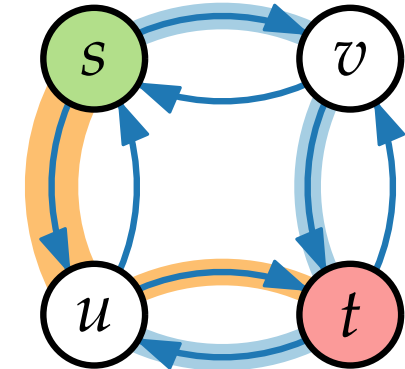
1. Struktur einer optimalen Lösung charakterisieren



2. Wert einer optimalen Lösung rekursiv definieren

3. Wert einer optimalen Lösung berechnen (meist bottom-up)

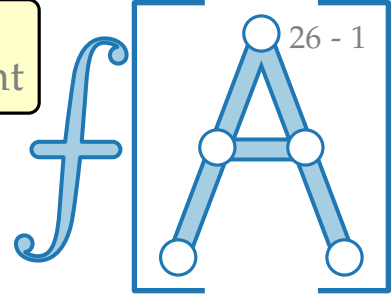
*) Es ist NP-schwer für (G, s, t, k) zu entscheiden, ob G einen einfachen s - t -Weg der Länge k enthält. (Vgl. Hamilton-Weg!)



Längste Wege in azyklischen Graphen

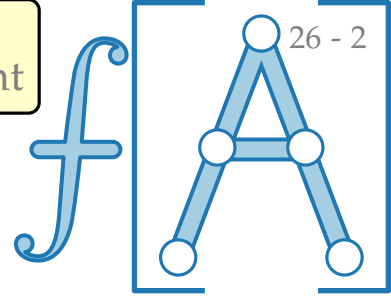
Problem. LÄNGSTE WEGE IN AZYKLISCHEN GRAPHEN

nicht
prüfungsrelevant



Längste Wege in azyklischen Graphen

nicht
prüfungsrelevant



Problem. LÄNGSTEWEGEINAZYKLISCHENGRAPHEN

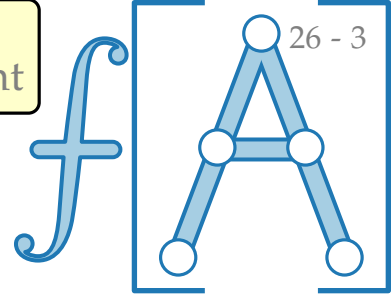
Geg.

gewichteter gerichteter **kreisfreier** Graph

$G = (V, E; w)$ mit $s, t \in V$, $s \neq t$ und t von s erreichbar.

Längste Wege in azyklischen Graphen

nicht
prüfungsrelevant

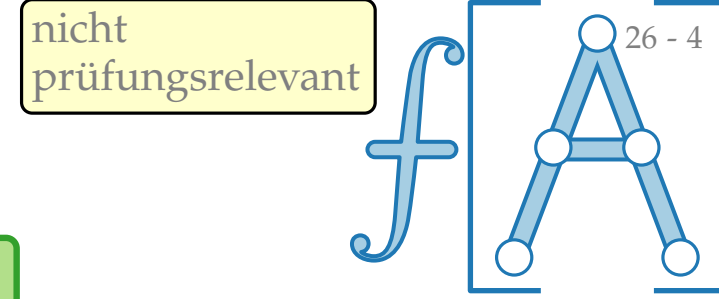


Problem. LÄNGSTE WEGE IN AZYKLISCHEN GRAPHEN

Geg. gewichteter gerichteter **kreisfreier** Graph
 $G = (V, E; w)$ mit $s, t \in V$, $s \neq t$ und t von s erreichbar.

Ges. ein längster s - t -Weg.

Längste Wege in azyklischen Graphen



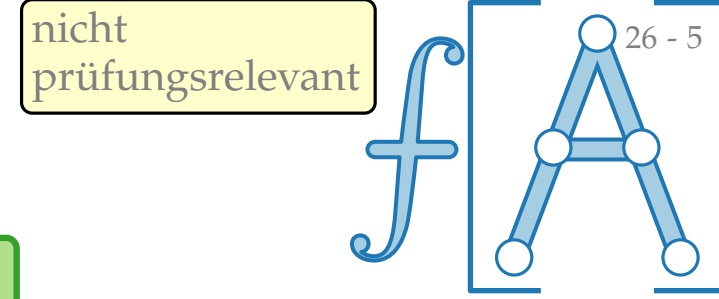
Problem. LÄNGSTEWEGEINAZYKLISCHENGRAPHEN

Geg. gewichteter gerichteter **kreisfreier** Graph
 $G = (V, E; w)$ mit $s, t \in V$, $s \neq t$ und t von s erreichbar.

Ges. ein längster s - t -Weg.

Beobachtung 1. In kreisfreien Graphen sind alle Wege einfach.

Längste Wege in azyklischen Graphen



Problem. LÄNGSTEWEGEINAZYKLISCHENGRAPHEN

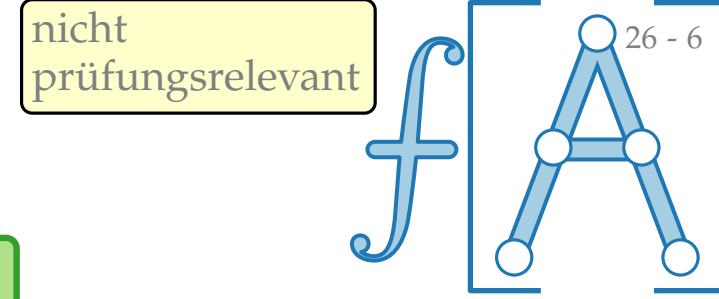
Geg. gewichteter gerichteter **kreisfreier** Graph
 $G = (V, E; w)$ mit $s, t \in V, s \neq t$ und t von s erreichbar.

Ges. ein längster s - t -Weg.

Beobachtung 1. In kreisfreien Graphen sind alle Wege einfach.

Beobachtung 2. Dieses Problem hat optimale Teilstruktur, denn:

Längste Wege in azyklischen Graphen



Problem. LÄNGSTE WEGE IN AZYKLISCHEN GRAPHEN

Geg. gewichteter gerichteter **kreisfreier** Graph
 $G = (V, E; w)$ mit $s, t \in V$, $s \neq t$ und t von s erreichbar.

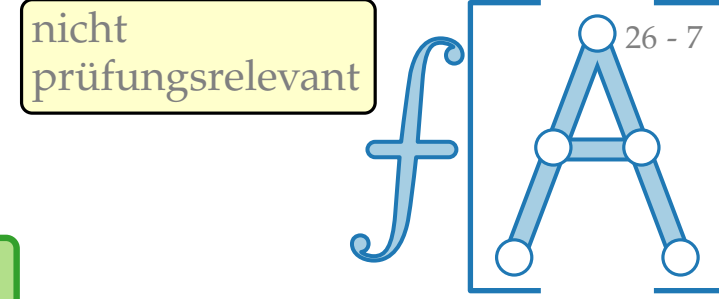
Ges. ein längster s - t -Weg.

Beobachtung 1. In kreisfreien Graphen sind alle Wege einfach.

Beobachtung 2. Dieses Problem hat optimale Teilstruktur, denn:
Angenommen, ein längster s - t -Weg π geht durch u , d.h.

$$\pi = s \xrightarrow{\pi_{su}} u \xrightarrow{\pi_{ut}} t.$$

Längste Wege in azyklischen Graphen



Problem. LÄNGSTE WEGE IN AZYKLISCHEN GRAPHEN

Geg. gewichteter gerichteter **kreisfreier** Graph
 $G = (V, E; w)$ mit $s, t \in V$, $s \neq t$ und t von s erreichbar.

Ges. ein längster s - t -Weg.

Beobachtung 1. In kreisfreien Graphen sind alle Wege einfach.

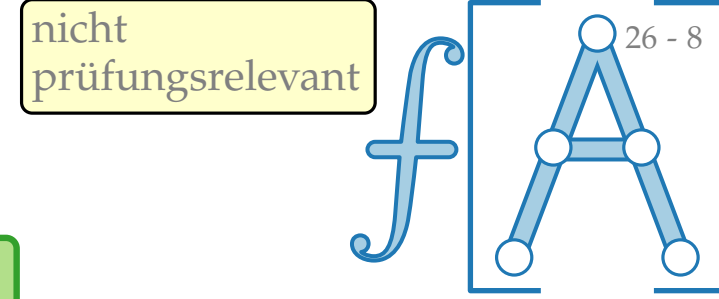
Beobachtung 2. Dieses Problem hat optimale Teilstruktur, denn:
Angenommen, ein längster s - t -Weg π geht durch u , d.h.

$$\pi = s \xrightarrow{\pi_{su}} u \xrightarrow{\pi_{ut}} t.$$

Dann gilt:

π_{su} ist längster s - u -Weg; π_{ut} ist längster u - t -Weg –

Längste Wege in azyklischen Graphen



Problem. LÄNGSTEWEGEINAZYKLISCHENGRAPHEN

Geg. gewichteter gerichteter **kreisfreier** Graph
 $G = (V, E; w)$ mit $s, t \in V$, $s \neq t$ und t von s erreichbar.

Ges. ein längster s - t -Weg.

Beobachtung 1. In kreisfreien Graphen sind alle Wege einfach.

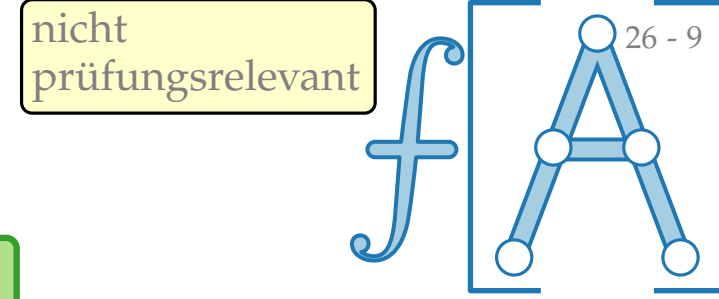
Beobachtung 2. Dieses Problem hat optimale Teilstruktur, denn:
Angenommen, ein längster s - t -Weg π geht durch u , d.h.

$$\pi = s \xrightarrow{\pi_{su}} u \xrightarrow{\pi_{ut}} t.$$

Dann gilt:

π_{su} ist längster s - u -Weg; π_{ut} ist längster u - t -Weg –
sonst wäre π kein längster s - t -Weg.

Längste Wege in azyklischen Graphen



Problem. LÄNGSTE WEGE IN AZYKLISCHEN GRAPHEN

Geg. gewichteter gerichteter **kreisfreier** Graph
 $G = (V, E; w)$ mit $s, t \in V$, $s \neq t$ und t von s erreichbar.

Ges. ein längster s - t -Weg.

Beobachtung 1. In kreisfreien Graphen sind alle Wege einfach.

Beobachtung 2. Dieses Problem hat optimale Teilstruktur, denn:
Angenommen, ein längster s - t -Weg π geht durch u , d.h.
$$\pi = s \xrightarrow{\pi_{su}} u \xrightarrow{\pi_{ut}} t.$$

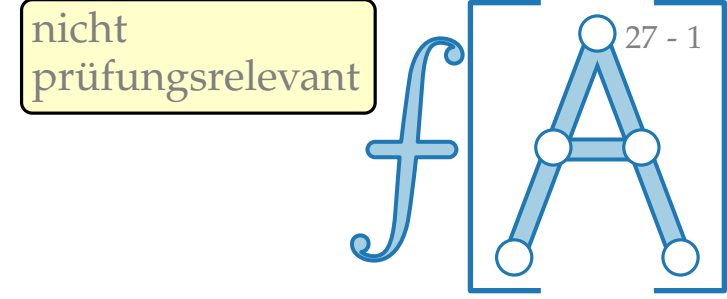
Dann gilt:

π_{su} ist längster s - u -Weg; π_{ut} ist längster u - t -Weg –
sonst wäre π kein längster s - t -Weg.

Außerdem gilt $V(\pi_{su}) \cap V(\pi_{ut}) = \{u\}$;
sonst gäbe es einen Kreis!

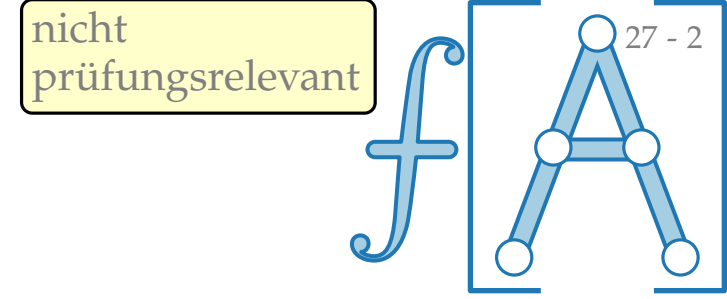
Algorithmus nach Fahrplan

1. Struktur einer optimalen Lösung charakterisieren ✓



Algorithmus nach Fahrplan

1. Struktur einer optimalen Lösung charakterisieren ✓
2. Wert einer optimalen Lösung rekursiv definieren

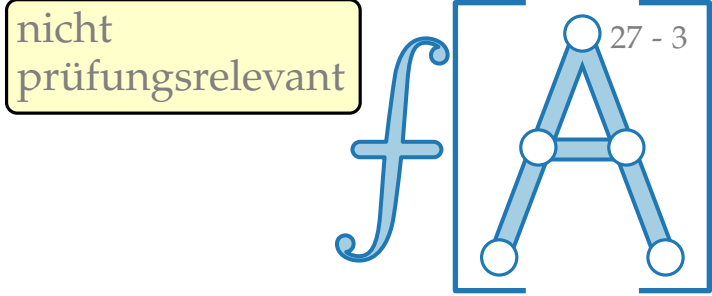


Algorithmus nach Fahrplan

1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

$d_s =$ // Länge eines längsten s - v -Wegs

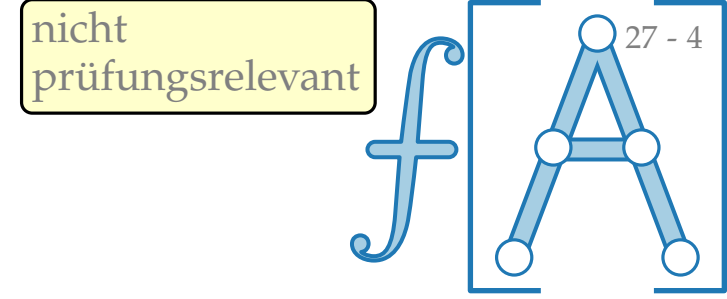


Algorithmus nach Fahrplan

1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$



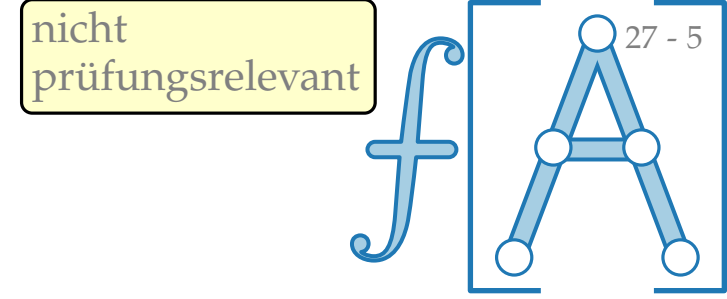
Algorithmus nach Fahrplan

1. Struktur einer optimalen Lösung charakterisieren ✓

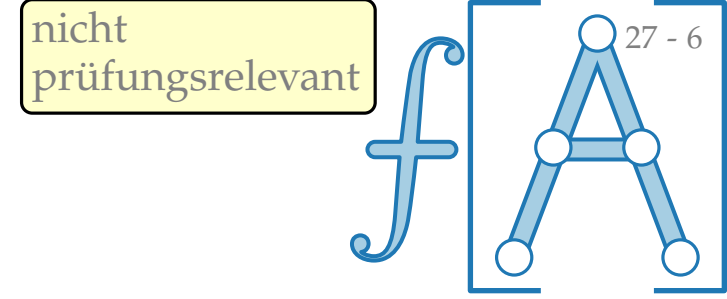
2. Wert einer optimalen Lösung rekursiv definieren

$d_s = 0$ // Länge eines längsten s - v -Wegs

$d_v =$



Algorithmus nach Fahrplan



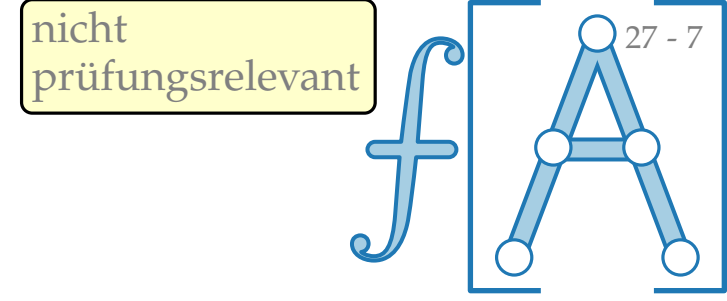
1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$

$$d_v = \max_{u: uv \in E} d_u + w(u, v)$$

Algorithmus nach Fahrplan



1. Struktur einer optimalen Lösung charakterisieren ✓

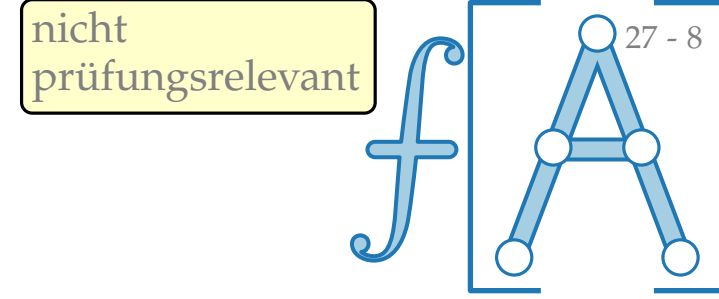
2. Wert einer optimalen Lösung rekursiv definieren

$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$

$$d_v = \max_{u: uv \in E} d_u + w(u, v)$$

3. Wert einer optimalen Lösung berechnen (hier bottom-up)

Algorithmus nach Fahrplan



1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

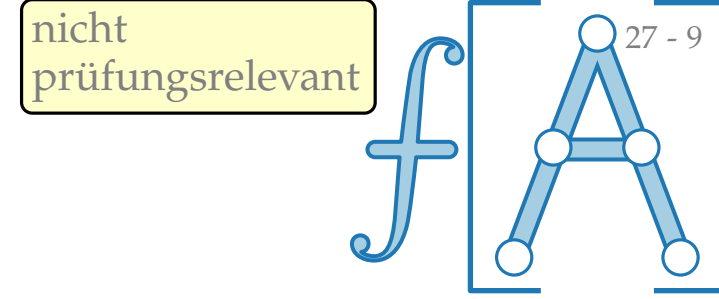
$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$

$$d_v = \max_{u: uv \in E} d_u + w(u, v)$$

3. Wert einer optimalen Lösung berechnen (hier bottom-up)

■ G topologisch sortieren

Algorithmus nach Fahrplan



1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$

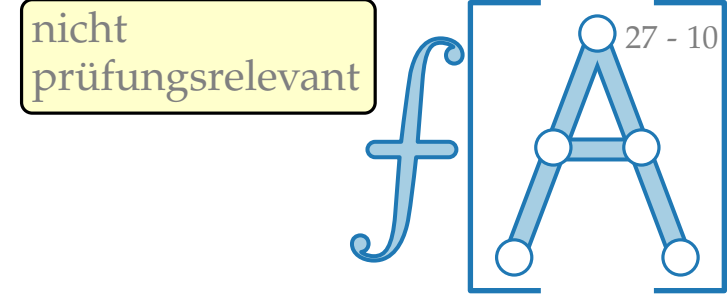
$$d_v = \max_{u: uv \in E} d_u + w(u, v)$$

3. Wert einer optimalen Lösung berechnen (hier bottom-up)

■ G topologisch sortieren

■ d -Werte initialisieren: $d_s = 0$ und $d_v = -\infty$ für alle $v \neq s$

Algorithmus nach Fahrplan



1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

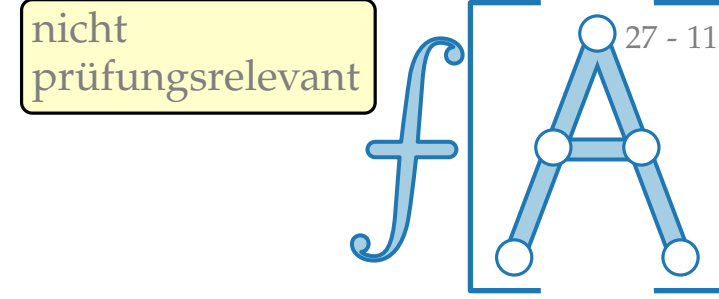
$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$

$$d_v = \max_{u: uv \in E} d_u + w(u, v)$$

3. Wert einer optimalen Lösung berechnen (hier bottom-up)

- G topologisch sortieren
- d -Werte initialisieren: $d_s = 0$ und $d_v = -\infty$ für alle $v \neq s$
- **for**-Schleife durch Knoten v.l.n.r. d -Werte berechnen

Algorithmus nach Fahrplan



1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

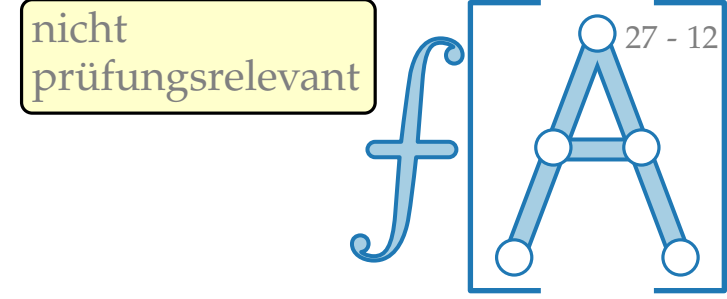
$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$

$$d_v = \max_{u: uv \in E} d_u + w(u, v) \quad \leftarrow \text{so!}$$

3. Wert einer optimalen Lösung berechnen (hier bottom-up)

- G topologisch sortieren
- d -Werte initialisieren: $d_s = 0$ und $d_v = -\infty$ für alle $v \neq s$
- **for**-Schleife durch Knoten v.l.n.r. d -Werte berechnen

Algorithmus nach Fahrplan



1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$

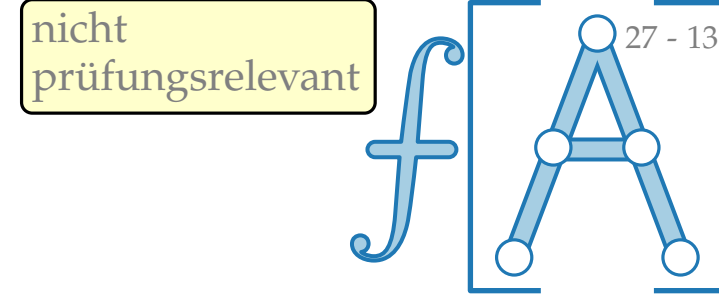
$$d_v = \max_{u: uv \in E} d_u + w(u, v) \quad \leftarrow \text{so!}$$

3. Wert einer optimalen Lösung berechnen (hier bottom-up)

- G topologisch sortieren
- d -Werte initialisieren: $d_s = 0$ und $d_v = -\infty$ für alle $v \neq s$
- **for**-Schleife durch Knoten v.l.n.r. d -Werte berechnen

Übrigens: *Kürzeste* Wege in kreisfreien Graphen

Algorithmus nach Fahrplan



1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$

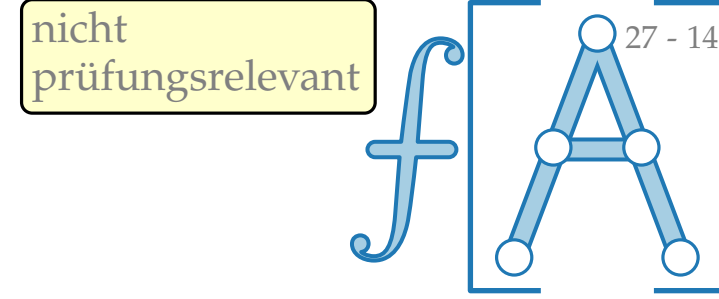
$$d_v = \max_{u: uv \in E} d_u + w(u, v) \quad \leftarrow \text{so!}$$

3. Wert einer optimalen Lösung berechnen (hier bottom-up)

- G topologisch sortieren
- d -Werte initialisieren: $d_s = 0$ und $d_v = -\infty$ für alle $v \neq s$
- **for**-Schleife durch Knoten v.l.n.r. d -Werte berechnen

Übrigens: Kürzeste Wege in kreisfreien Graphen kann man genauso berechnen (mit min statt max und $+\infty$ statt $-\infty$).

Algorithmus nach Fahrplan



1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

$$d_s = 0 \quad // \text{ Länge eines längsten } s\text{-}v\text{-Wegs}$$

$$d_v = \max_{u: uv \in E} d_u + w(u, v) \quad \leftarrow \text{so!}$$

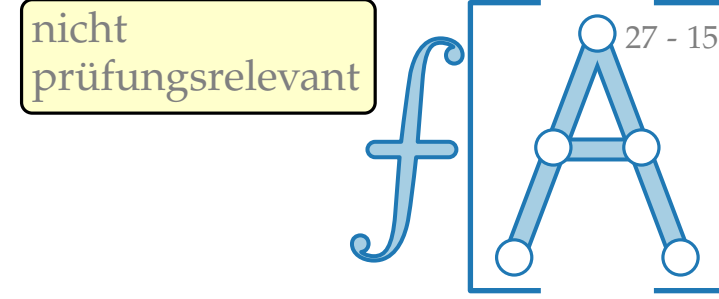
3. Wert einer optimalen Lösung berechnen (hier bottom-up)

- G topologisch sortieren
- d -Werte initialisieren: $d_s = 0$ und $d_v = -\infty$ für alle $v \neq s$
- **for**-Schleife durch Knoten v.l.n.r. d -Werte berechnen

Übrigens: *Kürzeste* Wege in kreisfreien Graphen kann man genauso berechnen (mit min statt max und $+\infty$ statt $-\infty$).

Genauso kann man auch das T9-Problem lösen ($\cdot$ statt $+$).

Algorithmus nach Fahrplan



1. Struktur einer optimalen Lösung charakterisieren ✓

2. Wert einer optimalen Lösung rekursiv definieren

$$d_s = 0$$

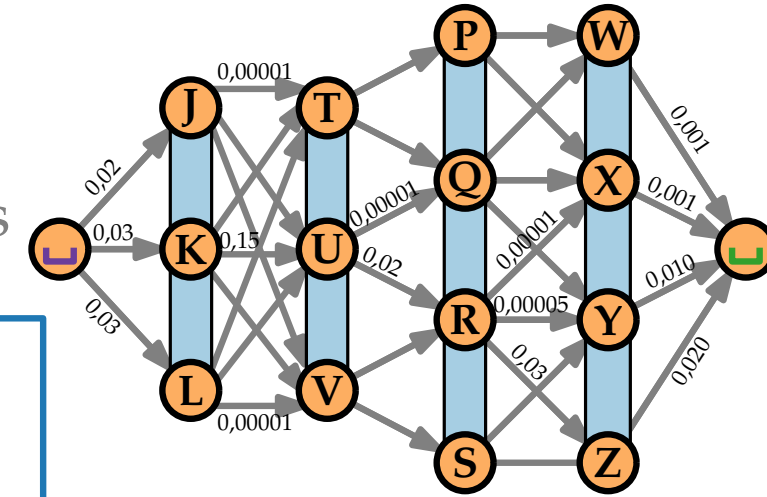
// Länge eines längsten s - v -Wegs

$$d_v = \max_{u: uv \in E} d_u + w(u, v)$$

so!

3. Wert einer optimalen Lösung berechnen (hier bottom-up)

- G topologisch sortieren
- d -Werte initialisieren: $d_s = 0$ und $d_v = -\infty$ für alle $v \neq s$
- for-Schleife durch Knoten v.l.n.r. d -Werte berechnen

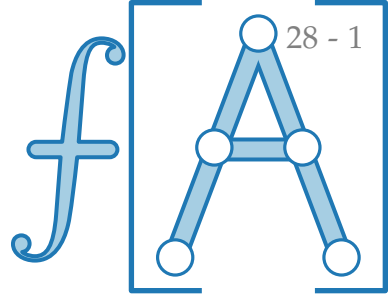


Übrigens: Kürzeste Wege in kreisfreien Graphen kann man genauso berechnen (mit min statt max und $+\infty$ statt $-\infty$).

Genauso kann man auch das T9-Problem lösen ($\cdot$ statt $+$).

Und jetzt?

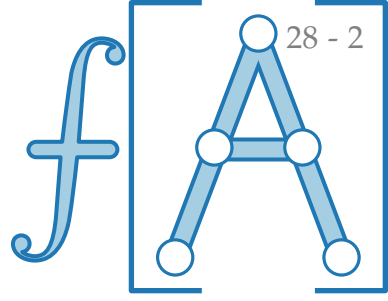
Im Buch [IA] werden weitere, praxisrelevante Probleme mit dynamischem Programmieren gelöst:



Und jetzt?

Im Buch [IA] werden weitere, praxisrelevante Probleme mit dynamischem Programmieren gelöst:

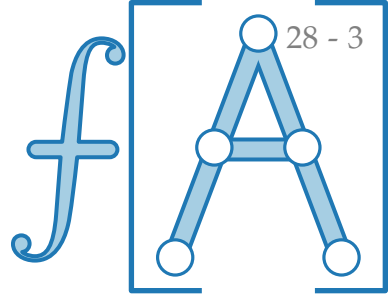
- Ketten von Matrixmultiplikationen



Und jetzt?

Im Buch [IA] werden weitere, praxisrelevante Probleme mit dynamischem Programmieren gelöst:

- Ketten von Matrixmultiplikationen
- Längste gemeinsame Teilfolge (in Zeichenketten)



Und jetzt?

Im Buch [IA] werden weitere, praxisrelevante Probleme mit dynamischem Programmieren gelöst:

- Ketten von Matrixmultiplikationen
- Längste gemeinsame Teilfolge (in Zeichenketten)
- Optimale binäre Suchbäume

